

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра Систем віртуальної реальності

ПОЯСНЮВАЛЬНА ЗАПИСКА
до бакалаврської кваліфікаційної роботи на тему:

Розробка системи вивчення англійської мови з інтерактивними функціями та
підтримкою методів машинного навчання
(Development of an English Language Learning System with Interactive Features
and Machine Learning Support)

Студент(ка) групи КН-41д Зінченко Максим Сергійович
(шифр, прізвище та ініціали)

Керівник БКР
Консультанти

_____	_____
_____	_____
_____	_____
_____	_____

▪ **Рецензент**

_____	_____
_____	_____

Завідувач кафедри _____

“ ” _____ 2026р.

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Інститут ІКНІ Кафедра СВР
Спеціальність Комп'ютерні науки

ЗАТВЕРДЖУЮ

Зав.кафедрою _____

“ ” 202

бр

З А В Д А Н Н Я

на бакалавську кваліфікаційну роботу студенту

Зінченко Максим Сергійович

(прізвище, ім'я по батькові)

1. Тема проекту(роботи) Розробка системи вивчення англійської мови з інтерактивними функціями та підтримкою методів машинного навчання

(Development of an English Language Learning System with Interactive Features and Machine Learning Support)

затверджена наказом по університету від “07” квітня 2026 р. №1324-4-06

2. Термін здачі студентом закінченого проекту(роботи) “ ” 20_ р.

3. Вихідні дані до проекту(роботи)

4. Зміст розрахунково-пояснювальної записки(перелік питань, що їх належить розробити)

Розділ 1. Аналіз технологій та існуючих програмних рішень

Розділ 2. Проектування інформаційної системи.

Розділ 3. Розробка та реалізація програмного продукту

Розділ 4. Тестування та оцінювання ефективності системи

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6. Консультанти по проекту(роботі), із зазначенням розділів проекту, що стосується їх

Розділ	Консультанти	Підпис, дата	
		Завдання видав	Завдання прийняв

7. Дата видачі завдання “ ” _____ 2026 р.

Керівник

_____ (підпис)

Завдання прийняв до виконання

_____ (підпис)

КАЛЕНДАРНИЙ ПЛАН

Пор. №	Назва етапів дипломного проекту(роботи)	Термін виконання етапів проекту	Примітка
1	Аналіз предметної області та вимог до системи	04.04.2026 – 13.04.2026	
2	Проектування архітектури та бази даних	14.04.2026 – 23.04.2026	
3	Розробка серверної частини та основних модулів	24.04.2026 – 10.05.2026	
4	Розробка клієнтської частини та ML/NLP-модулів	11.05.2026 – 20.05.2026	
5	Тестування та оцінювання системи	21.05.2026 – 27.05.2026	
6	Оформлення пояснювальної записки та підготовка до захисту дипломної роботи	28.05.2026 – 02.06.2026	

Студент-дипломник _____
(підпис)

Керівник проекту _____
(підпис)

АНОТАЦІЯ

У кваліфікаційній роботі розроблено веборієнтовану інформаційну систему для вивчення англійської мови, що поєднує засоби інтерактивного навчання, технології обробки природної мови та методи машинного навчання. Основною метою роботи є підвищення ефективності засвоєння лексичного та граматичного матеріалу шляхом персоналізації навчального процесу та автоматизації оцінювання результатів навчання.

У роботі проведено аналіз існуючих освітніх платформ і технологій, досліджено сучасні підходи до контекстного аналізу лексичних одиниць, інтервального повторення та автоматичного оцінювання письмових робіт. На основі проведеного аналізу спроектовано архітектуру програмної системи, функціональну структуру та інформаційну модель бази даних.

Розроблена система забезпечує інтерактивне читання електронних книг у форматі EPUB, контекстне визначення значення слів із використанням алгоритмів Word Sense Disambiguation, формування персонального словника користувача, реалізацію інтервального повторення за алгоритмом SM-2, автоматизоване оцінювання письмових робіт за шкалою CEFR та перевірку відповідності тексту заданій темі. Додатково реалізовано ігрові механіки для закріплення знань, модуль синтезу мовлення та підсистему аналізу навчального прогресу.

Програмний продукт реалізовано на основі фреймворку Django з використанням PostgreSQL, Redis, SentenceTransformers, spaCy, XGBoost та ONNX Runtime. Розгортання системи виконано у хмарному середовищі Amazon Web Services із застосуванням контейнеризації та підходу Infrastructure as Code.

Проведене тестування підтвердило працездатність розробленої системи та доцільність використання запропонованих архітектурних і алгоритмічних рішень для підтримки процесу вивчення англійської мови.

Структура пояснювальної записки: кваліфікаційна робота складається зі вступу, чотирьох розділів, висновків, переліку використаних джерел та додатків. Пояснювальна записка містить ____ сторінок, ____ рисунків, ____ таблиць, ____ додатків та ____ використаних джерел.

Ключові слова: інформаційна система, вивчення англійської мови, NLP, машинне навчання, Word Sense Disambiguation, CEFR, інтервальне повторення, Django, PostgreSQL, AWS.

ABSTRACT

This qualification thesis presents the development of a web-based information system for learning English, integrating interactive learning tools, natural language processing techniques, and machine learning methods. The main goal of the project is to improve the efficiency of vocabulary and grammar acquisition through personalized learning pathways and automated assessment of user performance.

The work includes an analysis of existing educational platforms and technologies, as well as modern approaches to lexical processing, spaced repetition learning, and automated essay evaluation. Based on this analysis, the system architecture, functional structure, and database model were designed.

The developed system provides interactive reading of EPUB books, context-based word sense disambiguation using NLP techniques, a personal vocabulary management module, and an implementation of the SM-2 spaced repetition algorithm. It also includes automated essay evaluation based on CEFR level classification and topic relevance analysis. Additional features include gamified learning modules, speech synthesis, and learning progress analytics.

The software system is implemented using the Django framework with PostgreSQL as the database management system. Machine learning and NLP components are built using SentenceTransformers, spaCy, XGBoost, and ONNX Runtime. The system is deployed in the Amazon Web Services cloud environment using containerization and Infrastructure as Code principles.

The results of testing confirm the correctness of the implemented solution and demonstrate the effectiveness of the proposed architectural and algorithmic approaches for supporting English language learning.

Structure of the explanatory note: the qualification thesis consists of an introduction, four chapters, conclusions, references, and appendices. The explanatory note contains ____ pages, ____ figures, ____ tables, ____ appendices, and ____ references.

Keywords: information system, English learning, NLP, machine learning, Word Sense Disambiguation, CEFR, spaced repetition, Django, PostgreSQL, AWS.

ЗМІСТ

ВСТУП.....	9
РОЗДІЛ 1. АНАЛІЗ ТЕХНОЛОГІЙ ТА ІСНУЮЧИХ ПРОГРАМНИХ РІШЕНЬ	
12	
1.1. Основні принципи побудови систем вивчення іноземних мов.....	12
1.2. Огляд сучасних платформ для вивчення англійської мови.....	13
1.3. Технології машинного навчання та обробки природної мови в освітніх системах.....	15
1.4. Алгоритми персоналізованого навчання та системи інтервального повторення.....	19
1.5. Методи оцінювання мовних навичок та застосування машинного навчання.....	21
Висновки до розділу 1.....	22
РОЗДІЛ 2. ПРОЄКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ.....	24
2.1. Мета та функціональні вимоги до системи.....	24
2.2. Загальна архітектура програмної системи.....	31
2.3. Проєктування функціональної структури системи.....	36
2.4. Проєктування інформаційної моделі та бази даних.....	37
2.5. Обґрунтування вибору технологій та інструментів.....	40
Висновок до розділу 2.....	41
РОЗДІЛ 3. РОЗРОБКА ТА РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ.....	43
3.1. Загальна характеристика реалізованої системи.....	43
3.2. Реалізація серверної частини системи.....	43
3.3. Реалізація інтелектуальних алгоритмів системи.....	48
3.4. Реалізація користувацького інтерфейсу.....	69
3.5. Розгортання системи у хмарному середовищі AWS.....	80
Висновок до розділу 3.....	91
РОЗДІЛ 4. ТЕСТУВАННЯ ТА ОЦІНЮВАННЯ ЕФЕКТИВНОСТІ СИСТЕМИ	
93	
4.1. Загальна характеристика готового програмного продукту перед тестуванням.....	93
4.2. Підхід до тестування та тестове середовище.....	93
4.3. Функціональне тестування системи.....	95
4.4. Інтеграційне тестування системи.....	97
4.5. API тестування системи.....	99
4.6. Безпека та стабільність системи.....	100
Висновок до розділу 4.....	102
ВИСНОВКИ.....	104
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	105

ВСТУП

Володіння англійською мовою є однією з ключових компетентностей у сучасному освітньому, професійному та науковому середовищі. Умови глобалізації та стрімке зростання обсягів цифрової інформації зумовлюють необхідність використання ефективних, адаптивних та інтерактивних засобів навчання. У цьому контексті особливої актуальності набувають веборієнтовані освітні системи, що забезпечують доступність навчального процесу та його персоналізацію.

Існуючі платформи для вивчення іноземних мов здебільшого орієнтовані на окремі аспекти навчання, такі як тренування лексики, граматичні вправи або тестування знань. При цьому вони часто не забезпечують глибокого аналізу навчального контенту, контекстного розуміння лексичних одиниць та автоматизованого оцінювання продуктивних навичок, зокрема письма. Також недостатньо розвиненими залишаються механізми адаптації навчального процесу до індивідуального рівня користувача.

Перспективним напрямом розвитку таких систем є застосування методів обробки природної мови (NLP) та машинного навчання. Їх використання дозволяє реалізувати контекстне визначення значення слів, автоматичне оцінювання текстів, класифікацію рівня володіння мовою та побудову адаптивних алгоритмів навчання. Додатково важливу роль відіграють алгоритми інтервального повторення, які підвищують ефективність довготривалого запам'ятовування лексичних одиниць.

Актуальність теми бакалаврської кваліфікаційної роботи полягає у необхідності створення веборієнтованої інформаційної системи для вивчення англійської мови, яка поєднує інтерактивне навчання, автоматизований аналіз тексту та інтелектуальні методи обробки мовних даних у межах єдиної платформи.

Метою роботи є розробка веборієнтованої інформаційної системи для вивчення англійської мови, що забезпечує інтерактивну роботу з текстами, управління словниковим запасом, автоматизоване оцінювання письмових робіт, контекстний аналіз лексичних одиниць та підтримку ігрових навчальних механізмів.

Для досягнення поставленої мети необхідно виконати такі завдання:

- проаналізувати існуючі рішення у сфері цифрового вивчення іноземних мов;
- дослідити методи NLP та машинного навчання, що застосовуються в освітніх системах;
- сформулювати функціональні та нефункціональні вимоги до системи;
- спроектувати архітектуру веборієнтованої системи та бази даних;
- реалізувати серверну та клієнтську частини програмного продукту;
- інтегрувати модулі контекстного аналізу, оцінювання письма та роботи зі словником;
- реалізувати механізми інтервального повторення та гейміфікації навчання;
- забезпечити розгортання системи у хмарному середовищі AWS;
- провести тестування та оцінити ефективність розробленої системи.

Об'єктом дослідження є процес вивчення англійської мови з використанням веборієнтованих інформаційних систем.

Предметом дослідження є методи та програмні засоби створення системи вивчення англійської мови з використанням NLP та машинного навчання.

У роботі використано методи системного аналізу, проєктування програмних систем, обробки природної мови, машинного навчання та веброзробки.

Практичне значення роботи полягає у створенні повнофункціональної веборієнтованої системи, яка може використовуватися для самостійного та

адаптивного вивчення англійської мови. Система підтримує роботу з електронними текстами, контекстний аналіз слів, автоматизоване оцінювання письмових робіт, інтервальне повторення лексики та навчальні ігрові сценарії.

Особливістю розробленої системи є інтеграція методів машинного навчання та обробки природної мови у всі ключові навчальні процеси, що дозволяє забезпечити персоналізоване, адаптивне та більш ефективне навчання в межах єдиної вебплатформи.

РОЗДІЛ 1. АНАЛІЗ ТЕХНОЛОГІЙ ТА ІСНУЮЧИХ ПРОГРАМНИХ РІШЕНЬ

1.1. Основні принципи побудови систем вивчення іноземних мов

Сучасні системи вивчення іноземних мов є складними програмними продуктами, які поєднують у собі елементи інформаційних систем, освітніх платформ та інтерактивних навчальних середовищ. Їх основною метою є забезпечення ефективного засвоєння мовного матеріалу шляхом поєднання різних типів навчальної діяльності, таких як вивчення лексики, виконання граматичних вправ, розвиток навичок читання та письма, а також тренування сприйняття мови на слух.

Базовим принципом побудови таких систем є інтерактивність навчального процесу, яка передбачає активну взаємодію користувача із навчальним контентом. На відміну від традиційних методів навчання, цифрові платформи дозволяють реалізувати динамічну подачу матеріалу, адаптацію складності завдань та миттєвий зворотний зв'язок, що підвищує ефективність засвоєння інформації.

Іншим важливим принципом є персоналізація навчання. Сучасні системи намагаються враховувати індивідуальні особливості користувача, такі як рівень знань, швидкість засвоєння матеріалу та типові помилки. Це дозволяє формувати індивідуальні навчальні траєкторії, які підвищують ефективність навчального процесу та зменшують когнітивне навантаження.

Окрему роль у побудові таких систем відіграє принцип багатокomпонентності, відповідно до якого навчальна платформа включає декілька функціональних модулів. Зазвичай це модулі роботи з лексикою, граматичні тренажери, системи тестування, інструменти для читання текстів та компоненти оцінювання мовних навичок. Така структура дозволяє охопити різні аспекти мовної компетентності в межах єдиної системи.

Також важливим є принцип адаптивності, який полягає у зміні складності навчальних завдань залежно від результатів користувача.

Адаптивні системи використовують статистичні методи або алгоритми машинного навчання для аналізу успішності виконання завдань та подальшого коригування навчального процесу.

Сучасні підходи також передбачають використання елементів гейміфікації, які включають систему балів, рівнів, досягнень та інших ігрових механік. Це сприяє підвищенню мотивації користувачів та збільшенню залученості до навчального процесу.

Таким чином, системи вивчення іноземних мов базуються на поєднанні інтерактивності, персоналізації, адаптивності та модульної архітектури, що забезпечує комплексний підхід до організації навчального процесу та підвищує його ефективність.

1.2. Огляд сучасних платформ для вивчення англійської мови

Сучасний ринок цифрових рішень для вивчення іноземних мов представлений великою кількістю веб та мобільних платформ, які забезпечують користувачам доступ до навчальних матеріалів у інтерактивному форматі. Такі системи спрямовані на розвиток лексичних, граматичних та комунікативних навичок і активно використовуються як для самостійного навчання, так і як допоміжні інструменти в освітньому процесі.

Однією з найпоширеніших платформ є Duolingo, яка базується на гейміфікованому підході до навчання. Система пропонує користувачам короткі інтерактивні вправи, що охоплюють переклад, аудіювання та базову граматику. Основною перевагою даної платформи є висока залученість користувачів завдяки ігровим механікам, системі рівнів та щоденним цілям. Водночас суттєвим недоліком є обмежена глибина опрацювання мовного матеріалу, а також недостатній контекстний аналіз лексики, що ускладнює формування природного мовного середовища.

Іншим популярним інструментом є Anki, який реалізує систему інтервального повторення (Spaced Repetition System). Дана платформа дозволяє користувачам створювати власні набори карток для

запам'ятовування слів та виразів. Основною перевагою Anki є висока ефективність запам'ятовування завдяки математично обґрунтованому алгоритму повторення. Однак система має низький рівень інтерактивності та не забезпечує контекстного навчання, оскільки робота з лексикою відбувається переважно у вигляді ізольованих карток.

Платформа Quizlet також використовує підхід карток для навчання, але доповнює його ігровими режимами та тестовими завданнями. Quizlet є зручною для швидкого запам'ятовування термінів і базової лексики, однак, подібно до Anki, не забезпечує глибокого контекстного аналізу мовних одиниць та персоналізації навчального процесу на рівні змісту текстів.

Окрему категорію становлять контекстні словникові сервіси, зокрема Reverso Context. Дана система надає переклади слів і виразів у реальних контекстах, використовуючи приклади з корпусів текстів. Основною перевагою є можливість бачити слово у природному мовному середовищі, що підвищує якість розуміння значення. Разом з тим, система обмежується функцією перекладу та не забезпечує повноцінного навчального циклу або адаптивного контролю знань користувача.

Також слід відзначити платформу Babbel, яка орієнтована на структуроване навчання мов з акцентом на граматику та практичні діалоги. Babbel забезпечує більш академічний підхід до подачі матеріалу порівняно з гейміфікованими системами, однак має обмежену гнучкість у персоналізації навчального процесу та не використовує глибокі методи машинного навчання для аналізу мовної поведінки користувача.

Узагальнюючи, можна зазначити, що розглянуті платформи мають низку переваг у своїх окремих напрямках, однак кожна з них вирішує лише частину задачі навчання іноземної мови. Вони або фокусуються на запам'ятовуванні лексики, або на контекстних перекладах, або на граматичних вправах, але не забезпечують комплексного підходу, який би поєднував інтерактивне читання, контекстний аналіз, автоматизовану оцінку

письма та адаптивне навчання в межах єдиної системи. Саме ця проблема обґрунтовує необхідність розробки інтегрованої веборієнтованої системи, що поєднує різні підходи до навчання.

1.3. Технології машинного навчання та обробки природної мови в освітніх системах

Сучасні інформаційні системи для вивчення іноземних мов дедалі частіше базуються на застосуванні методів машинного навчання (Machine Learning) та обробки природної мови (Natural Language Processing), що дозволяє переходити від статичних навчальних матеріалів до інтелектуальних адаптивних систем. Такі підходи забезпечують можливість аналізу текстових даних на різних рівнях - від морфологічного до семантичного - та створюють основу для реалізації персоналізованого навчального процесу.

Загалом задачі NLP у навчальних системах полягають у перетворенні неструктурованого тексту у формалізоване подання, придатне для подальшої обробки алгоритмами машинного навчання. Це включає етапи токєнізації, лематизації, визначення частин мови (Part-of-Speech tagging), синтаксичного аналізу та побудови семантичних представлень тексту. Кожен із цих етапів виконує окрему роль у формуванні структурованого представлення мовного матеріалу, яке може бути використане для оцінювання, класифікації або порівняння текстів.

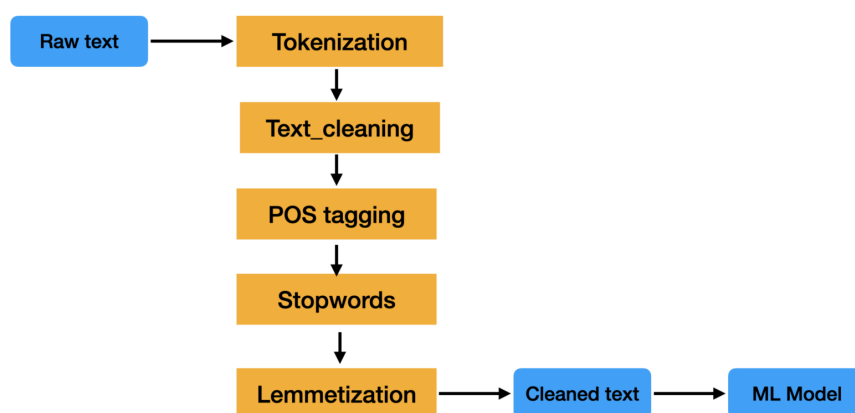


Рисунок 1.1 – Приклад обробки природної мови в освітній системі

Одним із ключових елементів сучасних NLP-систем є використання векторних представлень слів і речень (word embeddings, sentence embeddings). Їх основна ідея полягає у відображенні лексичних одиниць у багатовимірному векторному просторі таким чином, щоб семантично близькі слова мали близькі векторні представлення. Це дозволяє перейти від символічного представлення тексту до числового, що є необхідною умовою для застосування алгоритмів машинного навчання.

У практичних системах часто використовуються попередньо навчені моделі, такі як Sentence Transformers, які дозволяють отримувати контекстуальні ембедінги не лише для окремих слів, але й для цілих речень. Це особливо важливо для освітніх систем, де значення слова суттєво залежить від контексту використання.

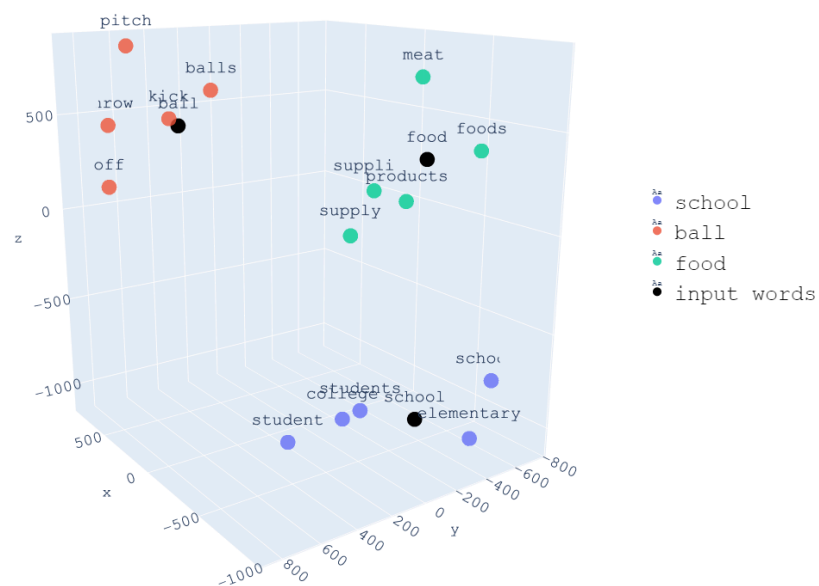


Рисунок 1.2 – Семантичний простір векторних представлень слів

Використання embedding-представлень дозволяє реалізувати обчислення семантичної схожості між текстовими фрагментами. Найчастіше для цього застосовується косинусна подібність, яка визначає кут між векторами у багатовимірному просторі. Чим менший кут між векторами, тим

більш семантично схожими є відповідні текстові одиниці. У контексті освітніх систем це дозволяє порівнювати речення, визначати релевантність прикладів або оцінювати відповідність відповіді користувача заданому контексту.

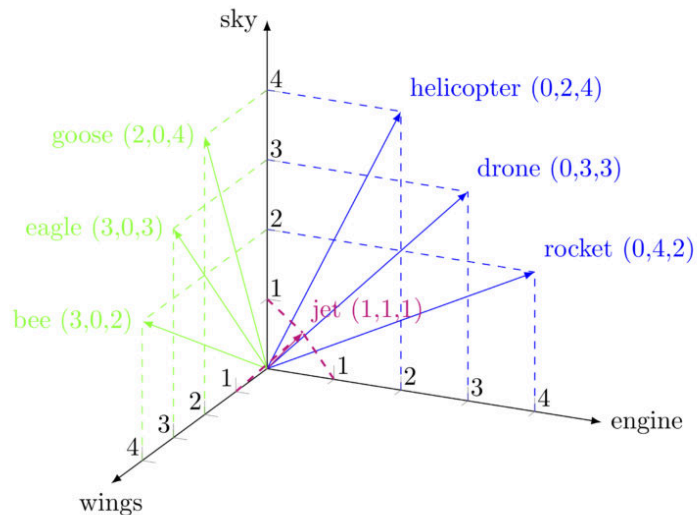


Рисунок 1.3 – Обчислення семантичної подібності між словами

Окремим важливим завданням є визначення значення слова в контексті (Word Sense Disambiguation, WSD). У природній мові багато слів є багатозначними, тому їх правильна інтерпретація залежить від контексту. Наприклад, слово “bank” може означати як фінансову установу, так і берег річки. У навчальних системах це є критично важливим, оскільки неправильне трактування значення слова може призвести до помилкового засвоєння матеріалу.

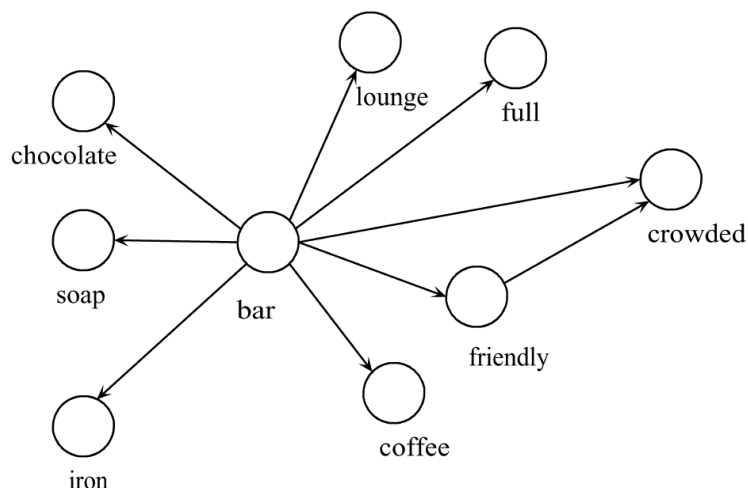


Рисунок 1.4 – Граф контекстних зв'язків багатозначного слова «bar» для задачі Word Sense Disambiguation

Ще одним важливим напрямом є використання алгоритмів машинного навчання для класифікації текстових даних. У навчальних системах це може включати визначення рівня володіння мовою (наприклад, CEFR A1–C2), оцінювання якості письмових робіт або виявлення типових помилок користувача. Одним із ефективних підходів у таких задачах є використання градієнтного бустингу, зокрема алгоритму XGBoost, який демонструє високу точність при роботі зі структурованими ознаками.

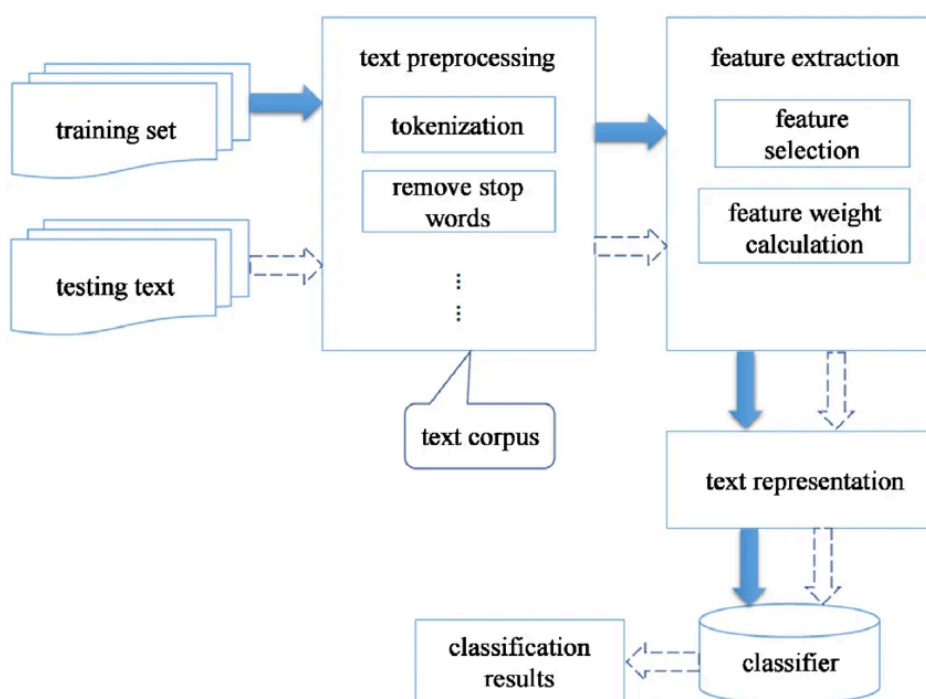


Рисунок 1.5 – Приклад реалізації процесу класифікації текстових даних за допомогою ML-конвеєра

Для побудови моделі текст спочатку перетворюється у набір ознак, що включає лексичні характеристики, синтаксичні параметри, показники складності речень та статистичні метрики. Далі ці ознаки подаються на вхід моделі, яка навчається визначати відповідний клас або числову оцінку.

Таким чином, застосування методів машинного навчання та обробки природної мови в освітніх системах дозволяє суттєво підвищити їх

функціональність. Зокрема, забезпечується можливість аналізу тексту на різних рівнях, врахування контексту при визначенні значення слів, автоматизоване оцінювання мовних навичок користувача та побудова адаптивних навчальних сценаріїв. Це переводить освітні системи з рівня статичних тренажерів у категорію інтелектуальних платформ, здатних підлаштовуватися під потреби користувача.

1.4. Алгоритми персоналізованого навчання та системи інтервального повторення

Алгоритми персоналізованого навчання та системи інтервального повторення є одним із ключових підходів, що використовуються в сучасних освітніх інформаційних системах для підвищення ефективності запам'ятовування навчального матеріалу. Їх основна ідея полягає в тому, щоб організувати процес повторення таким чином, щоб інформація повторювалась у моменти, коли користувач найбільш схильний її забути. Це дозволяє суттєво зменшити втрати знань у довготривалій пам'яті та оптимізувати навчальне навантаження.

Теоретичною основою таких систем є модель «кривої забування», яка описує зміну рівня запам'ятовування інформації з плином часу. У загальному випадку вона має експоненційний характер: без повторення обсяг збереженої інформації поступово зменшується, причому найбільш інтенсивне забування відбувається одразу після первинного засвоєння матеріалу. Саме ця властивість пам'яті людини є базовим обґрунтуванням необхідності регулярного повторення навчального контенту.

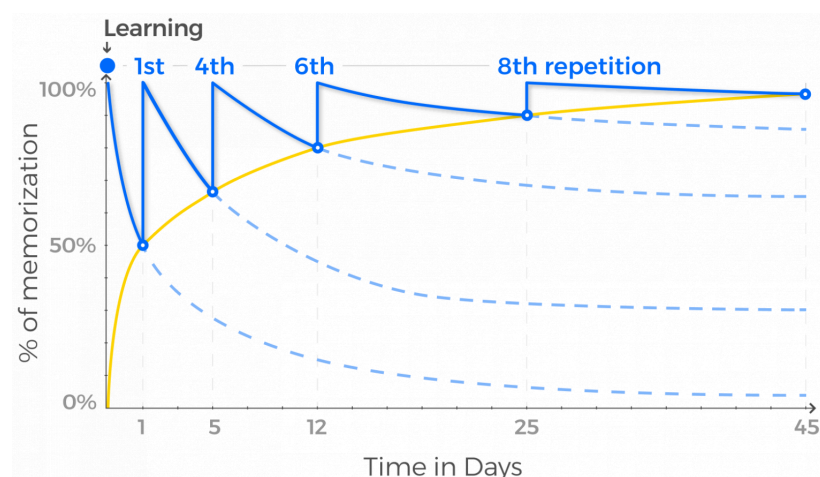


Рисунок 1.6 – Графік кривої забування Еббінгауза та вплив інтервальних повторень на збереження інформації у довготривалій пам'яті

У спрощеному вигляді залежність рівня запам'ятовування від часу може бути подана експоненційною функцією, де зі збільшенням часу після вивчення матеріалу ймовірність його відтворення зменшується. Практичне значення цієї моделі полягає в тому, що вона дозволяє формувати оптимальні інтервали повторення, орієнтуючись не на фіксований графік, а на стан засвоєння конкретного користувача.

На основі цієї концепції були розроблені алгоритми інтервального повторення, які адаптують навчальний процес під індивідуальні особливості пам'яті користувача. Найбільш відомим класичним підходом є алгоритм SM-2, який використовується для динамічного визначення моментів повторення навчального матеріалу. Його робота базується на оцінюванні якості відповіді користувача після кожного повторення та подальшому коригуванні інтервалу наступного перегляду.

У межах цього підходу кожна відповідь користувача оцінюється за шкалою, яка відображає ступінь засвоєння матеріалу. Якщо користувач демонструє високий рівень відтворення, інтервал між повтореннями збільшується, що зменшує частоту показу вже добре засвоєних одиниць. У випадку помилок або низької якості відповіді інтервал скорочується, що дозволяє швидше повернутися до проблемного матеріалу.

Важливою складовою алгоритму є коефіцієнт легкості запам'ятовування, який динамічно оновлюється в процесі навчання. Він відображає, наскільки легко користувач засвоює конкретну одиницю інформації, і впливає на швидкість зростання інтервалів повторення. Таким чином, система поступово «підлаштовується» під індивідуальні особливості пам'яті користувача.

Окрім базового механізму SM-2, сучасні системи інтервального повторення можуть враховувати додаткові фактори, такі як складність

навчального матеріалу, частота появи лексичних одиниць у реальному мовному середовищі та загальний рівень підготовки користувача. Це дозволяє більш точно моделювати процес забування та підвищувати ефективність навчання.

Окремо слід відзначити підхід дострокового повторення, коли користувач сам ініціює повторення матеріалу до запланованого моменту. У таких випадках система може використовувати результати такого повторення як додатковий сигнал для корекції моделі пам'яті, однак у більшості реалізацій вони мають менший вплив на основний графік навчання, щоб не спотворювати статистичну модель засвоєння.

Узагальнюючи, системи інтервального повторення є важливим інструментом персоналізації навчального процесу, оскільки вони дозволяють адаптувати частоту повторення матеріалу до реального стану знань користувача. Використання таких підходів у освітніх платформах значно підвищує ефективність довготривалого запам'ятовування та забезпечує більш раціональне використання часу навчання.

1.5. Методи оцінювання мовних навичок та застосування машинного навчання

Оцінювання рівня володіння іноземною мовою є важливою складовою сучасних освітніх систем, оскільки дозволяє визначати прогрес користувача та формувати персоналізовані навчальні траєкторії. Найбільш поширеним підходом до стандартизації мовних компетенцій є система CEFR (Common European Framework of Reference for Languages), яка поділяє рівні володіння мовою на діапазон від A1 до C2.

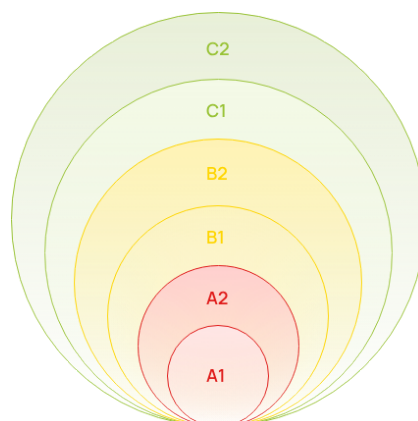


Рисунок 1.7 – Схема вкладеності рівнів володіння мовою відповідно до загальноєвропейських рекомендацій CEFR

У традиційних освітніх підходах оцінювання здійснюється викладачем на основі тестових завдань або письмових робіт. У цифрових системах цей процес може бути автоматизований за допомогою методів машинного навчання та обробки природної мови. У такому випадку текст користувача аналізується за набором лінгвістичних ознак, які включають складність речень, граматичну правильність, лексичне різноманіття та семантичну узгодженість.

На основі цих ознак можуть застосовуватись алгоритми класифікації, які відносять текст до певного рівня CEFR або прогнозують числову оцінку мовної компетенції користувача. У практичних реалізаціях часто використовуються методи градієнтного бустингу або нейронні мережі, які демонструють високу точність при роботі з текстовими даними.

Таким чином, поєднання стандартів CEFR із методами машинного навчання дозволяє створювати автоматизовані системи оцінювання, які забезпечують об'єктивність, масштабованість та можливість інтеграції в адаптивні освітні платформи.

Висновки до розділу 1

У даному розділі було проведено аналітичний огляд предметної області, пов'язаної з розробкою сучасних систем вивчення іноземних мов, а також розглянуто основні технології, методи та алгоритми, що застосовуються у подібних програмних рішеннях.

Проаналізовано ключові принципи побудови освітніх систем, серед яких інтерактивність, персоналізація, адаптивність, модульність та використання елементів гейміфікації. Показано, що саме поєднання цих підходів забезпечує підвищення ефективності навчального процесу та покращує залученість користувачів.

У межах огляду існуючих програмних платформ (Duolingo, Anki, Quizlet, Reverso Context, Babbel) встановлено, що кожна з них вирішує лише окремі задачі мовного навчання. Зокрема, одні системи орієнтовані на запам'ятовування лексики, інші - на контекстний переклад або структуроване навчання, однак жодна з них не забезпечує повноцінного комплексного підходу до розвитку всіх мовних навичок у межах єдиної адаптивної системи.

Окремо розглянуто методи машинного навчання та обробки природної мови, які є основою сучасних інтелектуальних освітніх систем. Показано, що використання токенізації, лематизації, векторних представлень тексту (word embeddings, sentence embeddings), а також моделей семантичної подібності дозволяє здійснювати глибокий аналіз мовних даних та реалізовувати автоматизоване оцінювання навчального контенту.

Додатково проаналізовано системи інтервального повторення, які базуються на когнітивній моделі забування інформації. Встановлено, що використання експоненційної моделі та алгоритмів типу SM-2 дозволяє адаптивно керувати інтервалами повторення та значно підвищує ефективність довготривалого запам'ятовування навчального матеріалу.

Також розглянуто підходи до автоматизованого оцінювання рівня мовних навичок із використанням стандарту CEFR та методів машинного навчання, що дозволяє формалізувати процес визначення рівня володіння мовою та інтегрувати його в адаптивні навчальні системи.

Узагальнюючи, можна зробити висновок, що сучасні системи вивчення іноземних мов базуються на поєднанні методів штучного інтелекту, обробки природної мови та когнітивних моделей пам'яті. Водночас існуючі програмні рішення не забезпечують повної інтеграції цих підходів, що підтверджує актуальність розробки комплексної веборієнтованої системи, здатної адаптуватися до індивідуальних потреб користувача та забезпечувати ефективний процес навчання.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

2.1. Мета та функціональні вимоги до системи

Метою розробки є створення веборієнтованої навчальної системи для вивчення англійської мови, що поєднує інтерактивну роботу з текстами, персоналізоване навчання та інтелектуальні методи аналізу мовних даних.

Система орієнтована на підвищення ефективності засвоєння матеріалу за рахунок адаптації навчального процесу до рівня користувача, а також використання методів обробки природної мови та машинного навчання для аналізу текстів і автоматизації перевірки навчальних результатів.

У межах реалізації система забезпечує підтримку повного навчального циклу користувача - від роботи з навчальними матеріалами до оцінювання знань та аналізу прогресу.

Основні завдання

Для досягнення мети система повинна забезпечувати:

- інтерактивну роботу з навчальними текстами у форматі EPUB;
- контекстний аналіз лексичних одиниць;
- формування та підтримку персонального словника;
- інтервальне повторення навчального матеріалу;
- автоматизоване оцінювання письмових робіт;
- ігрові механіки для закріплення знань;
- збір та аналіз навчальної статистики;
- синтез мовлення для аудіопідтримки;

Функціональні вимоги

Функціональні вимоги визначають набір можливостей, які повинна реалізовувати система для забезпечення повного навчального циклу користувача.

Таблиця 2.1

Функціональні підсистеми та їх призначення

№	Назва підсистеми	Опис функціональності
F-1	Автентифікація та користувачі	Реєстрація, авторизація (логін/email), зберігання профілю та навчальних параметрів
F-2	Інтерактивне читання	Робота з EPUB, навігація, виділення слів, збереження прогресу
F-3	Контекстний аналіз слів	Визначення значення слова в контексті на основі embeddings та cosine similarity
F-4	Персональний словник	Автоматичне формування словника з контекстом, транскрипцією та прикладами
F-5	Інтервальне повторення	Реалізація SRS-алгоритму для довготривалого запам'ятовування
F-6	Оцінювання письмових робіт	Автоматична перевірка есе, визначення рівня CEFR та аналіз відповідності темі
F-7	Гейміфікація навчання	Інтерактивні ігри для закріплення лексики та граматики
F-8	Аналітика прогресу	Збір статистики навчання: активність, слова, результати ігор
F-9	Аудіопідтримка	Синтез мовлення для розвитку аудіювання та вимови

Нефункціональні вимоги

Нефункціональні вимоги визначають якісні характеристики програмної системи, що впливають на ефективність її функціонування, стабільність та зручність використання.

- **Продуктивність:** обробка запитів контекстного аналізу з мінімальною затримкою з використанням кешування та попередніх обчислень.

- **Безпека:** захист даних користувачів, хешування паролів, CSRF-захист та контроль доступу після автентифікації.
- **Масштабованість:** підтримка горизонтального масштабування за рахунок stateless-архітектури та контейнеризації.
- **Надійність:** стабільна робота під навантаженням і коректна обробка помилок без втрати даних.
- **Зручність використання:** адаптивний інтерфейс для різних пристроїв.
- **Автономність:** виконання ключових ML-алгоритмів локально без залежності від зовнішніх API.

Діаграми прецедентів

Діаграми прецедентів UML моделюють функціональні вимоги системи, відображаючи взаємодію між акторами та сценаріями використання. У цій роботі вони застосовані для формалізації основних варіантів взаємодії користувача з платформою та визначення меж системи.

№1 Читання та робота з граматичним контентом

Функціональні можливості системи, пов'язані з опрацюванням навчальних матеріалів, відображено на діаграмі прецедентів (рис. 2.1). Вона деталізує взаємодію користувача з платформою, зокрема процеси читання електронних книг, роботи з лексикою та вивчення граматики.

Цей функціональний блок не визначає архітектуру системи, однак доповнює загальну модель, розкриваючи логіку навчальних сценаріїв та інтеграцію окремих інструментів у єдину освітню платформу.

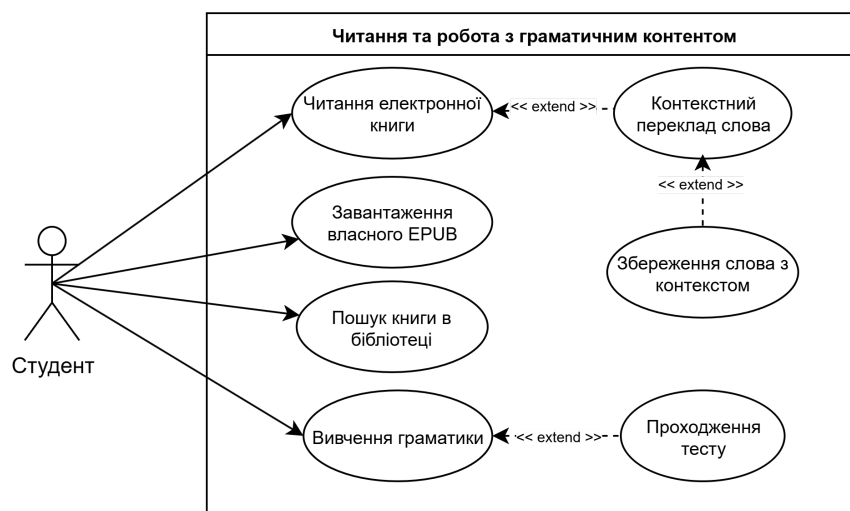


Рисунок 2.1 – Діаграма прецедентів «Читання та робота з граматичним КОНТЕНТОМ»

№2 Інтервальне повторення та керування словником

Процеси організації регулярного повторення вивченого матеріалу та менеджменту лексичної бази користувача ілюструє наступна діаграма прецедентів (див. рис. 2.2). На ній унаочнено алгоритми взаємодії з персональним словником, зокрема механізми ручного додавання нових слів та запуску інтерактивних тренувальних сесій.

Цей компонент розкриває важливу підсистему автоматизації навчання, що базується на методиці інтервальних повторень (Spaced Repetition). Наведена діаграма візуалізує архітектурне розширення платформи, демонструючи, як інтеграція допоміжних тренажерів забезпечує циклічність та безперервність процесу засвоєння мовного матеріалу.

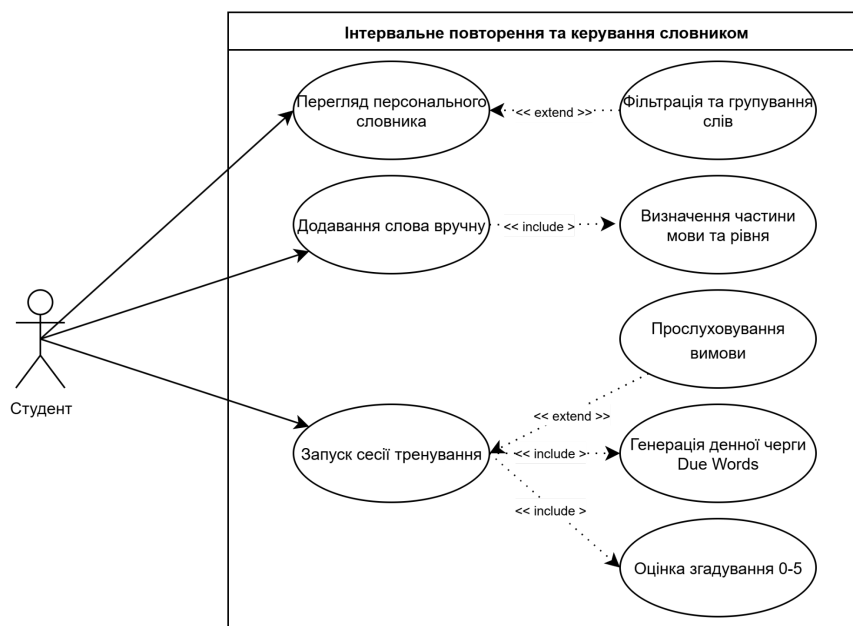


Рисунок 2.2 – Діаграма прецедентів «Інтервальне повторення та керування словником»

№3 Інтерактивні форми контролю знань та аналітика успішності

Аналіз функціональних можливостей системи, що відповідають за оцінювання знань користувача та збір аналітичних даних, представлено на

діаграмі прецедентів (див. рис. 2.3). Модель демонструє алгоритми перевірки прогресу студента через різнопланову активність: створення індивідуальних тестів, участь у мовних іграх та безпосереднє написання текстових робіт.

Важливою особливістю цієї підсистеми є автоматизація оцінювання та логування результатів. Зокрема, виконання письмових завдань передбачає обов'язкове залучення допоміжних аналітичних модулів для прогнозування поточного рівня знань за шкалою CEFR та перевірки відповідності тексту заданій темі. Наведена модель унаочнює, як контрольні-діагностичні інструменти інтегруються у загальний контур управління навчальним процесом, забезпечуючи зворотний зв'язок для користувача.

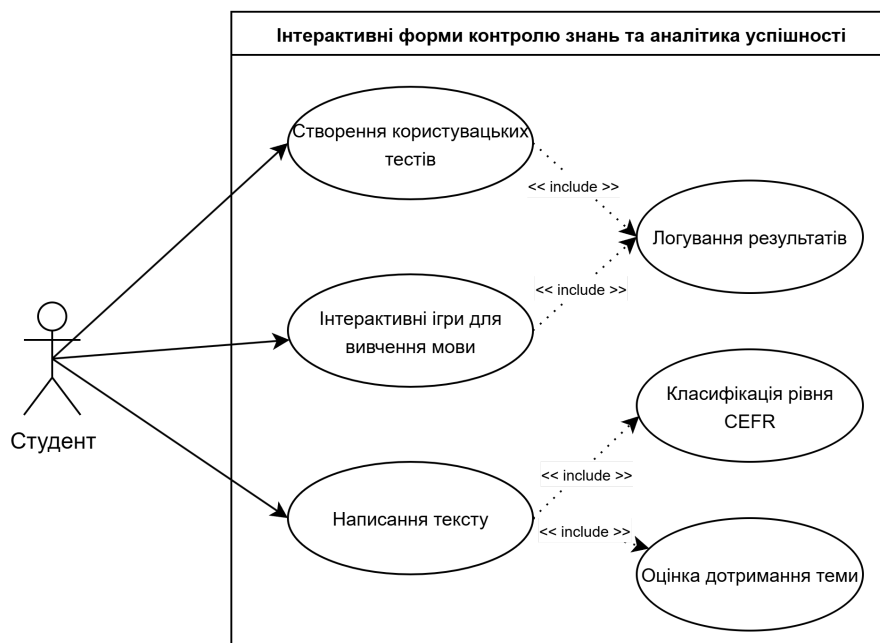


Рисунок 2.3 – Діаграма прецедентів «Інтерактивні форми контролю знань та аналітика успішності»

№4 Адміністрування та керування профілем

Питання безпеки особистих даних, поділ прав доступу та інструменти розширення навчального наповнення платформи розкрито на діаграмі прецедентів (див. рис. 2.4). Наведена схема окреслює межі повноважень Студента та Адміністратора, показуючи, які саме завдання виконує кожен із них задля забезпечення життєдіяльності системи.

Особлива увага тут приділена розмежуванню обов'язків між відвідувачем та керівником. Звичайний учень зосереджений на початковому вході до системи, зміні власних відомостей та перегляді наочних звітів про здобуті знання. Натомість керівні дії передбачають наповнення навчального середовища, що полягає у формуванні нових мовних занять та додаванні свіжих зразків мовлення до спільної бази. Така побудова наочно ілюструє взаємодію внутрішньої технічної частини з користувацьким середовищем, що робить систему стійкою та зручною в управлінні.

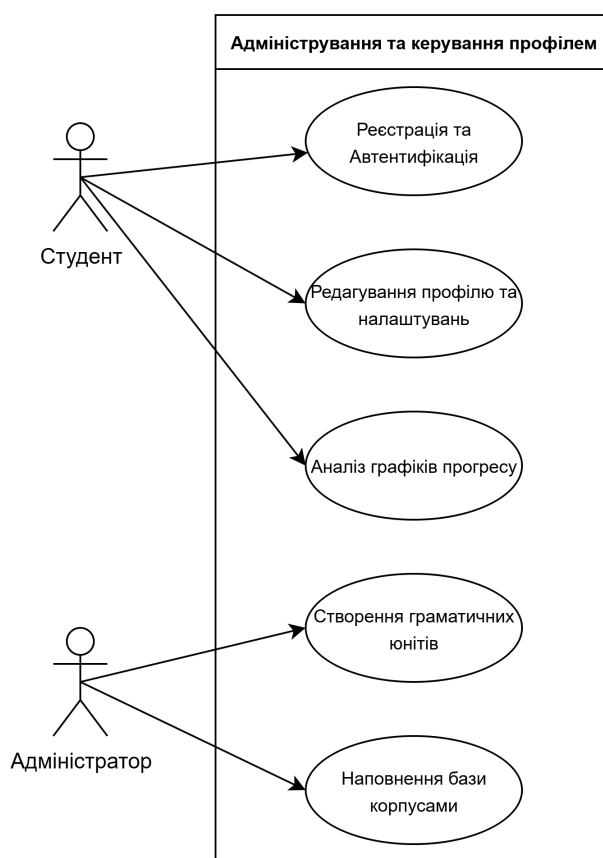


Рисунок 2.4 – Діаграма прецедентів «Адміністрування та керування профілем»

Сценарії взаємодії користувача (User Flow)

Сценарії взаємодії користувача з системою відображають послідовність виконання основних дій у межах веборієнтованої освітньої платформи та дозволяють формалізувати логіку користувацьких процесів залежно від обраної навчальної активності. Загальний сценарій роботи користувача

наведено на рисунку 2.3. Як видно з діаграми, взаємодія починається з етапу авторизації, після чого користувач переходить до головного дашборду, який є центральною точкою навігації та забезпечує доступ до всіх функціональних модулів системи

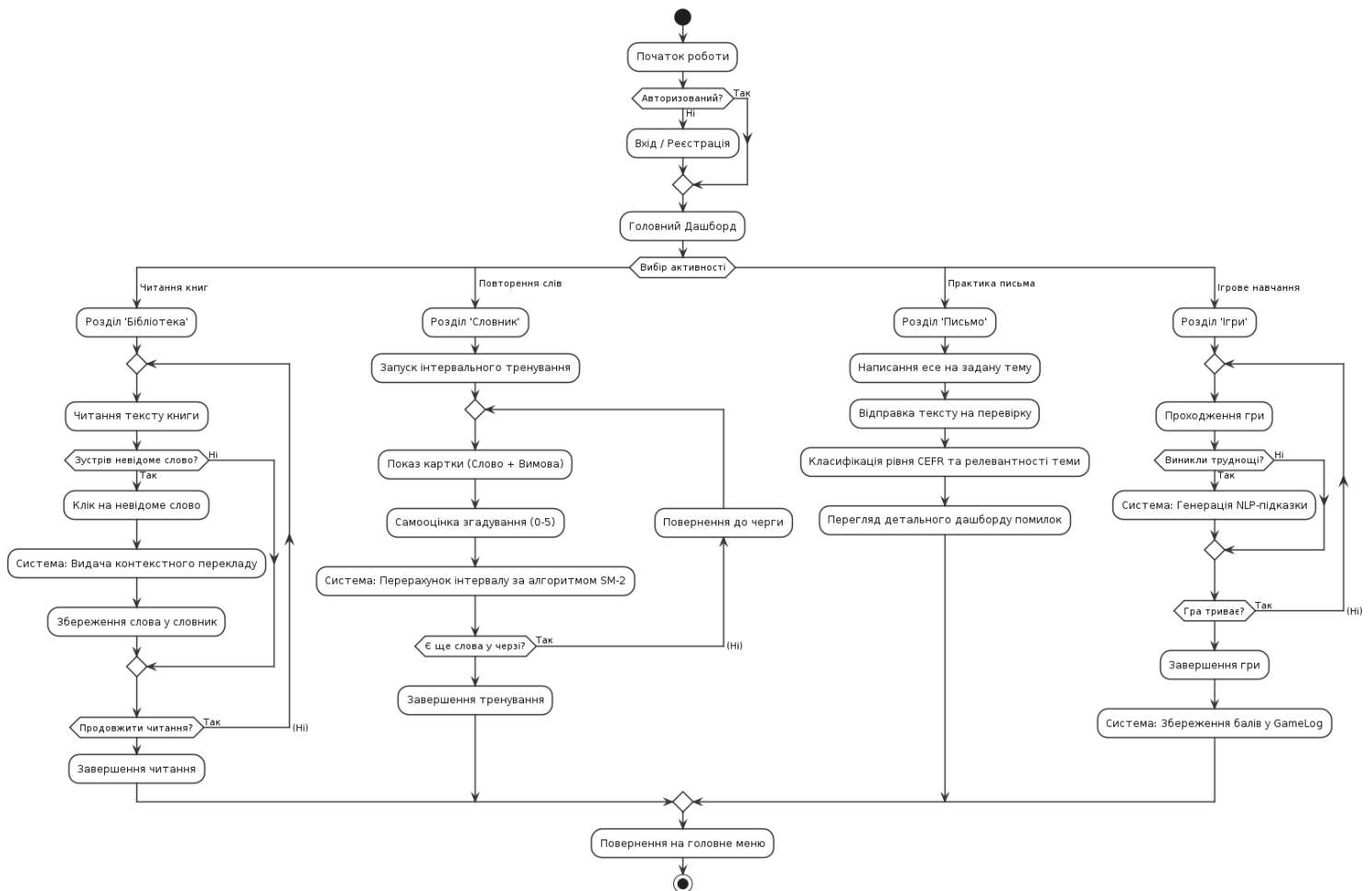


Рисунок 2.5 – Сценарій роботи користувача з освітньою платформою

Подальша взаємодія реалізується через вибір одного з основних сценаріїв: читання книг, повторення слів, практики письма або ігрового навчання. У межах кожного сценарію користувач виконує відповідні навчальні дії, такі як робота з текстом і контекстним перекладом, інтервальне повторення лексики, написання та перевірка письмових робіт або проходження навчальних ігор з NLP-підказками. Після завершення будь-якої активності користувач повертається до головного дашборду, що забезпечує циклічну та інтуїтивно зрозумілу структуру взаємодії з системою.

2.2. Загальна архітектура програмної системи

Архітектурний підхід до побудови системи

Для реалізації системи обрано багаторівневу клієнт-серверну архітектуру, що забезпечує розділення відповідальності між компонентами та спрощує супровід і масштабування.

Система побудована на монолітному backend із інтегрованим модулем машинного навчання, що зменшує затримки між бізнес-логікою та інференсом моделей і уникає складності мікросервісного підходу. Розгортання здійснюється в AWS із використанням контейнеризації, що поєднує переваги моноліту з масштабованістю хмарної інфраструктури. Основні сервіси (БД, кеш, балансування, сховище та синтез мовлення) винесені в керовані сервіси AWS.

Система структурно поділяється на чотири логічні рівні:

- рівень представлення (Frontend);
- рівень прикладної логіки (Backend);
- рівень обробки даних та машинного навчання (ML/NLP Layer);
- рівень збереження даних (Data Layer).

Рівень представлення (Presentation Layer)

Рівень представлення реалізує користувацький інтерфейс системи та забезпечує взаємодію користувача з функціоналом платформи.

Frontend побудований на основі HTML5, CSS (TailwindCSS) та JavaScript. Інтерактивні компоненти реалізовані за допомогою асинхронних запитів (Fetch API), що дозволяє виконувати взаємодію з сервером без перезавантаження сторінки.

Для рендерингу електронних книг використовується бібліотека epub.js, яка забезпечує клієнтське відображення EPUB-файлів із підтримкою навігації та виділення тексту. Додатково використовується Web Audio API для відтворення синтезованого мовлення.

Рівень прикладної логіки (Application Layer)

Бізнес-логіка системи реалізована на основі фреймворку Django, який виконує роль центрального контролера всієї системи.

Основні функції цього рівня включають:

- обробку HTTP-запитів та маршрутизацію;
- управління автентифікацією та сесіями користувачів;
- реалізацію API-ендпоінтів через стандартні Django views;
- координацію взаємодії між фронтендом, базою даних та ML-модулем.

Особливістю реалізації є відмова від Django REST Framework на користь легких JSON-відповідей через стандартні механізми Django, що зменшує складність системи та підвищує продуктивність.

Рівень машинного навчання та обробки природної мови (ML/NLP Layer)

Цей рівень відповідає за реалізацію інтелектуальних функцій системи, включаючи контекстний аналіз слів, оцінювання письмових робіт та синтез мовлення.

До складу ML/NLP шару входять:

- SentenceTransformers (all-MiniLM-L6-v2) - для побудови семантичних векторних представлень;
- spaCy - для морфологічного аналізу та POS-tagging;
- XGBoost - для класифікації рівня CEFR текстів;
- ONNX Runtime (Kokoro TTS) - для локального синтезу мовлення.

Інференс моделей виконується безпосередньо на сервері в синхронному режимі, що дозволяє уникнути залежності від зовнішніх API та забезпечує повну автономність системи.

Рівень збереження даних (Data Layer)

Рівень даних реалізований на основі реляційної СУБД PostgreSQL, яка забезпечує ACID-сумісність, високу продуктивність та підтримку JSONB-структур.

У системі зберігаються:

- профілі користувачів;
- словникові записи;
- результати тестування та ігрової активності;
- логування процесів читання.

Загальна архітектура системи

Загальну структуру взаємодії між основними рівнями системи наведено на рисунку 2.5.

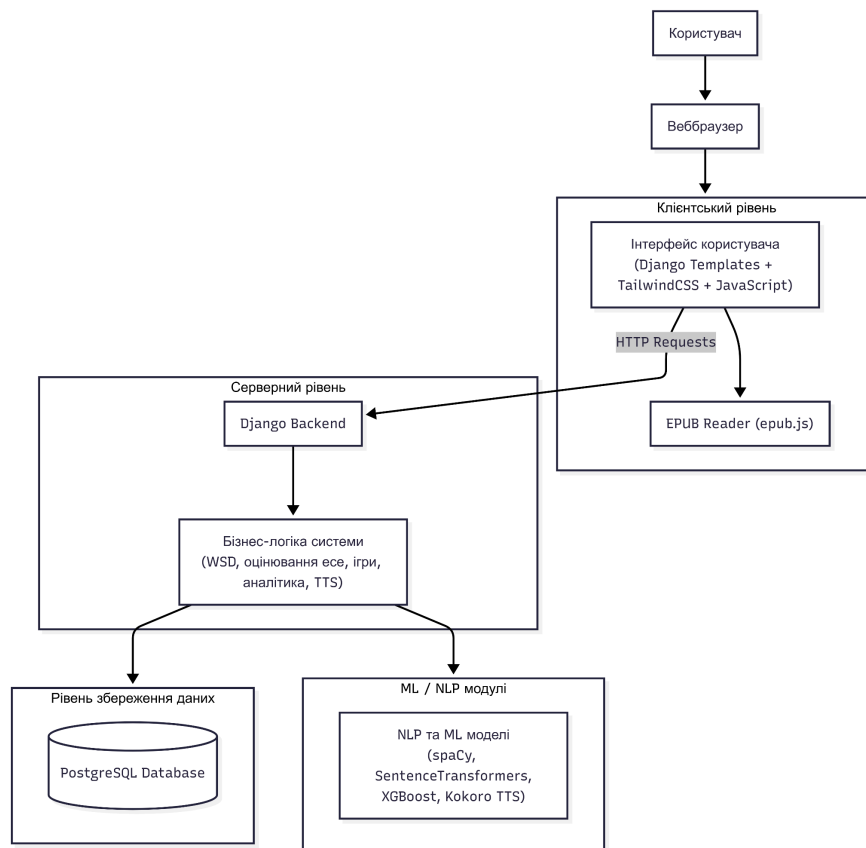


Рисунок 2.6 – Загальна архітектура системи

Як видно з рисунка 2.6, архітектура системи побудована за принципом багаторівневого розподілу відповідальності. Користувацька взаємодія здійснюється через веббраузер, який забезпечує доступ до клієнтського рівня. Frontend-компоненти виконують візуалізацію контенту та обмін даними із серверною частиною через HTTP-запити. Центральним елементом

архітектури виступає Django backend, що координує взаємодію між бізнес-логікою, модулем машинного навчання та рівнем збереження даних.

Інформаційна взаємодія компонентів

Система працює за клієнт-серверною моделлю: дії користувача у вебінтерфейсі ініціюють запити до сервера, який опрацьовує дані через відповідні функціональні модулі, залучає базу даних та підсистему машинного навчання.

Типовий приклад це контекстний аналіз слова під час читання. Після вибору слова система зчитує його контекст, знаходить можливі значення у словниковій БД, застосовує алгоритм Word Sense Disambiguation (WSD) і повертає користувачу найрелевантніший результат.

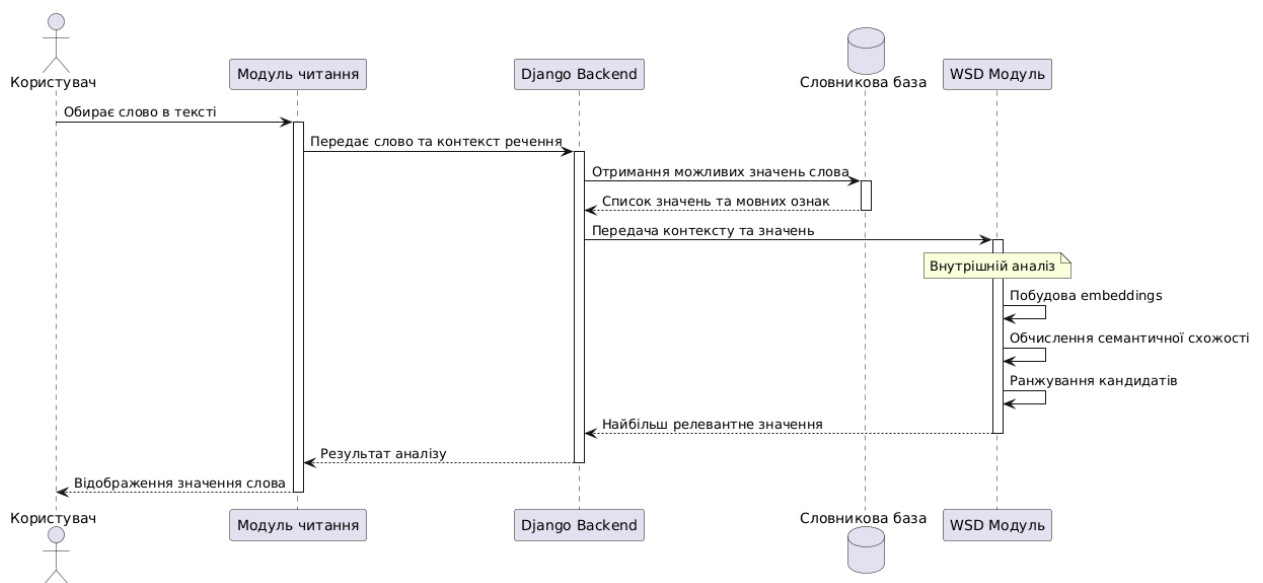


Рисунок 2.7 – Діаграма послідовності взаємодії компонентів системи під час контекстного аналізу слова

Як видно з рис. 2.7, процес починається з вибору користувачем слова в модулі читання. Клієнтська частина надсилає HTTP-запит до Django-сервера, який запитує можливі значення слова у словниковій БД і передає їх разом із контекстом речення до підсистеми семантичного аналізу.

Тут визначається найбільш релевантне значення, після чого результат повертається на сервер. Django формує JSON-відповідь і надсилає її до браузера, де дані відображаються у спливаючому вікні без перезавантаження сторінки. Ця схема демонструє базовий принцип взаємодії компонентів та є типовою для більшості функціональних модулів платформи.

Хмарна архітектура розгортання системи

Для забезпечення масштабованості, автоматизації розгортання, безпеки та відмовостійкості програмну систему було розгорнуто у хмарному середовищі AWS із використанням підходу Infrastructure as Code (IaC). Управління інфраструктурою реалізовано за допомогою Terraform, що забезпечує відтворюваність середовища та автоматизацію конфігурації ресурсів.

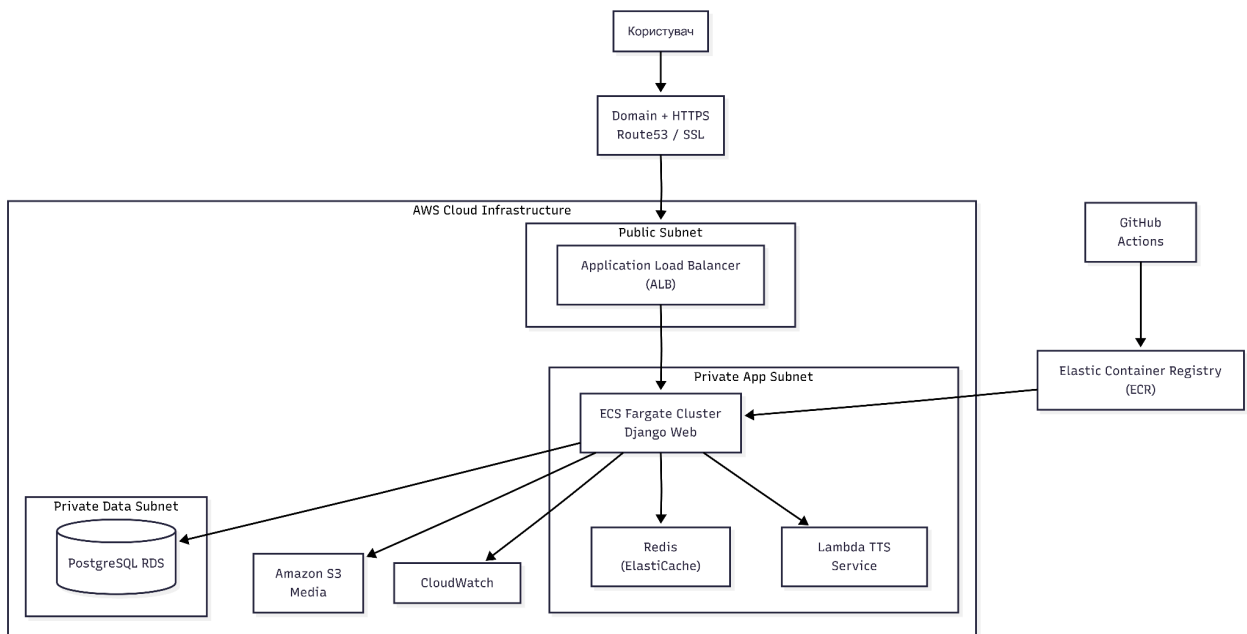


Рисунок 2.8 – Хмарна архітектура розгортання системи

Як видно з рисунка 2.8, доступ користувачів до системи здійснюється через доменне ім'я із використанням захищеного HTTPS-з'єднання. Вхідний трафік обробляється Application Load Balancer (ALB), який розподіляє запити між контейнеризованими екземплярами вебзастосунку, що виконуються у середовищі AWS ECS Fargate.

Серверна частина взаємодіє з реляційною базою даних PostgreSQL, розгорнутою в AWS RDS у приватній мережі, що виключає прямий доступ із зовнішнього середовища. Для оптимізації швидкодії та реалізації механізмів кешування використовується Redis (AWS ElastiCache).

Медіафайли та статичні ресурси зберігаються у сховищі Amazon S3. Підсистема синтезу мовлення реалізована як окремий контейнеризований сервіс AWS Lambda, що викликається серверною частиною за потреби генерації аудіо.

Автоматизація процесів збірки та доставки змін реалізована за допомогою CI/CD pipeline на основі GitHub Actions, що забезпечує автоматичне оновлення контейнерів після внесення змін до репозиторію.

2.3 Проєктування функціональної структури системи

Функціональна структура програмної системи побудована за модульним принципом, що забезпечує розподіл відповідальності між окремими компонентами та спрощує супровід, масштабування і подальший розвиток програмного забезпечення. Такий підхід дозволяє ізолювати бізнес-логіку окремих підсистем, зменшити зв'язність між модулями та підвищити повторне використання програмного коду.

Реалізація системи базується на архітектурному підході MVC (Model–View–Controller) із використанням фреймворку Django, у межах якого функціональні можливості платформи розподілено між окремими застосунками (applications). Загальна структура функціональних модулів системи наведена на рисунку 2.9.

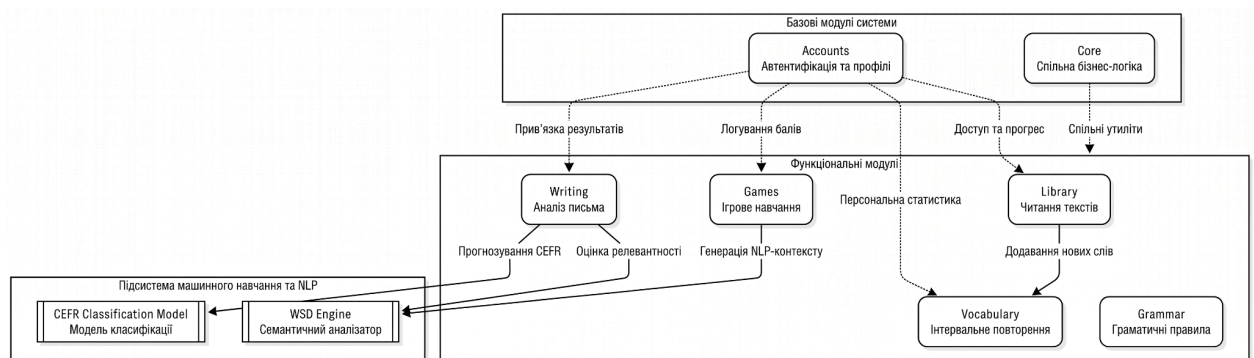


Рисунок 2.9 – Загальна структура функціональних модулів системи

Як видно з рисунка 2.9, архітектура системи складається з трьох рівнів: базових модулів, функціональних модулів та підсистеми машинного навчання і обробки природної мови.

Базовий рівень включає модулі accounts і core. Перший забезпечує автентифікацію, керування профілем і навчальним прогресом, другий - спільну бізнес-логіку та взаємодію між компонентами.

Функціональний рівень містить модулі library, vocabulary, grammar, games та writing, які відповідають за ключові навчальні сценарії: читання, роботу з лексикою, граматику, ігрові вправи та оцінювання письмових робіт.

Підсистема машинного навчання включає WSD Engine для контекстного визначення значення слів та CEFR Classification Model для автоматичного визначення рівня володіння мовою на основі результатів користувача.

2.4 Проектування інформаційної моделі та бази даних

Для збереження користувацьких даних, навчального контенту та результатів роботи системи використовується реляційна система керування базами даних PostgreSQL. Вибір цієї СУБД обумовлений її надійністю, продуктивністю та підтримкою складних зв'язків між сутностями.

PostgreSQL забезпечує підтримку транзакцій, зовнішніх ключів і механізмів контролю цілісності даних, що є важливим для коректного збереження інформації про користувачів, навчальний прогрес та результати виконаних завдань.

Додатковою перевагою є підтримка форматів JSON та JSONB, які використовуються для зберігання словникових даних, прикладів використання слів, варіантів відповідей у тестах та інших структур зі змінним набором атрибутів.

СУБД також добре інтегрується з фреймворком Django через ORM, що спрощує розробку та супровід програмного забезпечення. Таким чином,

PostgreSQL забезпечує необхідний рівень продуктивності, гнучкості та надійності для реалізації розробленої системи.

Структура моделей даних

Інформаційна модель системи побудована відповідно до модульної архітектури програмного комплексу та охоплює підсистеми керування користувачами, інтерактивного читання, словникового навчання, граматичного контенту та навчальних ігор.

Центральним елементом структури є користувач, з яким пов'язані персональні налаштування, статистика навчання та результати діяльності в системі. Підсистема читання забезпечує збереження книг, прогресу читання та користувацьких позначок у тексті.

Модуль словникового навчання відповідає за збереження лексичних одиниць, їх групування та ведення історії повторень відповідно до алгоритму інтервального навчання. Для реалізації контекстного перекладу та семантичного аналізу використовується словникова база, що містить мовні визначення, приклади використання та інші лінгвістичні характеристики.

Структура граматичного модуля побудована за ієрархічним принципом і включає навчальні матеріали та тестові завдання. Ігрова підсистема забезпечує збереження даних про проходження вправ і результатів користувачів.

Для підвищення гнучкості зберігання інформації в системі використовується поєднання реляційних зв'язків та JSON-полів, що дозволяє ефективно працювати як зі структурованими даними, так і зі складними мовними об'єктами.

ER-діаграма бази даних

Структуру інформаційної моделі системи наведено на рисунку 2.10.

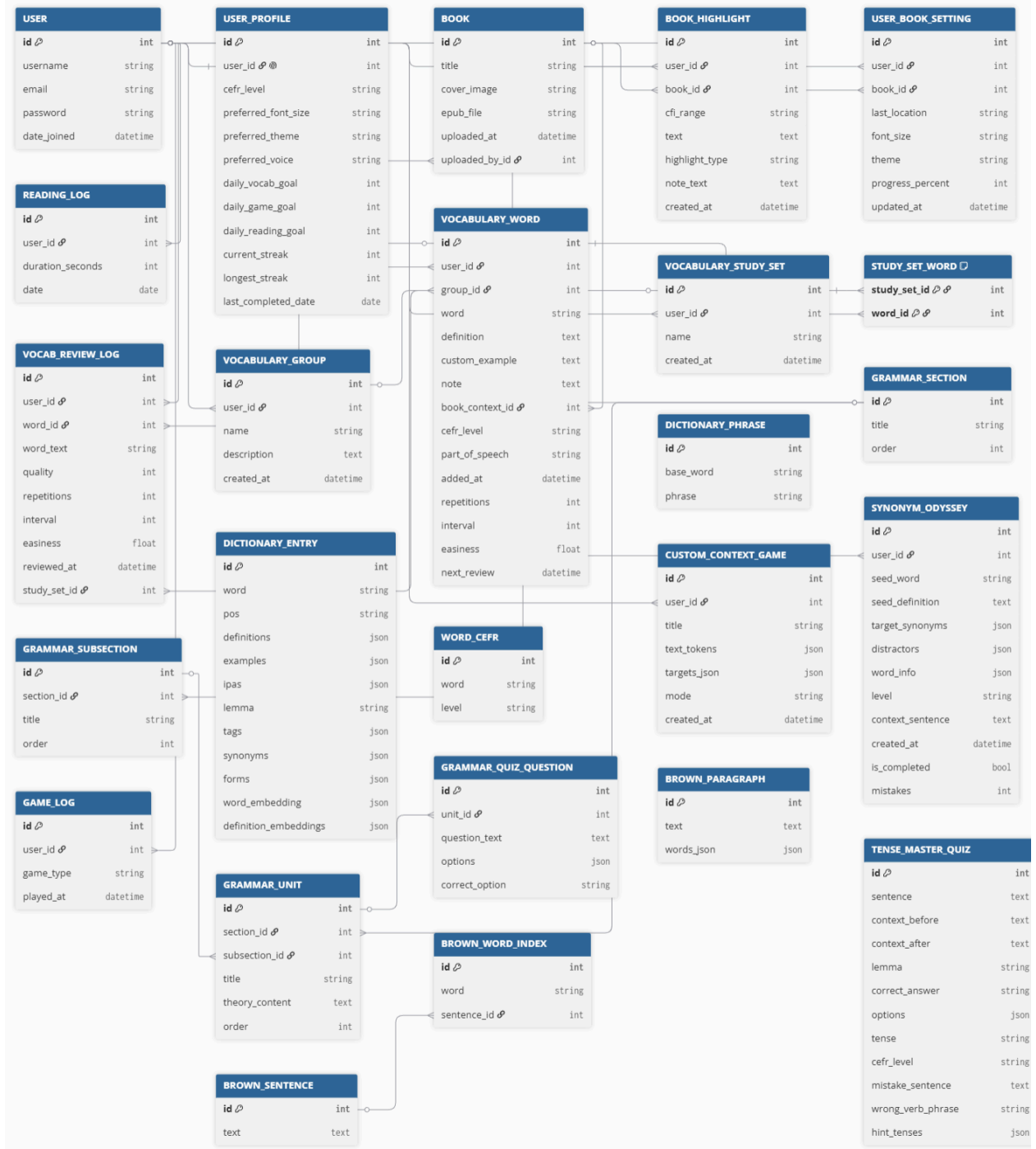


Рисунок 2.10 – ER-діаграма бази даних системи

Як видно з рисунка 2.10, структура бази даних побудована за принципом функціонального розділення сутностей відповідно до архітектури програмної системи. Центральне місце займає таблиця користувачів, з якою пов'язана більшість функціональних модулів через зовнішні ключі.

Підсистема керування профілем забезпечує збереження індивідуальних характеристик користувача, тоді як модулі читання, словникового навчання та ігрової взаємодії використовують окремі сутності для логування активності та збереження навчального прогресу.

Особливістю структури є використання зв'язків типу «багато до багатьох» через проміжні таблиці. Наприклад, взаємозв'язок між навчальними наборами та словниковими словами реалізовано через таблицю словникового сету, що дозволяє використовувати одну лексичну одиницю в декількох наборах одночасно.

Також важливою особливістю моделі є інтеграція словникових та NLP-даних безпосередньо в базу даних. Це дозволяє виконувати контекстний аналіз слів без звернення до зовнішніх API та забезпечує автономність системи.

2.5 Обґрунтування вибору технологій та інструментів

Вибір технологічного стеку здійснювався з урахуванням вимог до продуктивності, масштабованості, простоти супроводу та інтеграції компонентів штучного інтелекту.

Для реалізації серверної частини розглядалися Django, Flask та FastAPI. Незважаючи на високу продуктивність FastAPI та гнучкість Flask, було обрано Django, оскільки він надає готові механізми автентифікації, ORM, адміністрування та захисту, що дозволило зменшити обсяг допоміжного коду та прискорити розробку системи.

Клієнтська частина реалізована засобами HTML5, CSS, JavaScript і TailwindCSS. Використання React або Vue було визнано недоцільним через відсутність складної клієнтської логіки. Обраний підхід забезпечує простоту підтримки та достатню інтерактивність інтерфейсу. Для роботи з електронними книгами використовується бібліотека epub.js.

Як систему керування базами даних обрано PostgreSQL. Порівняно з SQLite вона краще підходить для багатокористувацьких вебсистем, а

використання NoSQL-рішень не було виправданим через значну кількість зв'язків між сутностями предметної області. Додатковою перевагою PostgreSQL є підтримка JSON-структур та висока продуктивність при роботі зі складними запитами.

Для реалізації функцій штучного інтелекту використано SentenceTransformers, spaCy та XGBoost. Такий набір інструментів дозволяє виконувати семантичний аналіз тексту, визначення граматичних характеристик та автоматизоване оцінювання письмових робіт без використання зовнішніх сервісів. Синтез мовлення реалізовано через ONNX Runtime, що забезпечує локальне виконання моделей та зменшує залежність від сторонніх API.

Для розгортання системи використовується хмарна інфраструктура AWS. Контейнеризація реалізована через ECS Fargate, зберігання даних - у PostgreSQL (RDS), кешування - за допомогою Redis, а файлів користувачів - у S3. Така архітектура забезпечує масштабованість, високу доступність та спрощує супровід системи.

Одним із ключових архітектурних рішень стало використання монолітної архітектури замість мікросервісного підходу. Для навчальної платформи з тісно пов'язаними компонентами це дозволило зменшити складність реалізації, спростити обмін даними між модулями та мінімізувати накладні витрати на міжсервісну взаємодію. Крім того, більшість алгоритмів машинного навчання виконуються локально, що підвищує автономність системи та знижує експлуатаційні витрати.

Висновок до розділу 2

У другому розділі було виконано проєктування інформаційної системи для вивчення англійської мови, включаючи визначення функціональних та нефункціональних вимог, моделювання сценаріїв взаємодії користувача та розробку загальної архітектури системи.

Описано структурну організацію програмного комплексу, що базується на багаторівневій клієнт-серверній архітектурі з інтеграцією модулів машинного навчання та обробки природної мови. Окрему увагу приділено хмарній інфраструктурі AWS, яка забезпечує масштабованість, відмовостійкість та кероване розгортання компонентів системи.

Визначено основні функціональні підсистеми, які забезпечують повний навчальний цикл користувача - від роботи з навчальними матеріалами до автоматизованого оцінювання результатів та аналізу прогресу.

Також спроектовано інформаційну модель системи та базу даних на основі PostgreSQL, що забезпечує збереження навчального контенту, користувацьких даних та результатів взаємодії з системою. Окремо обґрунтовано вибір технологічного стеку та архітектурних рішень, що забезпечують продуктивність, масштабованість і автономність програмного продукту. Отримані результати створюють основу для подальшої реалізації та тестування системи в наступних розділах роботи.

РОЗДІЛ 3. РОЗРОБКА ТА РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ

3.1 Загальна характеристика реалізованої системи

Розроблена система є веборієнтованою платформою для вивчення англійської мови, яка поєднує роботу з електронними книгами, персональним словником, граматичними матеріалами, навчальними іграми та засобами автоматизованого оцінювання знань. Система реалізована за клієнт-серверною архітектурою та забезпечує персоналізований навчальний процес через веббраузер.

Серверна частина побудована на основі фреймворку Django та відповідає за реалізацію бізнес-логіки, обробку запитів і взаємодію з базою даних PostgreSQL. Клієнтська частина реалізована із використанням HTML, Tailwind CSS та JavaScript. Для виконання інтелектуальних функцій інтегровано модулі обробки природної мови та машинного навчання.

Розгортання системи здійснюється в хмарному середовищі AWS. Архітектура побудована за модульним принципом та включає компоненти для роботи з користувачами, електронними книгами, словником, граматиною, письмовими роботами та навчальними іграми, що спрощує подальший розвиток і супровід системи.

3.2 Реалізація серверної частини системи

Серверна частина реалізована на базі фреймворку Django, який використовує архітектурний патерн MVT (Model–View–Template). Такий підхід забезпечує розділення даних, бізнес-логіки та інтерфейсу, що спрощує підтримку й розвиток системи.

Функціональність платформи розподілена між окремими Django-застосунками, кожен з яких відповідає за певну предметну область. Модуль accounts реалізує автентифікацію користувачів та керування персональними налаштуваннями. Модуль library забезпечує завантаження, зберігання та інтерактивне читання EPUB-книг. Робота зі словником та

алгоритмом інтервального повторення реалізована у модулі vocabulary, а теоретичні матеріали й тестові завдання - у модулі grammar.

Ігрові механіки та генерація навчальних вправ зосереджені у модулі games, тоді як модуль writing відповідає за оцінювання письмових робіт користувачів. Центральним компонентом є модуль core, який містить спільну бізнес-логіку, сервіси обробки природної мови та алгоритми машинного навчання, що використовуються іншими підсистемами платформи.

Реалізація API та механізму взаємодії між компонентами

Система побудована як клієнт-серверна архітектура, у якій взаємодія між користувацьким інтерфейсом (Frontend) та серверною частиною (Backend) реалізується через асинхронні HTTP-запити. Django виконує роль центрального API-шлюзу, що приймає JSON-запити від клієнта, координує виконання бізнес-логіки, взаємодіє з базою даних через ORM та, за необхідності, звертається до зовнішніх сервісів (зокрема AWS Lambda для синтезу мовлення). Після обробки запиту сервер повертає структуровану відповідь у форматі JSON.

Обмін даними між клієнтом і сервером реалізовано за допомогою асинхронних запитів fetch на стороні Frontend із використанням async/await, а також механізму JsonResponse на стороні Django. Такий підхід забезпечує динамічну взаємодію без необхідності перезавантаження сторінки.

Характерним прикладом є функціональність оцінювання письмових робіт. Користувач надсилає текст есе, після чого фронтенд формує POST-запит до відповідного API-ендпоінта:

```
async function submitEssay(text) {
  const response = await fetch('/api/evaluate-writing/', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ text: text })
  });
  return await response.json();
}
```

На стороні сервера Django View отримує запит, передає текст до модулів машинного навчання для аналізу та формує відповідь, яка повертається користувачу.

Подібний механізм взаємодії використовується в усіх ключових сценаріях системи:

- Контекстний переклад у Library (WSD): при виборі слова надсилається запит до `/api/translate/`, де виконується семантичний аналіз контексту та повертається переклад.
- Інтервальне повторення Vocabulary (SM-2): після оцінювання картки (0–5) викликається `/api/review-word/`, де оновлюються параметри повторення та зберігаються зміни в базі даних.
- Оцінювання письма Writing: текст есе передається до `/api/evaluate-writing/`, де ML-модель визначає рівень CEFR та формує аналітичний результат.

Схема взаємодії компонентів

Наведена нижче діаграма послідовності ілюструє типовий сценарій обробки запиту на оцінювання письмової роботи - від взаємодії користувача до повернення результату з використанням ML-моделі та бази даних.

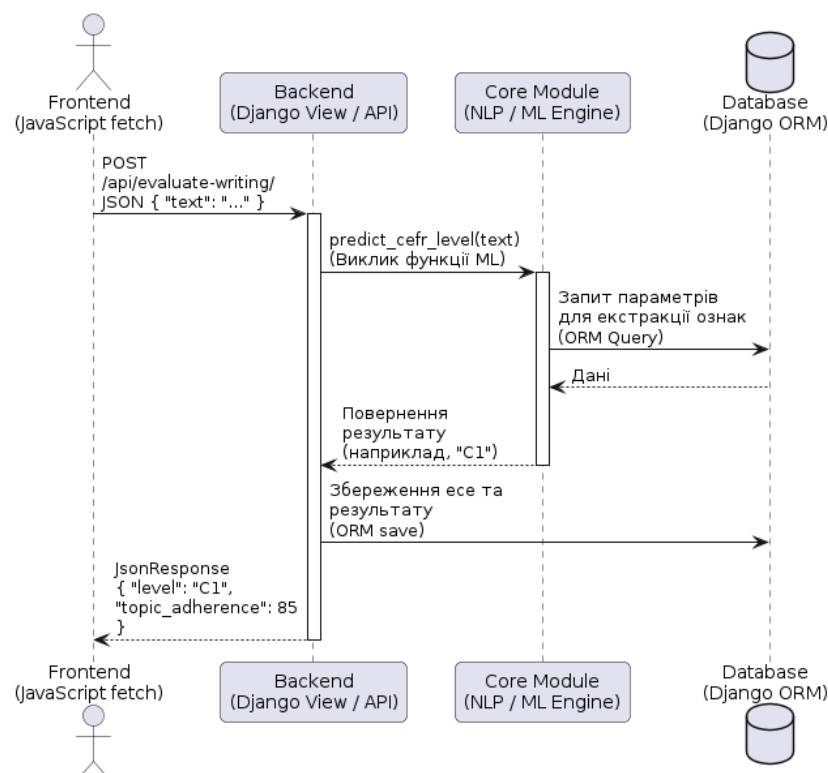


Рисунок 3.1 – Діаграма послідовності взаємодії клієнт–сервер у системі

Реалізація структури Django-додатків

Django-проект реалізовано у вигляді набору ізольованих застосунків (applications), кожен з яких відповідає за окрему предметну область системи. Такий підхід забезпечує модульність програмного коду, полегшує його супровід і дозволяє масштабувати систему без суттєвих змін у вже існуючих компонентах.

1. Фізична структура проєкту

На рівні файлової системи система розділена на головний конфігураційний модуль та набір Django-додатків, що реалізують функціональні частини платформи. Загальна структура проєкту має наступний вигляд:

```
LingoFlow/
├── english_learners/  # конфігурація Django (settings, urls, wsgi)
├── accounts/          # управління користувачами та профілями
├── core/              # NLP, ML та спільні сервіси системи
├── games/             # ігрові навчальні механіки
├── grammar/           # граматичний довідник і тестування
├── library/           # електронна бібліотека
├── vocabulary/        # словник та інтервальне повторення
├── writing/            # оцінювання письмових робіт
└── manage.py
```

Кожен із модулів є автономним Django-додатком, який може функціонувати як окремий логічний компонент, але водночас інтегрується в загальну систему через спільні інтерфейси та сервіси.

2. Внутрішня структура Django-додатків

Усі застосунки побудовані за єдиним принципом організації коду з розділенням відповідальностей. Типова структура модуля включає:

- **models.py** - опис ORM-сутностей, що відображають структуру бази даних та зв'язки між таблицями;
- **views.py** - реалізація бізнес-логіки на рівні обробки HTTP-запитів та API-ендпоінтів;

- **urls.py** - визначення маршрутів та зв'язок URL-адрес із відповідними контролерами;
- **forms.py** - серверна валідація вхідних даних;
- **signals.py** - обробка подій життєвого циклу моделей, зокрема автоматизація створення пов'язаних об'єктів;
- **apps.py** - конфігурація та реєстрація Django-додатка;
- **tests.py** - модуль автоматизованого тестування, що використовується для перевірки коректності роботи ключових функцій системи (зокрема API-ендпоінтів, бізнес-логіки та взаємодії між модулями). Тестування реалізовано на основі стандартного фреймворку Django TestCase;
- додаткові службові модулі - реалізація складної бізнес-логіки (наприклад, NLP- або ML-обробка в core).

Такий підхід дозволяє відокремити дані, логіку та представлення, що відповідає принципам чистої архітектури та спрощує подальшу підтримку системи.

3. Взаємодія між модулями системи

Попри модульну ізоляцію, Django-додатки активно взаємодіють між собою на рівні коду для забезпечення цілісності функціонування системи.

Центральну роль у цій взаємодії відіграє модуль core, який надає спільні сервіси для інших компонентів, зокрема NLP-обробку та ML-класифікацію. Наприклад, модуль games використовує функції семантичного рушія для генерації навчальних завдань, а модуль writing звертається до ML-класифікатора для оцінювання рівня складності текстів.

Модуль vocabulary інтегрується з core для обчислення семантичних подібностей та генерації навчальних сценаріїв, тоді як accounts виступає базовим шаром, що забезпечує доступ до користувацьких даних і використовується іншими модулями для прив'язки навчальної активності до конкретного користувача.

Таким чином, система зберігає баланс між модульною незалежністю та тісною взаємодією компонентів, що є критично важливим для реалізації інтелектуальної навчальної платформи.

3.3 Реалізація інтелектуальних алгоритмів системи

Реалізація контекстного аналізу слів (WSD)

Однією з ключових інтелектуальних функцій розробленої системи є автоматичне зняття лексичної багатозначності (Word Sense Disambiguation, WSD). У процесі читання англomовних текстів користувач часто стикається зі словами, які можуть мати кілька різних значень залежно від контексту вживання. Використання звичайного словникового перекладу без урахування контексту може призводити до неправильного розуміння тексту та формування помилкових асоціацій під час навчання.

Наприклад, слово *bank* в англійській мові може означати як фінансову установу (*financial institution*), так і берег річки (*river bank*). Для забезпечення точності навчального процесу система повинна автоматично визначати найбільш релевантне значення слова відповідно до речення, в якому воно використовується.

Загальна логіка роботи алгоритму

Реалізований алгоритм WSD побудований у вигляді послідовного конвеєра обробки даних (*pipeline*), у межах якого текстова інформація перетворюється у математичні векторні представлення для подальшого семантичного порівняння.

Процес роботи алгоритму включає кілька послідовних етапів:

1. Користувач обирає невідоме слово під час читання тексту.
2. Система отримує контекстне речення, у якому використано це слово.
3. Із словникової бази даних вибираються всі можливі дефініції слова.
4. Контекстне речення та словникові визначення перетворюються у багатовимірні вектори (*sentence embeddings*).

5. Для кожної дефініції обчислюється рівень семантичної близькості до контексту.
6. Виконується ранжування результатів.
7. Система повертає найбільш релевантне значення (best sense).

Нижче наведено схему роботи алгоритму контекстного визначення значення слова.

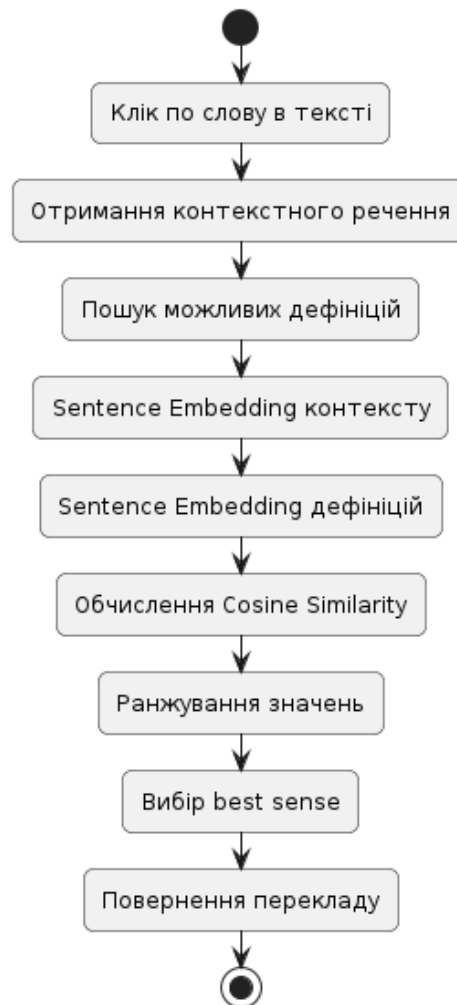


Рисунок 3.2 – Послідовність роботи алгоритму контекстного визначення значення слова (WSD)

Архітектура та технічна реалізація алгоритму

Технічно алгоритм реалізовано в модулі `core/semantic_engine.py`, який виконує функції семантичного аналізу та інтегрується із модулями `Library`, `Vocabulary` та `Games`.

Основою алгоритму є використання бібліотеки SentenceTransformers, яка дозволяє отримувати контекстно-залежні векторні представлення речень (sentence embeddings). На відміну від традиційних словникових підходів, дана технологія враховує повний контекст речення, а не лише окреме слово.

У системі використовується модель all-MiniLM-L6-v2, яка була обрана завдяки оптимальному співвідношенню між швидкодією та точністю семантичного аналізу. Дана модель генерує компактні векторні представлення тексту, що дозволяє виконувати обчислення в режимі реального часу без суттєвого навантаження на серверну інфраструктуру.

Під час роботи алгоритму контекстне речення та всі доступні словникові дефініції конвертуються у векторні представлення (embeddings). Після цього виконується оцінювання ступеня семантичної близькості між контекстом і кожним можливим значенням слова.

Математична модель семантичного порівняння

Для оцінювання схожості між контекстом слова та його словниковими дефініціями використовується метрика косинусної подібності (Cosine Similarity). Вона дозволяє визначити кутову відстань між двома векторами у багатовимірному просторі.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

де:

- $\mathbf{A}$ - вектор контекстного речення;
- $\mathbf{B}$ - вектор конкретної словникової дефініції.

Чим ближчим є значення косинусної подібності до 1, тим більш релевантною вважається дефініція щодо поточного контексту використання слова.

Програмна реалізація алгоритму

Ключова логіка обчислення семантичної близькості реалізована за допомогою тензорних операцій бібліотеки PyTorch. Контекстне речення

кодується у вектор, після чого виконується порівняння з усіма доступними дефініціями слова.

```
context_embedding = model.encode(
    context,
    convert_to_tensor=True
).cpu()

definition_embeddings = torch.tensor(
    entry.definition_embeddings,
    device='cpu'
)

similarity_scores = util.cos_sim(
    context_embedding,
    definition_embeddings
)[0]
```

Лістинг 3.1 – Обчислення семантичної подібності між контекстом та словниковими дефініціями

Оптимізація продуктивності алгоритму

Оскільки обчислення векторних представлень є ресурсоємною операцією, для забезпечення швидкодії системи було реалізовано механізм кешування попередньо обчислених ембеддингів. Вектори словникових дефініцій зберігаються безпосередньо у базі даних у полях `word_embedding` та `definition_embeddings`, що дозволяє уникнути повторної векторизації при кожному API-запиті.

Крім базового обчислення косинусної подібності, система використовує механізм динамічного повторного ранжування. Фінальна оцінка релевантності враховує не лише семантичну близькість, а й додаткові евристики, зокрема:

- відповідність частини мови (POS filtering);
- штрафи для архаїчних або маловживаних значень;
- перевагу простішим і більш уживаним дефініціям (simplicity bias).

Застосування цих механізмів дозволило підвищити стабільність роботи алгоритму та зменшити кількість помилкових перекладів у неоднозначних контекстах.

Реалізація алгоритму інтервального повторення (SM-2)

Ефективне довготривале запам'ятовування лексики є однією з ключових проблем у вивченні іноземної мови, оскільки без регулярного повторення нові слова швидко забуваються відповідно до кривої забування.

Для вирішення цієї проблеми в системі реалізовано алгоритм інтервального повторення SuperMemo-2 (SM-2), який адаптивно планує повторення лексичних одиниць залежно від рівня їх засвоєння. Основна ідея алгоритму полягає у збільшенні інтервалів для добре вивчених слів та їх скороченні для складних.

Загальна логіка роботи алгоритму

Алгоритм SM-2 реалізовано у вигляді послідовного процесу прийняття рішень, що залежить від оцінки користувача після кожного повторення слова.

Нижче наведено схему роботи алгоритму.

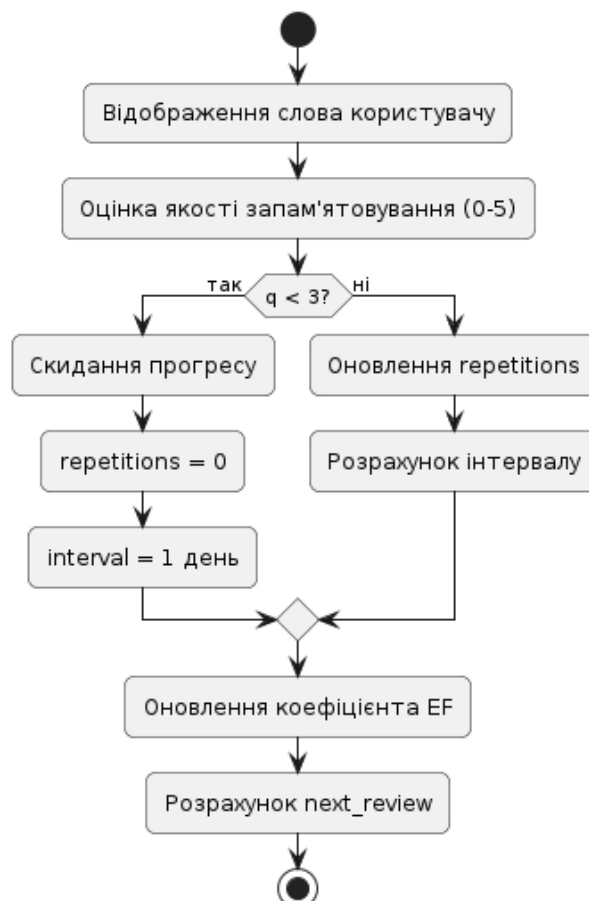


Рисунок 3.3 – Схема роботи алгоритму інтервального повторення SM-2

Математична модель алгоритму

Основою алгоритму SM-2 є адаптивне обчислення коефіцієнта легкості (E-Factor), а також інтервалів повторення.

Оновлення коефіцієнта легкості визначається за формулою:

$$EF' = EF + (0.1 - (5 - q)(0.08 + (5 - q) \cdot 0.02))$$

де:

- q - оцінка користувача (0–5),
- EF - коефіцієнт легкості запам'ятовування.

Інтервали повторення визначаються наступним чином:

$$I_1 = 1$$

$$I_2 = 6$$

$$I_n = I_{n-1} \cdot EF$$

Також у системі встановлено обмеження що запобігає надмірному зменшенню інтервалів повторення.

$$EF \geq 1.3$$

Особливості технічної реалізації в проєкті

У рамках Django-проєкту алгоритм SM-2 реалізовано у контролері `submit_word_review` модуля `vocabulary/views.py`. Логіка алгоритму враховує два сценарії: планове повторення та дострокове повторення слова.

Кожна лексична одиниця зберігає свій стан у базі даних за допомогою наступних параметрів:

- `repetitions` - кількість успішних повторень;
- `interval` - поточний інтервал повторення у днях;
- `easiness` - коефіцієнт складності запам'ятовування;
- `next_review` - дата наступного повторення.

Планове повторення

У випадку, якщо слово є актуальним для повторення, система оновлює параметри відповідно до оцінки користувача:

- при $q \geq 3$ - слово вважається засвоєним, і інтервал збільшується;
- при $q < 3$ - прогрес скидається, і слово повертається до початкового стану.

Дострокове повторення

У випадку дострокового повторення передбачено більш жорстку реакцію системи: при неправильній відповіді прогрес повністю скидається, а наступне повторення переноситься на наступний день. Такий підхід запобігає передчасному “штучному” покращенню результатів без реального засвоєння матеріалу.

Програмна реалізація алгоритму

Нижче наведено фрагмент коду, який реалізує ключову логіку SM-2 у системі:

```
if quality >= 3:
    if word.repetitions == 0:
        word.interval = 1
    elif word.repetitions == 1:
        word.interval = 6
    else:
        word.interval = int(round(word.interval * word.easiness))
    word.repetitions += 1
else:
    word.repetitions = 0
    word.interval = 1
```

Оновлення коефіцієнта легкості виконується окремо:

```
word.easiness = word.easiness + (
    0.1
    - (5 - quality) * (0.08 + (5 - quality) * 0.02)
)

if word.easiness < 1.3:
    word.easiness = 1.3
```

Логування та оптимізація даних

Для аналізу ефективності навчання реалізовано систему логування, яка зберігає кожен спробу повторення у моделі VocabularyReviewLog. Це дозволяє відстежувати динаміку засвоєння лексики та будувати аналітику прогресу користувача.

Для запобігання надмірного росту обсягу даних реалізовано механізм автоматичної оптимізації: у системі зберігаються лише останні 500 записів логів для кожного користувача, тоді як старі записи видаляються.

Такий підхід забезпечує баланс між аналітичними можливостями системи та ефективним використанням ресурсів бази даних.

Реалізація автоматичного оцінювання письма

Автоматичне оцінювання письмових робіт користувачів є однією з ключових функціональних можливостей системи. Воно дозволяє комплексно аналізувати есе з двох основних перспектив: мовної складності та тематичної відповідності.

На відміну від традиційних підходів, які базуються виключно на лексичних або граматичних ознаках, у даній системі реалізовано багаторівневу модель оцінювання, що поєднує методи машинного навчання та семантичного аналізу тексту.

Система складається з двох взаємодоповнюючих підсистем:

- **CEFR Classification Model** - визначає рівень володіння мовою на основі лінгвістичних та статистичних ознак тексту;
- **Topic Adherence Engine** - оцінює ступінь відповідності есе заданій темі за допомогою семантичного аналізу.

Комбінація цих двох компонентів дозволяє отримати більш об'єктивну та змістовну оцінку письмової роботи, яка враховує як форму, так і зміст тексту.

CEFR Classification Model

Модель класифікації рівня володіння англійською мовою (CEFR Classification Model) призначена для автоматичного визначення складності

тексту, написаного користувачем. Вона оцінює есе за шкалою CEFR (від A2 до C2) на основі набору лінгвістичних, статистичних та семантичних ознак.

Основною метою моделі є наближення процесу оцінювання до підходу викладача-експерта, який враховує не лише довжину та складність речень, але й лексичний рівень, синтаксичну структуру та загальну читабельність тексту.

Для реалізації цього завдання використано ансамблеву модель на основі XGBoost Classifier, яка демонструє високу ефективність при роботі з табличними ознаками та змішаними типами даних.

1. Формування ознак (Feature Engineering)

Ефективність моделі безпосередньо залежить від якості ознак, які витягуються з тексту. У даній системі використовується комбінований підхід, що включає лексичні, синтаксичні та показники читабельності, а також семантичне представлення тексту.

1.1 Лексичні ознаки

Лексичний рівень тексту визначається на основі словника Oxford 5000, де кожне слово має прив'язаний рівень складності (A1–C1).

Для кожного тексту обчислюються:

- частка слів рівня A1
- частка слів рівня A2
- частка слів рівня B1
- частка слів рівня B2
- частка слів рівня C1

Ці показники дозволяють оцінити загальну складність використаної лексики.

1.2 Синтаксичні ознаки

Для аналізу структури речень використовується бібліотека spaCy, яка дозволяє отримати частини мови (POS-tagging) та залежності між словами.

Основні ознаки:

- середня довжина речення
- середня довжина слова

- частка іменників, дієслів, прикметників та прислівників
- частка складних синтаксичних конструкцій (наприклад, підрядних речень)

Ці параметри дозволяють оцінити складність граматичної структури тексту.

1.3 Показники читабельності тексту

Для кількісної оцінки складності письмових текстів у системі використано набір класичних метрик читабельності, які дозволяють формалізовано визначити рівень мовної складності та наблизити її до шкали CEFR. Ці індекси базуються на статистичних характеристиках тексту, зокрема довжині речень, кількості слів та складності лексики.

Flesch-Kincaid Grade Level

Цей індекс оцінює рівень освіти, необхідний для розуміння тексту, на основі середньої довжини речень та середньої кількості складів у словах.

$$FK = 0.39 \cdot \frac{words}{sentences} + 11.8 \cdot \frac{syllables}{words} - 15.59$$

Gunning Fog Index

Індекс Fog визначає кількість років освіти, необхідних для розуміння тексту з першого прочитання. Він враховує довжину речень та частку складних слів.

$$GF = 0.4 \cdot \left(\frac{words}{sentences} + 100 \cdot \frac{complex\ words}{words} \right)$$

Coleman-Liau Index

Цей індекс базується не на складівній структурі слів, а на кількості символів у словах, що дозволяє ефективно оцінювати текст у цифровому форматі.

$$CLI = 0.0588 \cdot L - 0.296 \cdot S - 15.8$$

де:

- L - середня кількість літер на 100 слів

- S - середня кількість речень на 100 слів

1.4 Семантичні ознаки (Embeddings)

Окрім ручних ознак, використовується семантичне представлення тексту за допомогою моделі SentenceTransformers (all-MiniLM-L6-v2). Модель перетворює весь текст у вектор фіксованої розмірності, який відображає його загальний зміст. Це дозволяє враховувати не лише форму тексту, але й його смислову складність.

2. Формування вхідного простору ознак

Після видалення лінгвістичних та семантичних характеристик тексту формується єдиний багатовимірний простір ознак, який використовується як вхід для моделі машинного навчання.

У системі застосовується гібридний підхід до представлення даних, що поєднує:

- семантичні векторні представлення тексту
- нормалізовані числові лінгвістичні характеристики

Для обробки різнорідних типів даних використовується механізм ColumnTransformer, який дозволяє виконувати паралельну трансформацію ознак залежно від їх типу.

Текстові дані перетворюються у векторні представлення за допомогою SentenceTransformers, тоді як числові ознаки нормалізуються за допомогою StandardScaler для приведення всіх значень до єдиного масштабу.

У результаті кожен текст представляється як єдиний вектор, що включає:

- граматичну складність
- лексичний рівень
- читабельність
- семантичний зміст

3. Архітектура та навчання моделі (Pipeline та XGBoost)

На цьому етапі сформований вектор ознак використовується для побудови повноцінної моделі машинного навчання, яка виконує класифікацію

текстів за рівнями CEFR. Для організації всього процесу навчання застосовано підхід на основі конвеєра обробки даних (Pipeline), що дозволяє об'єднати всі етапи підготовки та навчання в єдину послідовність операцій.

Конвеєр обробки даних (Pipeline)

Реалізована система класифікації побудована як багаторівневий pipeline, який включає наступні етапи:

1. **Векторизація тексту:** Текстові дані перетворюються у семантичні embeddings за допомогою SentenceTransformers.
2. **Масштабування числових ознак:** Лінгвістичні та статистичні характеристики нормалізуються за допомогою StandardScaler.
3. **Об'єднання ознак:** Текстові та числові представлення конкатенуються в єдиний простір ознак.
4. **Класифікація:** На фінальному етапі використовується модель XGBoost, яка виконує багатокласову класифікацію рівня складності тексту.

Для задачі класифікації обрано алгоритм XGBoost Classifier, оскільки він:

- ефективно працює з табличними даними
- моделює нелінійні залежності
- стійкий до шуму у даних
- забезпечує високу точність при багатокласовій класифікації

Процес навчання моделі

Навчання моделі здійснюється у стандартному режимі з розподілом даних на навчальну та тестову вибірки:

- 80% даних використовується для навчання моделі;
- 20% даних - для тестування її узагальнювальної здатності.

Додатково застосовується стратифікований поділ (stratify), що дозволяє зберегти пропорції класів CEFR у кожній вибірці. Це є критично важливим для задачі багатокласової класифікації, оскільки рівні складності тексту можуть бути розподілені нерівномірно.

Для навчання моделі використано корпус текстів CEPOC (Cambridge English Exam Corpus), який містить реальні екзаменаційні есе та тексти, розмічені відповідно до рівнів CEFR.

Датасет складається з п'яти основних класів складності:

- A2 (KET рівень)
- B1 (PET рівень)
- B2 (FCE рівень)
- C1 (CAE рівень)
- C2 (CPE рівень)

Кожен текст у корпусі представляє собою автентичний зразок письмової відповіді, що дозволяє моделі навчатися на реальних мовних паттернах, характерних для різних рівнів володіння англійською мовою.

4. Оцінювання моделі класифікації

Оцінювання ефективності моделі здійснюється за допомогою стандартних метрик багатокласової класифікації, які дозволяють комплексно оцінити якість передбачення рівня володіння мовою. До таких метрик належать accuracy (загальна точність), precision, recall та F1-score, які обчислюються як для окремих класів, так і в усередненому вигляді (macro та weighted).

Отримана точність 61% є очікуваним результатом для багатокласової задачі CEFR-класифікації, оскільки рівні A2–C2 мають високу семантичну близькість, особливо між сусідніми класами (B2–C1–C2), що ускладнює чітке розділення меж між ними.

Результати класифікаційного звіту наведені на рисунку нижче.

	precision	recall	f1-score	support
A2	0.50	1.00	0.67	1
B1	1.00	0.75	0.86	4
B2	0.58	0.88	0.70	8
C1	0.50	0.33	0.40	6
C2	0.50	0.25	0.33	4
accuracy			0.61	23
macro avg	0.62	0.64	0.59	23
weighted avg	0.62	0.61	0.58	23

Рисунок 3.4 – Classification Report для моделі CEFR Classification

Для візуального аналізу помилок класифікації використовується матриця помилок (confusion matrix), яка відображає розподіл фактичних та передбачених класів CEFR. Вона дозволяє визначити, між якими рівнями модель найчастіше допускає помилки та які класи є найбільш складними для розпізнавання.

Основні закономірності, що спостерігаються на основі матриці помилок, наведені на рисунку нижче.

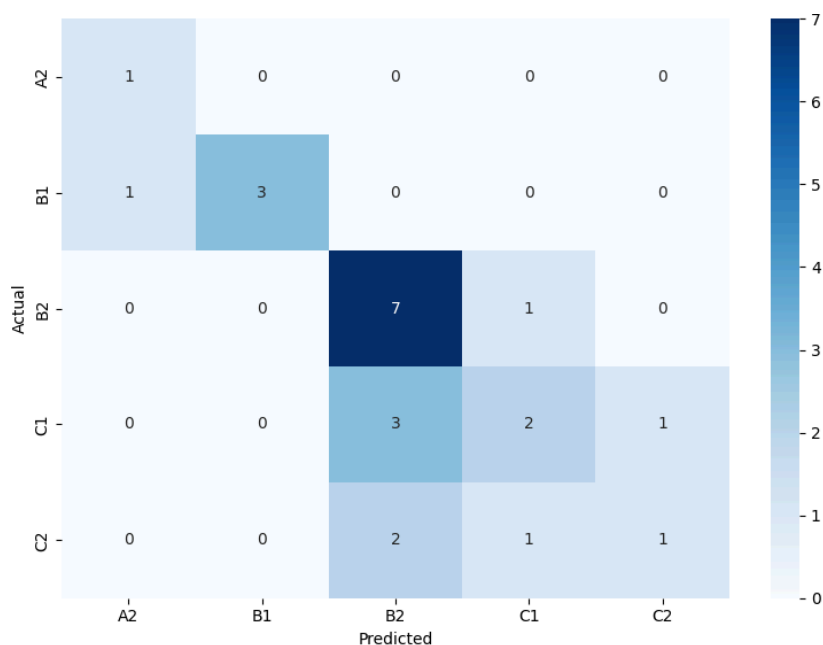


Рисунок 3.5 – Матриця помилок (Confusion Matrix) для моделі CEFR Classification

Вхідний датасет має нерівномірний розподіл класів, де домінують рівні B2 та C1, що є типовим для корпусів екзаменаційних есе. Це призводить до часткового зміщення моделі в бік більш представлених класів.

Для компенсації цього ефекту використано параметр `class_weight='balanced'`, який автоматично коригує ваги класів під час навчання, підвищуючи значущість менш представлених категорій.

Параметри моделі та налаштування

У процесі навчання використано модель XGBoost Classifier з наступними параметрами:

- кількість дерев: $n_estimators = 150$
- максимальна глибина дерев: $max_depth = 5$
- швидкість навчання: $learning_rate = 0.1$
- регуляризація: $reg_alpha = 0.1$, $reg_lambda = 1.0$

Для зменшення розмірності семантичних ознак застосовується PCA з 64 компонентами, що дозволяє зменшити шум у векторних представленнях та підвищити стабільність моделі.

Отримані результати свідчать про те, що запропонована модель демонструє стабільну ефективність у класифікації текстів середнього рівня складності та обмеження при розрізненні високих рівнів CEFR через їхню семантичну близькість. Незважаючи на це, модель є придатною для практичного використання в освітній системі та може бути покращена шляхом балансування даних та подальшої оптимізації гіперпараметрів.

Topic Adherence Engine

1. Призначення підсистеми

Підсистема оцінювання відповідності тексту заданій темі призначена для визначення ступеня релевантності есе до сформульованого завдання та є складовою системи оцінювання письмових робіт.

На відміну від підсистеми визначення рівня володіння мовою, яка аналізує граматичну та лексичну складність, дана підсистема зосереджена на змістовій відповідності та повноті розкриття теми. Основною задачею є виявлення випадків, коли текст є формально коректним, але не відповідає завданню або лише частково його розкриває.

Підсистема забезпечує автоматичну оцінку виконання завдання, визначення ступеня відхилення від теми та підвищення об'єктивності перевірки письмових робіт.

2. Загальна логіка роботи алгоритму

Алгоритм оцінювання відповідності есе заданій темі реалізовано як

процес семантичного аналізу тексту, спрямований на визначення ступеня відповідності змісту есе поставленому завданню як на рівні ключових слів, так і на рівні загальної семантики.

Першим етапом є отримання вхідних даних - тексту есе та умови завдання. Далі виконується лінгвістична обробка за допомогою spaCy, що включає токенизацію, лематизацію, видалення службових слів і розбиття тексту на речення.

Наступним кроком є побудова векторного представлення тексту за допомогою SentenceTransformers (all-MiniLM-L6-v2). Отримані вектори відображають семантичний зміст тексту та використовуються для подальшого порівняння у векторному просторі після нормалізації.

На основі отриманих векторних представлень виконується обчислення семантичної подібності між текстом есе та текстом завдання, що дозволяє визначити загальний рівень відповідності змісту відповіді поставленій темі.

Паралельно здійснюється перевірка тематичної відповідності за двома напрямками:

- лексична перевірка, яка базується на зіставленні слів тексту з набором ключових слів теми;
- семантична перевірка, що передбачає порівняння векторних представлень речень із тематичними векторними представленнями.

Такий підхід дозволяє одночасно враховувати як поверхневу лексичну відповідність, так і глибинну семантичну близькість тексту до заданої теми.

Далі виконується обчислення рівня покриття теми, яке характеризує ступінь охоплення основних тематичних аспектів у тексті. Окремо здійснюється аналіз речень з метою виявлення фрагментів, що відхиляються від заданої тематики.

Після цього виконується агрегація часткових результатів, на основі яких розраховується підсумковий показник відповідності темі. Отримане значення використовується для класифікації есе на один із трьох рівнів:

- низька відповідність темі;
- часткова відповідність темі;
- повна відповідність темі.

Завершальним етапом є формування фінального результату оцінювання, який повертається користувачу системи.

Схематичне представлення цього процесу наведено на рисунку нижче.

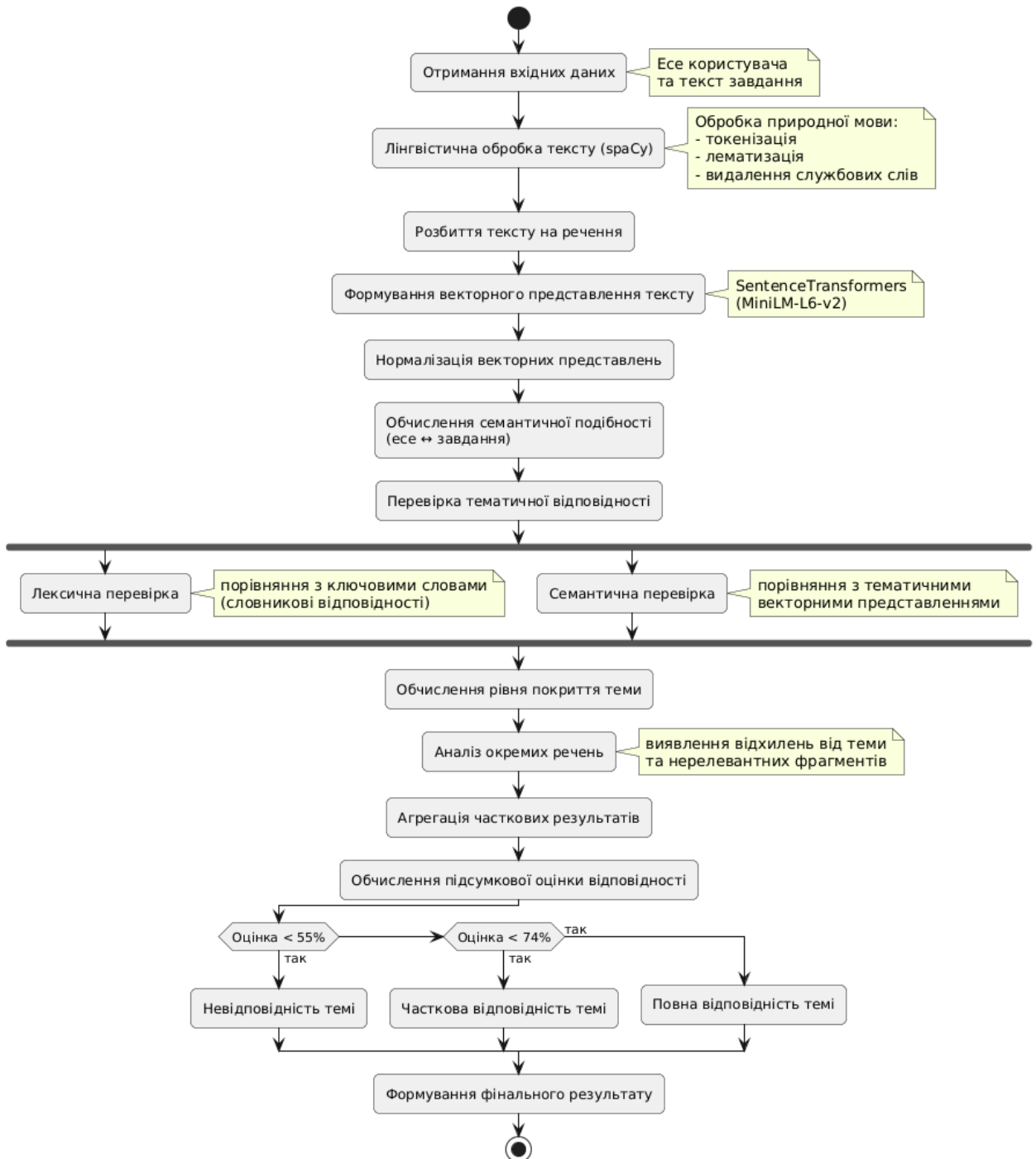


Рисунок 3.6 – Загальна логіка роботи алгоритму оцінювання

відповідності темі

3. Формалізація функції оцінювання відповідності темі

Після виконання етапів попередньої обробки тексту та обчислення часткових показників відповідності формується інтегральна оцінка, яка визначає ступінь відповідності есе заданій темі. Дана оцінка обчислюється як зважена комбінація декількох семантичних та лінгвістичних компонентів.

Основними складовими функції оцінювання є:

- семантична подібність між есе та текстом завдання;
- рівень покриття тематичних аспектів;
- середня семантична відповідність речень;
- лексична відповідність ключовим словам;
- штраф за відхилення окремих речень від теми.

Фінальний показник відповідності темі обчислюється за наступною узагальненою формулою:

$$Score = w_1P + w_2C + w_3R + w_4S + w_5T - D$$

де:

- P - семантична подібність між есе та текстом завдання;
- C - покриття тематичних аспектів;
- R - відповідність еталонним тематичним представленням;
- S - середня семантична подібність речень;
- T - лексичне покриття ключових слів;
- D - штраф за відхилення окремих речень від теми;
- $w_1 \dots w_5$ - вагові коефіцієнти, що визначають внесок кожної складової у фінальний результат.

З метою нормалізації результату всі компоненти приводяться до діапазону $[0;1]$, що забезпечує коректне агрегування показників різної природи.

Штрафна компонента D вводиться для зменшення впливу текстів, які містять окремі нерелевантні фрагменти, навіть за умови загальної

семантичної схожості із завданням.

Запропонований підхід до оцінювання відповідності темі дозволяє враховувати як поверхневу лексичну відповідність тексту, так і його глибинний семантичний зміст. Поєднання векторного представлення текстів, аналізу тематичних аспектів та механізму штрафування за змістові відхилення забезпечує більш стійке та об'єктивне оцінювання письмових відповідей.

На відміну від підходів, що базуються виключно на пошуку ключових слів, реалізований алгоритм здатний враховувати контекст висловлювань та визначати змістову близькість навіть у випадках використання синонімічних конструкцій або перефразування. Це дозволяє підвищити точність перевірки письмових робіт та мінімізувати випадки хибного визначення відповідності темі.

У результаті реалізована підсистема забезпечує автоматичне визначення ступеня змістової релевантності есе та виступає важливим компонентом комплексної системи автоматичного оцінювання письма в програмному продукті.

Реалізація інтелектуального словника на основі Wiktionary

1. Призначення підсистеми

Підсистема інтелектуального словника призначена для забезпечення автоматизованого доступу до структурованої лексичної інформації англійської мови, яка використовується в межах системи автоматичного оцінювання письмових робіт. Основною функцією даного модуля є надання детальної інформації про словникові одиниці, зокрема їх значення, граматичні характеристики, приклади вживання та семантичні зв'язки.

Дана підсистема виконує роль мовної бази знань і забезпечує підтримку таких процесів, як семантичний аналіз тексту, інтерпретація значень слів та підвищення точності роботи модулів оцінювання мовного рівня.

2. Джерело та формування словникової бази даних

Як джерело словникових даних використано структурований дамп

англомовного Wiktionary, отриманий з ресурсу Kaikki.org, який надає оброблену версію словника у форматі JSON Lines (JSONL). У даному форматі кожен рядок містить повний опис окремої лексичної одиниці.

Основні характеристики вхідного набору даних:

- формат даних: JSON Lines (JSONL);
- джерело: англомовний Wiktionary (Kaikki.org);
- розмір початкового файлу: приблизно 2.79 GB;
- кількість записів: понад 2.9 мільйона лексичних одиниць.

Після завантаження даних було виконано етап попередньої обробки, який включав фільтрацію надлишкової інформації, нормалізацію структурованих полів та відбір лише тих атрибутів, які використовуються в межах системи.

У результаті обробки сформовано приблизно 1 460 000 унікальних словникових записів, які були імпортовані до бази даних PostgreSQL.

3. Процес обробки та завантаження даних

Обробка словникового корпусу здійснювалася за допомогою спеціалізованого скрипта попередньої підготовки даних, який виконував:

- фільтрацію некоректних або неповних записів;
- видалення надлишкових метаданих;
- нормалізацію лексичних форм;
- уніфікацію структури записів.

Після завершення обробки дані були імпортовані до реляційної бази даних PostgreSQL, що забезпечує високу продуктивність при роботі з великими обсягами інформації та підтримує ефективну індексацію складних запитів.

4. Структура зберігання словникових даних

Для зберігання лексичних одиниць у системі використовується модель даних DictionaryEntry, реалізована в межах Django ORM. Структура моделі наведена нижче:

```

class DictionaryEntry(models.Model):
    word = models.CharField(max_length=255, db_index=True)
    pos = models.CharField(max_length=50)
    definitions = models.JSONField(default=list)
    examples = models.JSONField(default=list)
    ipas = models.JSONField(default=list)
    lemma = models.CharField(max_length=255, db_index=True,
blank=True, null=True)
    tags = models.JSONField(default=list)
    synonyms = models.JSONField(default=list)
    forms = models.JSONField(default=list)
    word_embedding = models.JSONField(blank=True, null=True)
    definition_embeddings = models.JSONField(blank=True, null=True)

class Meta:
    indexes = [
        models.Index(fields=['word', 'pos']),
        models.Index(fields=['lemma']),
    ]
    db_table = 'core_dictionaryentry'

```

Зазначена структура забезпечує зберігання як поверхневих лексичних характеристик, так і семантичних представлень слів, що використовуються у задачах аналізу тексту.

5. Зберігання семантичних векторних представлень

Для реалізації семантичного аналізу та задачі розпізнавання значень слів (Word Sense Disambiguation) використовуються векторні представлення, отримані за допомогою моделі all-MiniLM-L6-v2.

У системі зберігаються наступні типи векторів:

- word_embedding - векторне представлення слова;
- definition_embeddings - набір векторних представлень окремих визначень.

Векторні представлення зберігаються у форматі JSON як числові масиви, що забезпечує зручну інтеграцію з PostgreSQL та швидке завантаження даних для подальшого обчислення семантичної подібності.

6. Індексція та оптимізація доступу до даних

Для забезпечення високої продуктивності пошуку у словниковій базі даних реалізовано систему індексації на основі B-tree структур PostgreSQL.

У межах системи створено такі індекси:

- первинний індекс за ідентифікатором запису (id);
- індекс за полем word для забезпечення швидкого точного пошуку;
- індекс за полем lemma для підтримки лематизованого пошуку;
- складений індекс за полями (word, pos) для розрізнення значень слів залежно від частини мови.

Застосування індексації дозволяє зменшити часову складність пошуку з лінійної до логарифмічної, що є критично важливим при роботі з великими словниковими масивами.

3.4 Реалізація користувацького інтерфейсу

Реалізація інтерфейсу читання EPUB

Модуль читання EPUB забезпечує роботу користувача з електронними книгами безпосередньо у вебінтерфейсі системи. Інтерфейс підтримує навігацію між сторінками, збереження прогресу читання, контекстний переклад слів, створення нотаток та озвучування тексту.

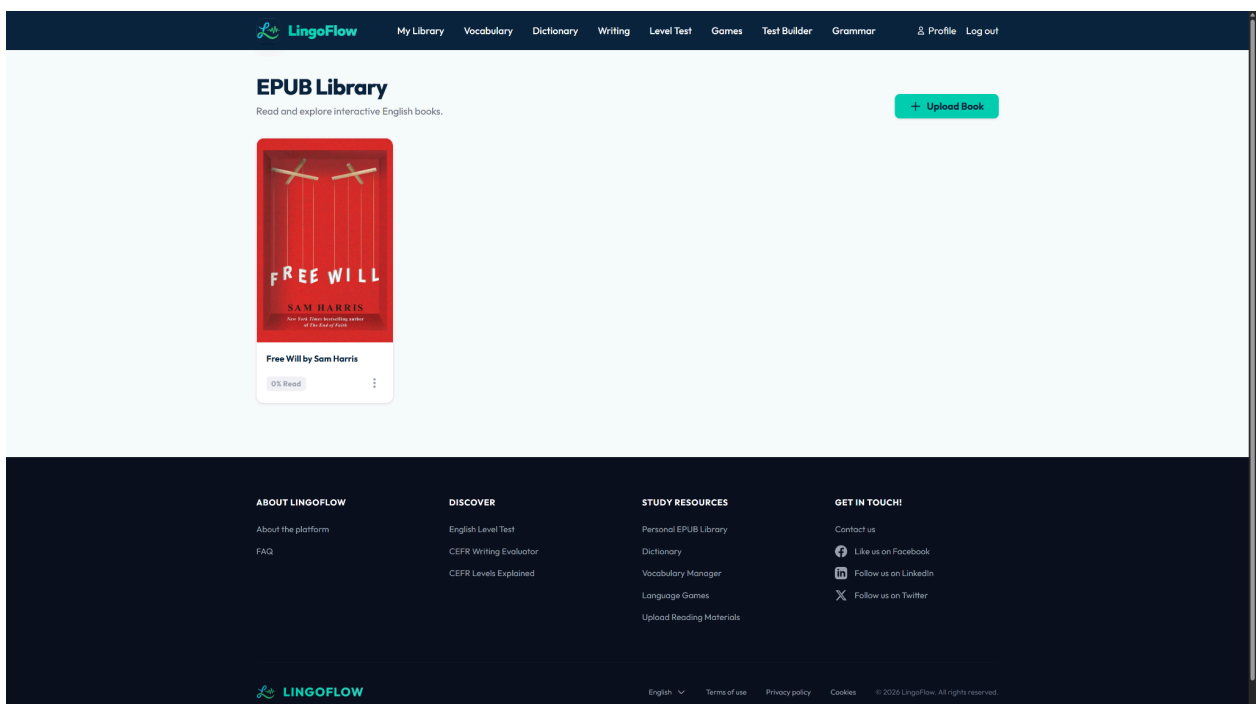


Рисунок 3.7 – Головне вікно бібліотеки користувача

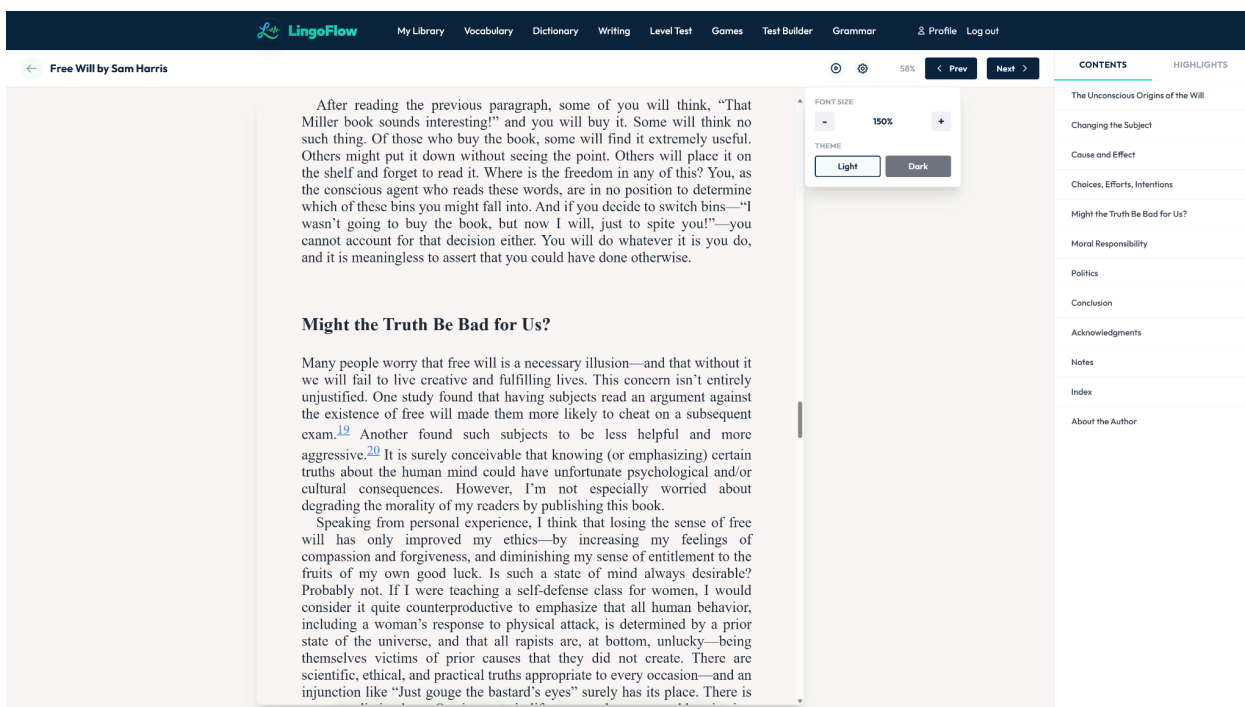


Рисунок 3.8 – Головне вікно інтерфейсу читання EPUB-книги

Під час читання користувач може натиснути на будь-яке слово для отримання його перекладу та словникової інформації з урахуванням контексту речення.

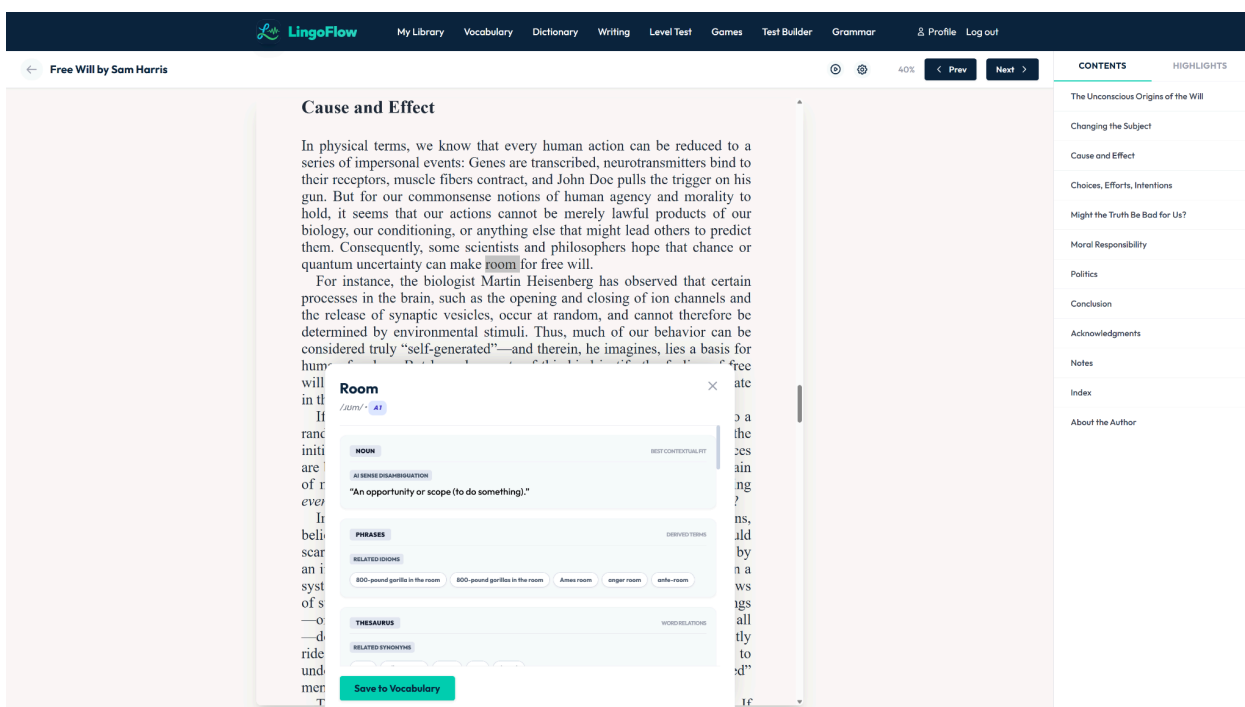


Рисунок 3.9 – Відображення перекладу вибраного слова у словнику

Система також підтримує створення текстових виділень та нотаток, які автоматично зберігаються та відновлюються при повторному відкритті книги.

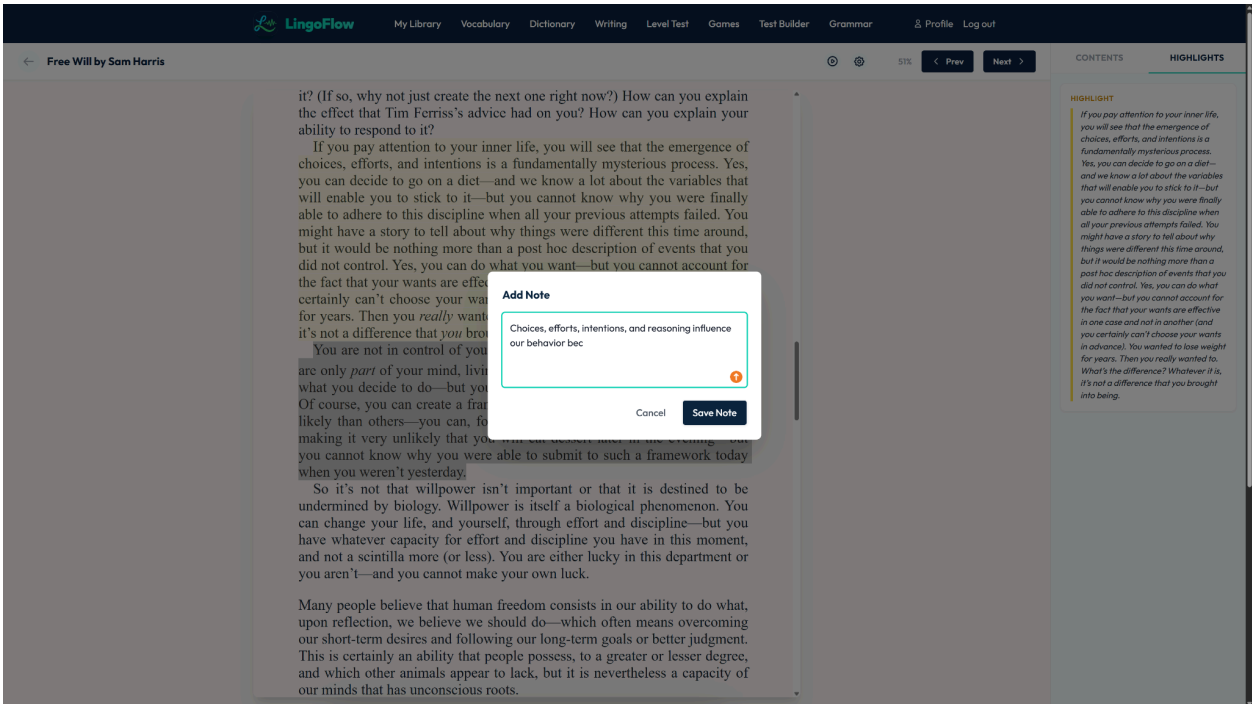


Рисунок 3.10 – Створення текстового виділення та нотатки

Для розвитку навичок аудіювання реалізовано функцію озвучування виділеного тексту за допомогою нейромережевого синтезу мовлення.

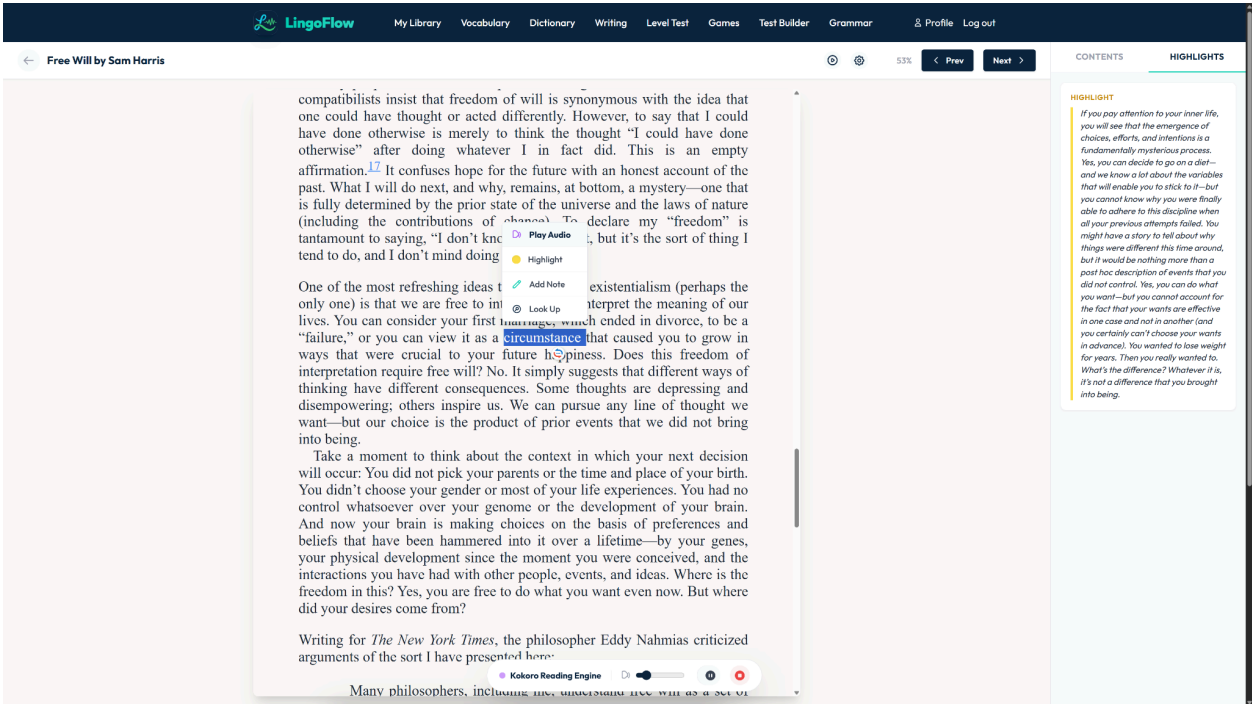


Рисунок 3.11 – Озвучування тексту в інтерфейсі читання

Усі дії користувача автоматично зберігаються, що дозволяє відновлювати останню позицію читання та відстежувати прогрес опрацювання книги.

Реалізація користувацького словника та повторення

Цей модуль забезпечує користувачу можливість накопичення нової лексики, її систематизації та ефективного засвоєння за допомогою механізму інтервальних повторень. Інтерфейс модуля дозволяє переглядати додані слова, об'єднувати їх у навчальні набори та проходити тренування у форматі флешкарток.

Головне вікно словника користувача наведено на рисунку нижче.

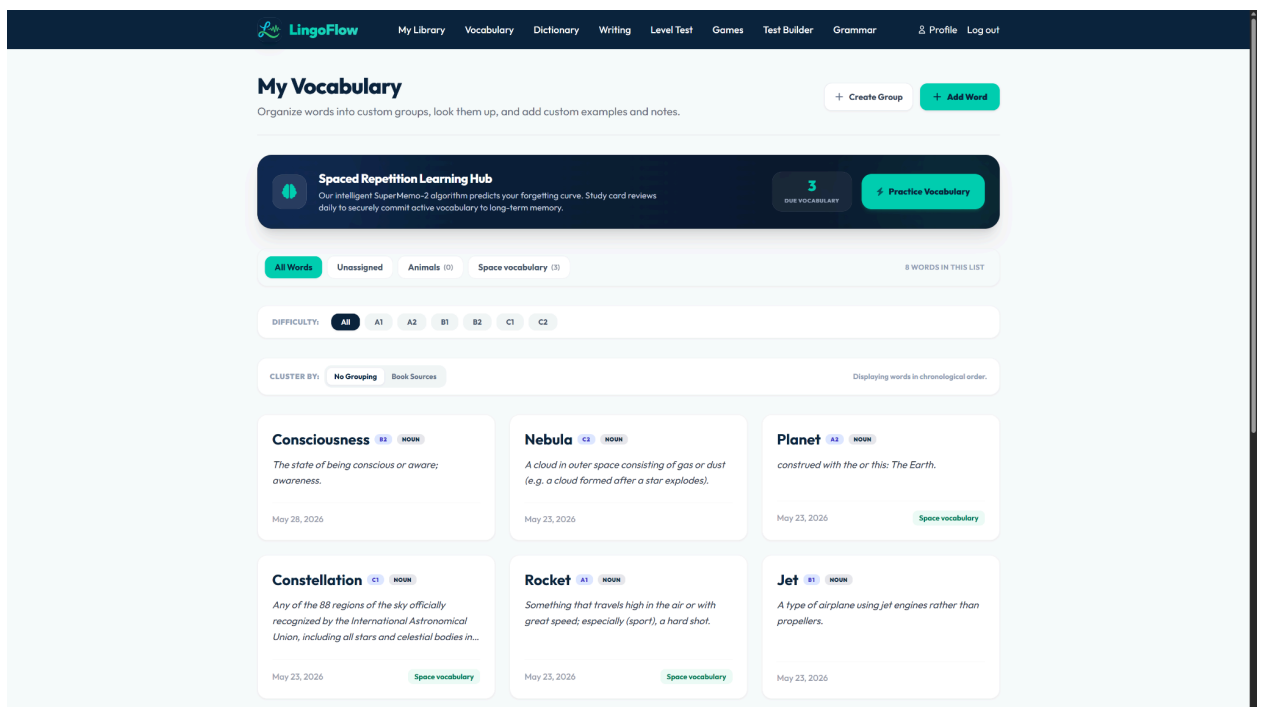


Рисунок 3.12 – Головне вікно персонального словника користувача

У цьому інтерфейсі користувач може переглядати список доданих слів а також фільтрувати лексику за навчальними групами та рівнем складності. Додатково передбачено можливість створення власних тематичних наборів для організації навчання.

Навчальні набори та сторінка повторення слів представлені на рисунку нижче.

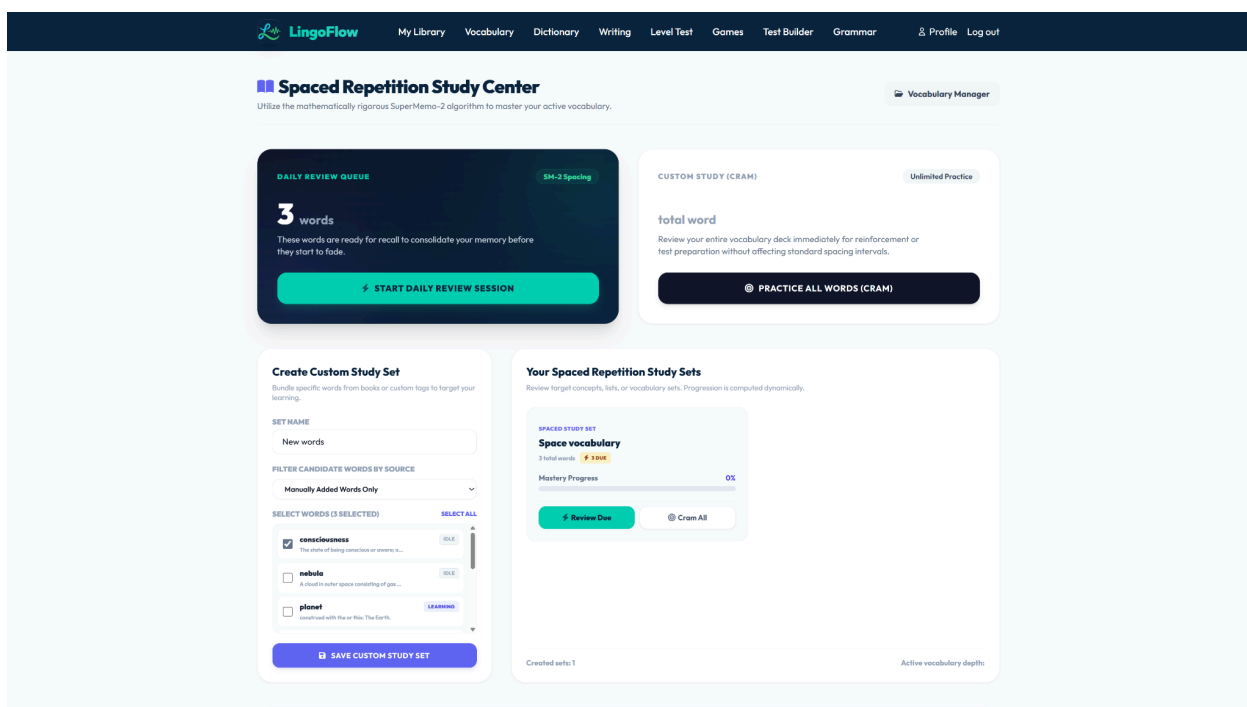


Рисунок 3.13 – Інтерфейс навчальних наборів та сторінка повторення слів

Система автоматично формує список слів, які підлягають повторенню відповідно до алгоритму інтервальних повторень. Процес навчання реалізовано у вигляді інтерактивних флешкарток, які забезпечують активне запам'ятовування нових слів.

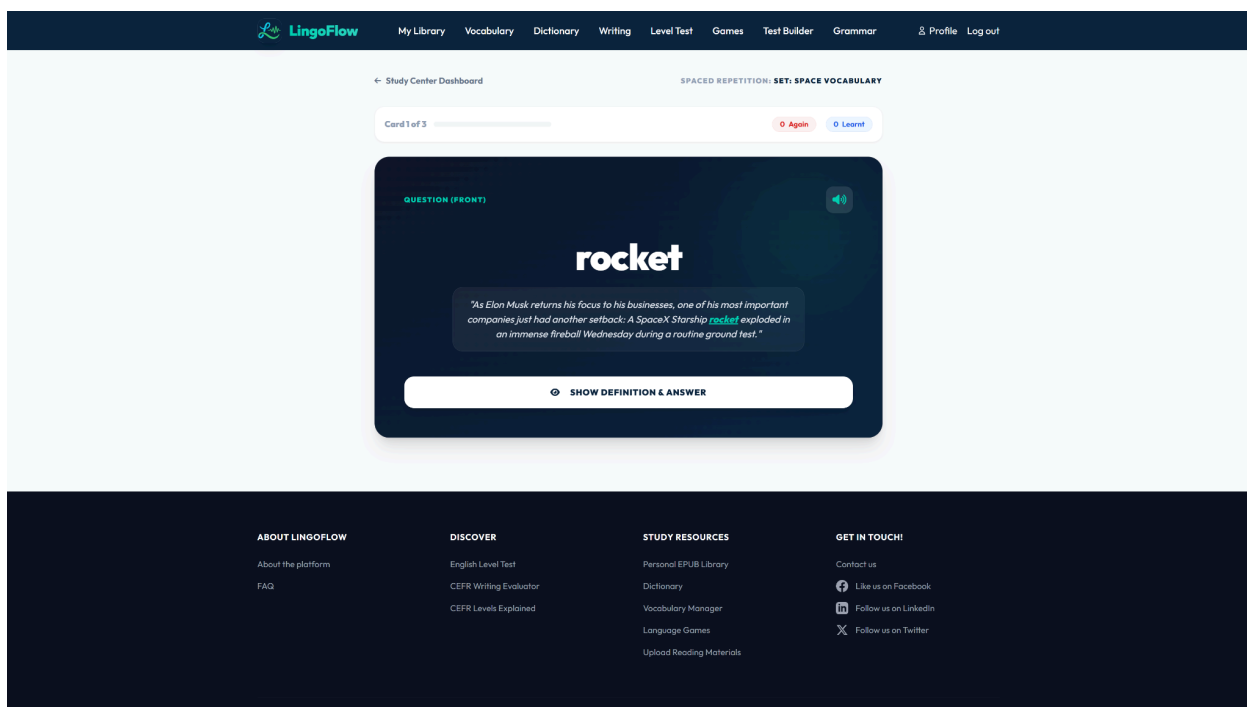


Рисунок 3.14 – Інтерфейс флешкарток для вивчення лексики

На лицьовій стороні картки відображається слово та приклад його використання, а на зворотній стороні словникове значення, контекстні приклади та додаткові нотатки користувача. Після відповіді користувач оцінює рівень складності запам'ятовування, що впливає на подальший інтервал повторення.

Реалізація інтерфейсу глобального словника

Інтерфейс глобального словника призначений для надання користувачеві доступу до великої лексичної бази обсягом 1,46 мільйона слів. Він дозволяє здійснювати пошук слів, перегляд їх значень, транскрипцій, прикладів уживання, синонімів та ідіом, а також додавати обрані слова до персонального словника користувача.

Головне вікно глобального словника наведено на рисунку нижче.

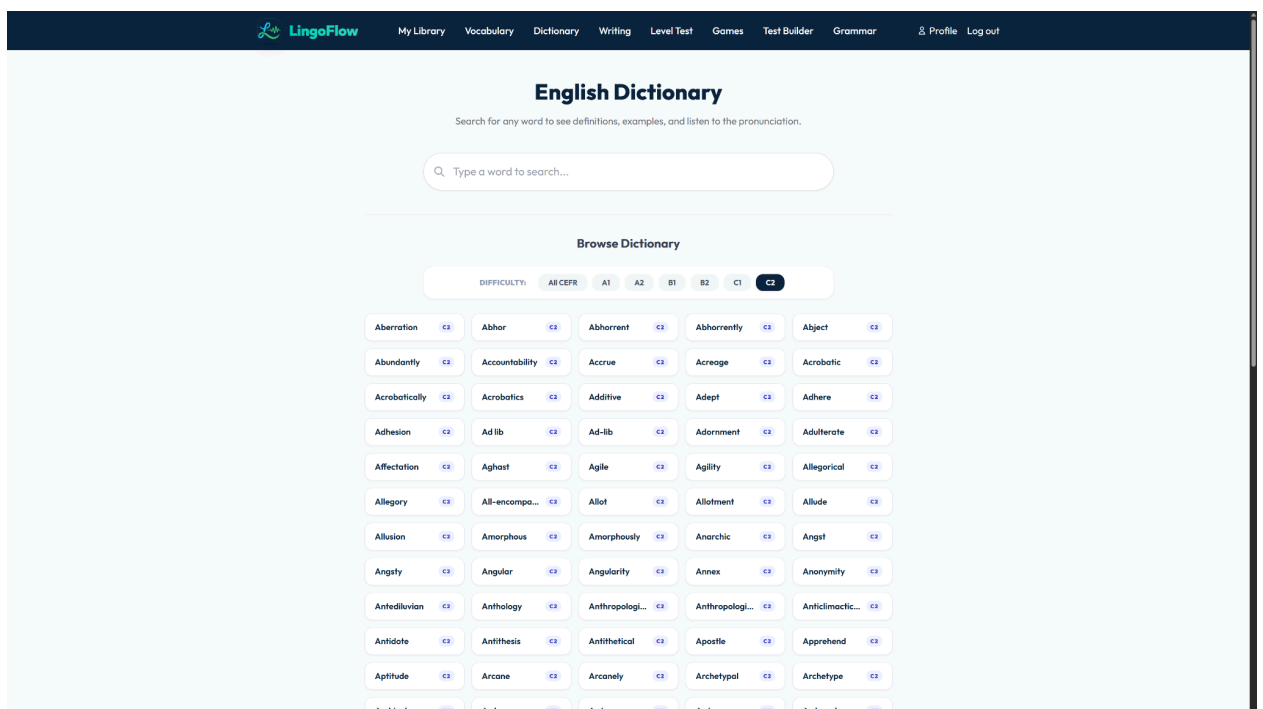


Рисунок 3.15 – Головне вікно глобального словника

У цьому інтерфейсі користувач може здійснювати швидкий пошук слів із підтримкою автодоповнення, що дозволяє значно прискорити навігацію по словнику. Для кожного слова відображається рівень складності за шкалою CEFR, транскрипція та основні значення.

Сторінка детального перегляду слова представлена на рисунку нижче.

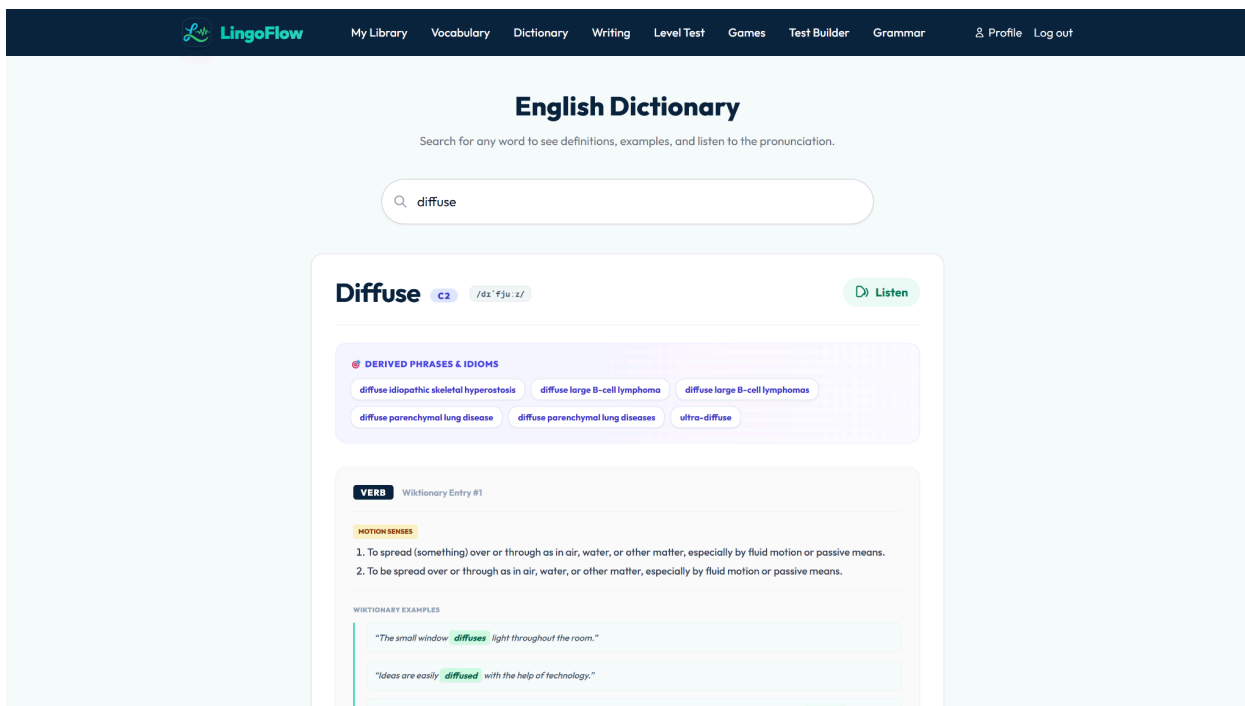


Рисунок 3.16 – Детальний перегляд слова у глобальному словнику

На цій сторінці користувач отримує повну інформацію про слово, включаючи визначення, приклади вживання, синоніми та ідіоматичні вирази. Також доступна функція прослуховування правильної вимови слова.

Реалізація навчальних ігор та гейміфікації

Ігровий модуль призначений для гейміфікації процесу навчання та закріплення лексичних і граматичних навичок користувача. Він реалізований як окремий Django-додаток. Уся ігрова активність користувача зберігається у базі даних у вигляді логів, що дозволяє відстежувати прогрес навчання та активність у системі.

Tense Master (гра на граматичні часи)

Гра призначена для перевірки знань часових форм англійського дієслова. Завдання попередньо зберігаються у базі даних та поділяються за рівнями складності CEFR.

Ігрова сесія складається з набору завдань різних типів: множинний вибір, введення правильної форми дієслова та знаходження граматичних помилок у реченнях. Перевірка відповідей виконується на сервері, після чого користувач отримує результат та пояснення.

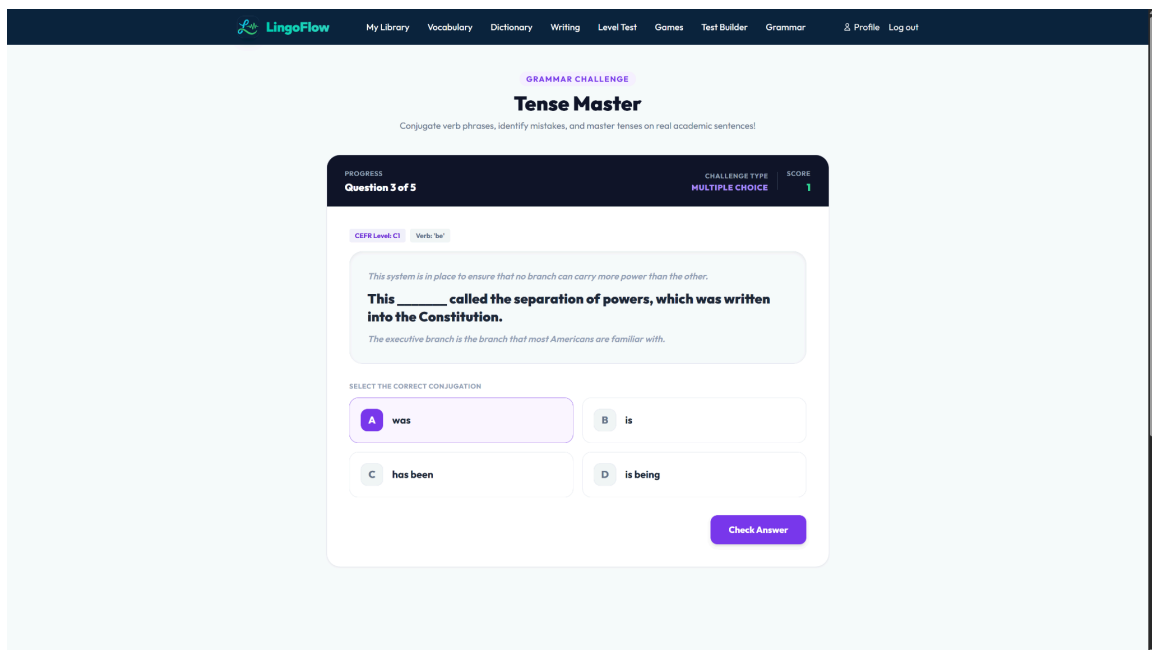


Рисунок 3.17 – Інтерфейс гри Tense Master

Synonym Odyssey (гра на синоніми)

Гра спрямована на розвиток словникового запасу та семантичних асоціацій. Система формує завдання на основі базового слова та набору правильних і хибних варіантів відповідей.

Користувач повинен визначити правильні синоніми, взаємодіючи з інтерактивними картками слів.

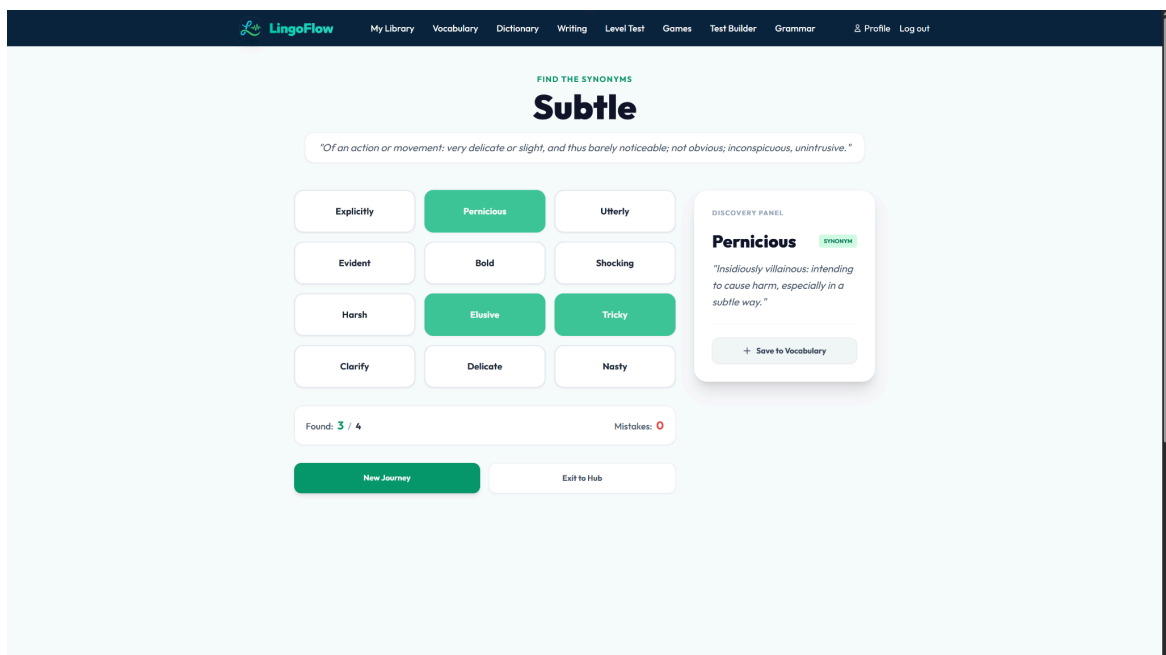


Рисунок 3.18 – Інтерфейс гри Synonym Odyssey

Spell Game (гра на швидкість)

Гра реалізує таймерний режим, у якому користувач формує англійські слова з набору літер за обмежений час. Генерація ігрового поля виконується на сервері, а перевірка слів здійснюється через валідацію наявності букв та перевірку словникової коректності.

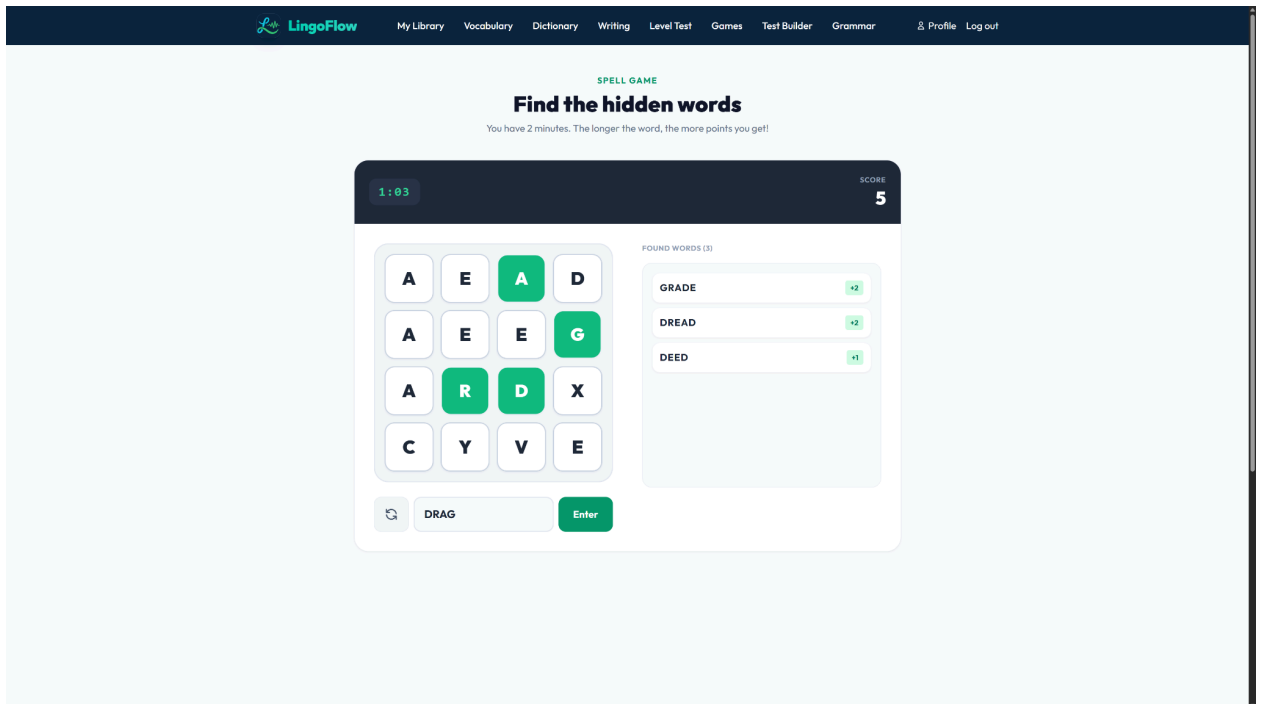


Рисунок 3.19 – Інтерфейс гри Spell Game

Context Detective (контекстний детектив)

Окремий режим тестування дозволяє користувачеві виконувати завдання на заповнення пропусків у тексті з метою перевірки розуміння контексту та семантичних зв'язків між словами.

Система автоматично генерує завдання на основі випадкових абзаців із текстового корпусу Brown, у яких частина слів замінюється пропусками. Вибір слів для пропусків виконується на сервері з урахуванням їх частиномовної приналежності та частотності, а також забезпечується формування правдоподібних варіантів відповідей.

Користувач повинен обрати правильні слова для кожного пропуску, спираючись на зміст тексту та контекст речення. Перевірка відповідей виконується автоматично на сервері з подальшим збереженням результатів.

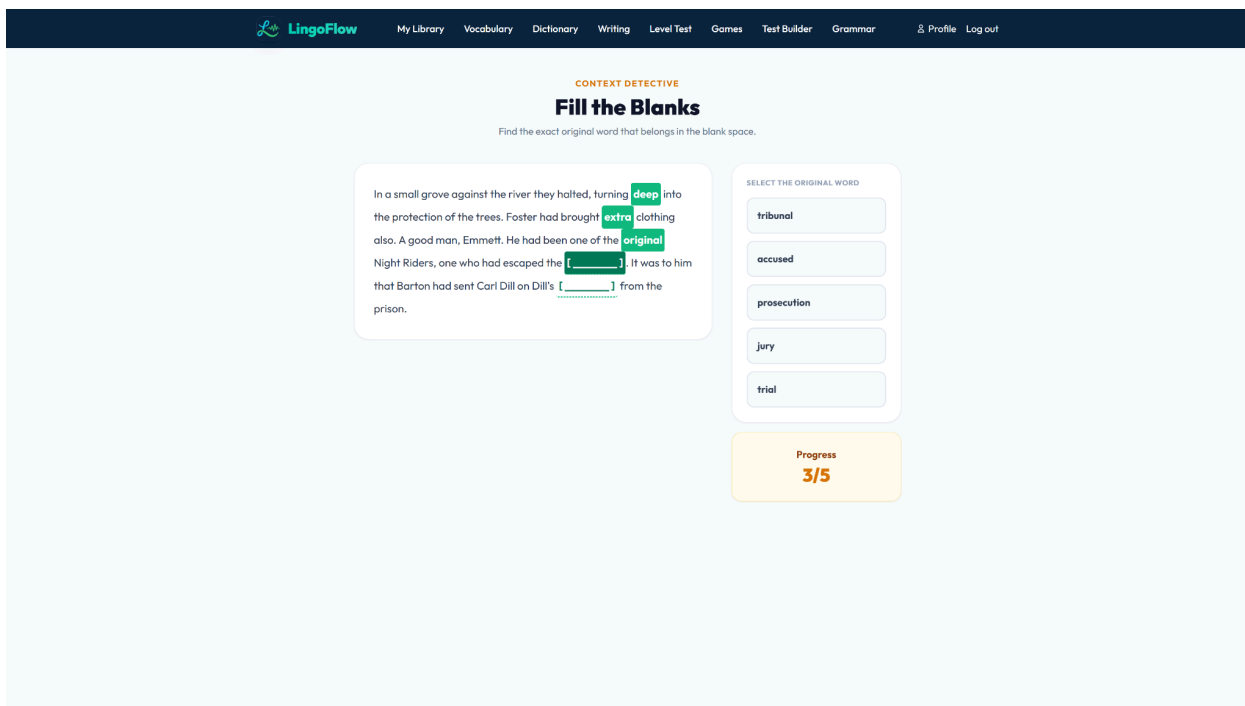


Рисунок 3.20 – Інтерфейс Context Detective

Реалізація конструктора користувацьких тестів

Конструктор користувацьких тестів є інтерактивним модулем, що дозволяє створювати персоналізовані навчальні тести на основі будь-якого тексту, зокрема у форматі Cloze Test для тренування лексичних і граматичних навичок.

Система підтримує кілька режимів генерації завдань, включаючи автоматичне та ручне приховування слів, а також граматичні режими (прийменники, модальні дієслова, словотворення та пропущені речення).

Формування тесту виконується автоматично: текст аналізується, розбивається на токени та обробляється лінгвістично, після чого визначаються елементи для приховування та формуються варіанти відповідей із правильними та дистракторами.

Інтерфейс дозволяє візуально редагувати тест, переглядати його в реальному часі та проходити у інтерактивному режимі. Після завершення система автоматично перевіряє відповіді, відображає результат і зберігає дані для аналізу прогресу користувача.

Рисунок 3.21 – Інтерфейс створення користувацького тесту

Рисунок 3.22 – Інтерфейс проходження користувацького тесту

Реалізація перевірки письма

Підсистема перевірки письма дозволяє користувачеві завантажувати або вводити власні тексти англійською мовою для автоматичного аналізу.

Після надсилання роботи система виконує оцінювання тексту та формує звіт з результатами перевірки.

У результатах відображається прогнозований рівень володіння мовою за шкалою CEFR, загальна оцінка роботи, а також рекомендації щодо покращення граматики, словникового запасу та структури тексту.

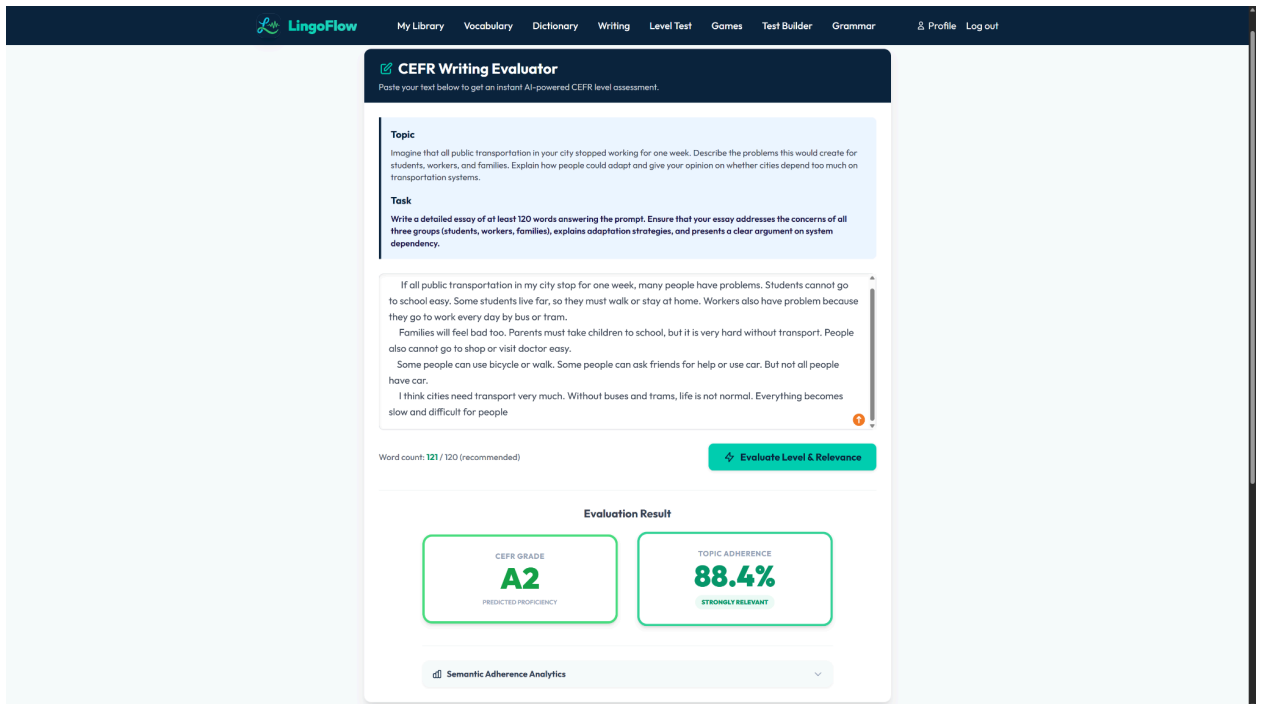


Рисунок 3.23 – Інтерфейс перевірки письмової роботи та відображення результатів аналізу

3.5 Розгортання системи у хмарному середовищі AWS

Загальна характеристика середовища розгортання

Розгортання програмної системи виконано у хмарному середовищі Amazon Web Services (AWS), яке забезпечує масштабованість, надійність та гнучкість інфраструктури. Використання хмарної платформи дозволяє уникнути залежності від фізичних серверів і забезпечує автоматичне масштабування залежно від навантаження.

Керування інфраструктурою реалізовано за підходом Infrastructure as Code (IaC) з використанням Terraform, що забезпечує відтворюваність конфігурації та автоматизацію розгортання.

Архітектура побудована за багаторівневим (multi-tier) принципом із

розділенням на обчислювальний рівень, зберігання даних, кешування та серверлес-компоненти, що підвищує ізоляцію модулів і відмовостійкість системи.

AWS обрано як основну платформу завдяки широкому набору керованих сервісів, підтримці контейнеризації та інтеграції з CI/CD процесами.

Реалізація інфраструктури як коду (Infrastructure as Code)

Для розгортання програмної системи у хмарному середовищі AWS використовується підхід Infrastructure as Code (IaC). Це дозволяє описувати всю інфраструктуру у вигляді конфігураційних файлів та автоматизувати процес створення, налаштування й оновлення хмарних ресурсів без ручного втручання.

У межах проєкту для реалізації IaC застосовується інструмент Terraform, який забезпечує відтворюваність інфраструктури та однакову конфігурацію для різних середовищ (розробка, тестування та продакшн). Завдяки цьому усі ресурси створюються автоматично за єдиним описом, що зменшує ймовірність помилок та спрощує підтримку системи.

Інфраструктура організована за модульним принципом. Кожен логічний компонент системи винесений в окремий модуль, що дозволяє повторно використовувати конфігурації та спрощує їх масштабування й супровід.

Загальна структура Terraform-проєкту представлена нижче:

```
terraform/
├── main.tf           # Головний файл конфігурації (Root Module)
├── providers.tf      # Оголошення та налаштування AWS-провайдера
├── variables.tf      # Опис вхідних змінних проєкту
├── outputs.tf        # Вихідні параметри інфраструктури
├── terraform.tfvars  # Значення змінних для конкретного середовища
└── modules/
    ├── vpc/          # Мережа, підмережі та security groups
    ├── s3/           # Об'єктне сховище для файлів та медіа
    ├── rds/          # Реляційна база даних PostgreSQL
    ├── redis/        # Кеш-система Redis
    ├── lambda/       # Серверless-функції (TTS сервіс)
    └── ecs/          # Оркестрація контейнерів (Fargate, ALB, ECR)
```

Такий підхід дозволяє автоматично розгорнути всю інфраструктуру однією командою, забезпечуючи повну ідентичність середовищ та швидке відновлення системи у разі необхідності.

Розгортання серверної частини (контейнеризація та ECS/Fargate)

Серверна частина системи реалізована у вигляді контейнеризованого веб-застосунку, що забезпечує уніфіковане середовище виконання на всіх етапах життєвого циклу програмного продукту. Використання контейнеризації дозволяє усунути залежність від локального середовища розробки та гарантує ідентичну поведінку системи при розгортанні у хмарній інфраструктурі.

Основою серверної частини є веб-застосунок, реалізований на базі Django, який виконується всередині Docker-контейнера. Контейнер включає всі необхідні залежності, бібліотеки та конфігурації, що забезпечує повну автономність середовища виконання.

Контейнеризація застосунку

Процес створення контейнера визначається за допомогою Dockerfile, який формалізує послідовність підготовки середовища, встановлення залежностей та запуску застосунку.

```
FROM python:3.10-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    libsndfile1 \
    build-essential \
    curl \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /app/

RUN pip install --no-cache-dir --upgrade pip

RUN pip install --no-cache-dir torch==2.5.1 --extra-index-url
```

```

https://download.pytorch.org/whl/cpu --default-timeout=1000

RUN pip install --no-cache-dir --default-timeout=1000 -r requirements.txt

RUN python -m spacy download en_core_web_sm

RUN python -m nltk.downloader brown words punkt punkt_tab
averaged_perceptron_tagger averaged_perceptron_tagger_eng

RUN python -c "from sentence_transformers import SentenceTransformer;
SentenceTransformer('all-MiniLM-L6-v2')"

COPY . /app/

RUN chmod +x /app/entrypoint.sh

EXPOSE 8000

ENTRYPOINT ["/app/entrypoint.sh"]

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]

```

Лістинг 3.2 – Dockerfile серверної частини системи

Dockerfile визначає базовий образ Python, встановлення системних залежностей, бібліотек обробки природної мови та машинного навчання, а також підготовку середовища для запуску веб-сервера. У результаті формується ізольований контейнер, що містить повністю готове до виконання середовище застосунку.

Після побудови Docker-образ завантажується до репозиторію Amazon ECR та використовується для створення контейнерів у середовищі Amazon ECS Fargate.

Для запуску застосунку використовується ECS Service, який автоматично підтримує необхідну кількість контейнерів у працездатному стані та забезпечує їх відновлення у випадку збою.

The screenshot displays the 'Service overview' page in the AWS Management Console. At the top, it shows the service status as 'Active', a progress bar for 'Tasks (1 Desired)' with '0 Pending' and '1 Running', the 'Task definition: revision' as 'lingoflow-family:?', and the 'Deployment status' as 'Success'. Below this is a navigation bar with tabs for 'Health and metrics', 'Tasks', 'Logs', 'Deployments', 'Events', 'Configuration and networking', 'Service auto scaling', 'Event history', and 'Tags'. The 'Status' section provides details: 'Service name' is 'lingoflow-family-service-qd3vj4qc', 'Service ARN' is 'arn:aws:ecs:eu-north-1:354102625770:service/lingoflow-cluster/lingoflow-family-service-qd3vj4qc', 'Deployments current state' is '1 Completed task', and 'Created at' is 'May 22, 2026, 17:35 (UTC+3:00)'. The 'Health check grace period' is '0 seconds'. A section for 'Load balancer target health' shows a table with columns: 'Container name:port', 'Target group', 'Resource type', 'Target health', and 'Load balancer'. The table contains one entry: 'web:8000' with target group 'lingoflow-tg', resource type 'Customer managed', target health '1 Healthy 0 Unhealthy', and load balancer 'lingoflow-alb'.

Рисунок 3.24 – Розгорнутий сервіс застосунку в Amazon ECS

Процес оновлення застосунку полягає у створенні нової версії Docker-образу та її повторному розгортанні через ECS. Під час оновлення сервіс автоматично замінює старі контейнери новими, що дозволяє мінімізувати простої системи та забезпечити безперервність роботи.

Реалізація бази даних та кешування

Для зберігання даних платформи LingoFlow використовується реляційна система керування базами даних PostgreSQL, розгорнута за допомогою сервісу Amazon RDS. У базі даних зберігаються облікові записи користувачів, словникові картки, результати тестувань, статистика навчання, словник Wiktionary, дані бібліотеки EPUB та інша службова інформація системи.

Використання Amazon RDS дозволяє автоматизувати резервне копіювання, оновлення програмного забезпечення бази даних та моніторинг її стану без необхідності адміністрування серверів вручну.

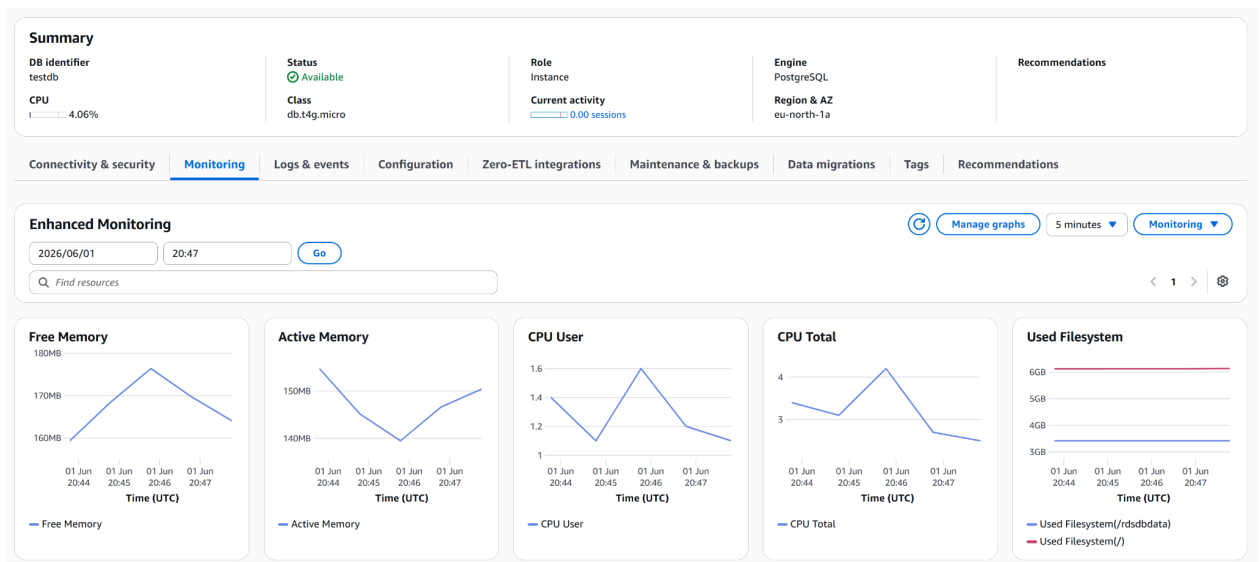


Рисунок 3.25 – Екземпляр бази даних PostgreSQL в Amazon RDS

Для підвищення продуктивності системи використовується Redis, розгорнутий за допомогою сервісу Amazon ElastiCache. Redis застосовується

для кешування часто використовуваних даних, зберігання сесій користувачів та тимчасових результатів обробки запитів.

Використання кешування дозволяє зменшити кількість звернень до бази даних, скоротити час відповіді сервера та покращити швидкодію платформи під час одночасної роботи великої кількості користувачів.

	Cache name	Status	Description	Engine version	Configuration
	lingoflow-redis	Available	-	7.1	Serverless

Рисунок 3.26 – Кластер кешування Redis в Amazon ElastiCache

Підключення Django-додатку до PostgreSQL та Redis здійснюється через змінні середовища, що передаються контейнерам ECS під час запуску. Такий підхід забезпечує безпечне зберігання конфігураційних параметрів та спрощує розгортання системи в різних середовищах.

Реалізація сховища даних та медіафайлів (Amazon S3)

Для зберігання файлів та медіаресурсів системи використовується сервіс Amazon Simple Storage Service (Amazon S3). Використання об'єктного сховища дозволяє відокремити файлові дані від серверної частини застосунку та забезпечити їх надійне довготривале зберігання.

У сховищі розміщуються завантажені користувачами EPUB-книги, статичні файли застосунку, зображення, а також аудіофайли, що генеруються підсистемою синтезу мовлення. Централізоване зберігання даних спрощує доступ до файлів із різних компонентів системи та забезпечує їх доступність незалежно від стану окремих контейнерів застосунку.

Для взаємодії із сервісом Amazon S3 серверна частина використовує офіційний AWS SDK. Завантаження файлів здійснюється безпосередньо зі застосунку, після чого у базі даних зберігаються посилання на відповідні об'єкти у сховищі.

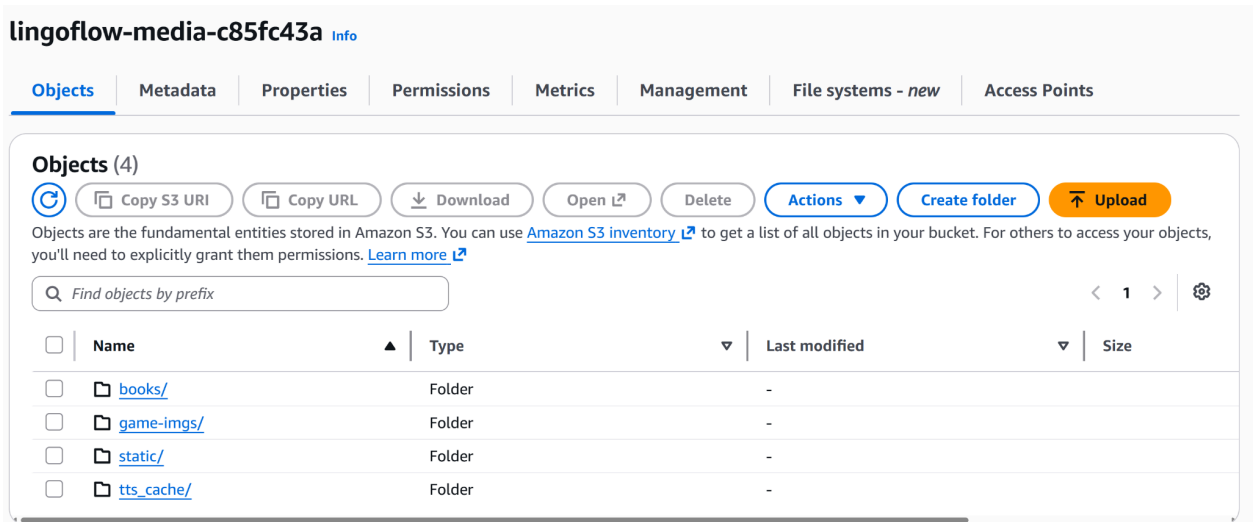


Рисунок 3.27 – Сховище файлів Amazon S3

Доступ до об'єктів контролюється за допомогою механізму IAM-політик та налаштувань доступу до кошика (Bucket Policy). Такий підхід дозволяє обмежити доступ до файлів лише авторизованим компонентам системи та підвищує загальний рівень безпеки.

Завдяки використанню Amazon S3 забезпечується висока надійність зберігання даних, автоматичне масштабування сховища та можливість обробки великої кількості файлів без необхідності розширення серверної інфраструктури.

Реалізація serverless-компонентів системи (AWS Lambda)

Для виконання ресурсоємних допоміжних операцій у системі використовується підхід Serverless Computing, реалізований за допомогою сервісу AWS Lambda. Це дозволяє виконувати окремі обчислювальні задачі без необхідності постійного запуску додаткових серверів або контейнерів.

У розробленій системі AWS Lambda використовується для реалізації підсистеми синтезу мовлення (Text-to-Speech, TTS). Під час запиту користувача на озвучування слова або текстового фрагмента серверна частина передає необхідні дані до Lambda-функції, яка виконує генерацію аудіофайлу та повертає результат застосунку.

Для забезпечення роботи бібліотек синтезу мовлення функція розгортається у вигляді контейнерного образу, що дозволяє використовувати власне програмне середовище та необхідні залежності. Контейнерний образ зберігається у репозиторії Amazon ECR та автоматично завантажується під час запуску функції.

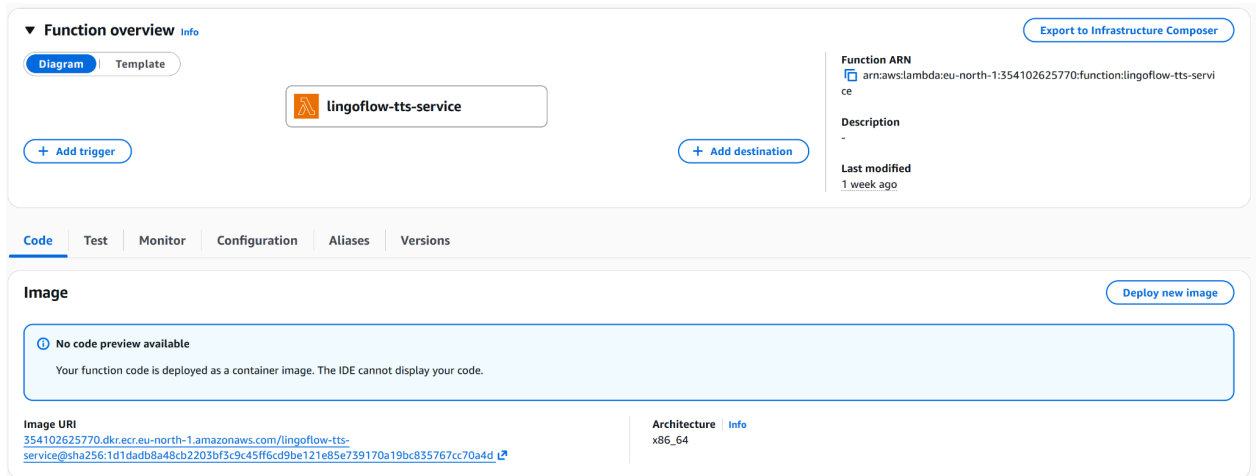


Рисунок 3.28 – AWS Lambda-функція підсистеми синтезу мовлення

Використання серверлес-архітектури дозволяє запускати обчислення лише за потреби, що зменшує витрати на інфраструктуру та спрощує масштабування сервісу. Крім того, AWS Lambda автоматично забезпечує розподіл навантаження між одночасними викликами функції без необхідності додаткового адміністрування.

Логування роботи функції та контроль її стану здійснюються за допомогою сервісу Amazon CloudWatch, що дозволяє відстежувати помилки виконання, тривалість обробки запитів та інші показники продуктивності.

Таким чином, використання AWS Lambda дозволило винести підсистему синтезу мовлення в окремий незалежний компонент, зменшити навантаження на основний вебзастосунок та забезпечити ефективне використання обчислювальних ресурсів.

Реалізація процесу CI/CD

Для автоматизації процесів тестування, збірки та розгортання програмного продукту реалізовано конвеєр безперервної інтеграції та

доставки (CI/CD) на базі GitHub Actions. Автоматизація дозволяє мінімізувати ручні операції під час оновлення системи та забезпечує швидке розгортання нових версій у хмарному середовищі AWS.

Процес CI/CD запускається автоматично після надсилання змін до основної гілки репозиторію (main).

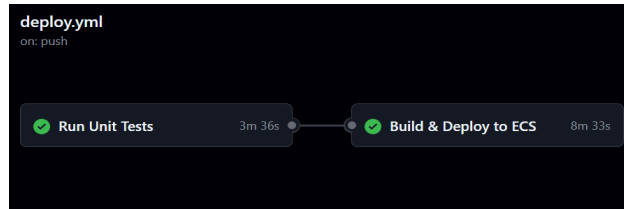


Рисунок 3.29 – Workflow автоматичного розгортання в GitHub Actions

Конвеєр складається з двох основних етапів:

1. Перевірка програмного коду (Test Stage)

На першому етапі GitHub Actions автоматично створює тимчасове середовище виконання, встановлює залежності проєкту та запускає набір модульних тестів Django.

У разі виявлення помилок процес розгортання автоматично припиняється, що запобігає потраплянню нестабільного коду до виробничого середовища.

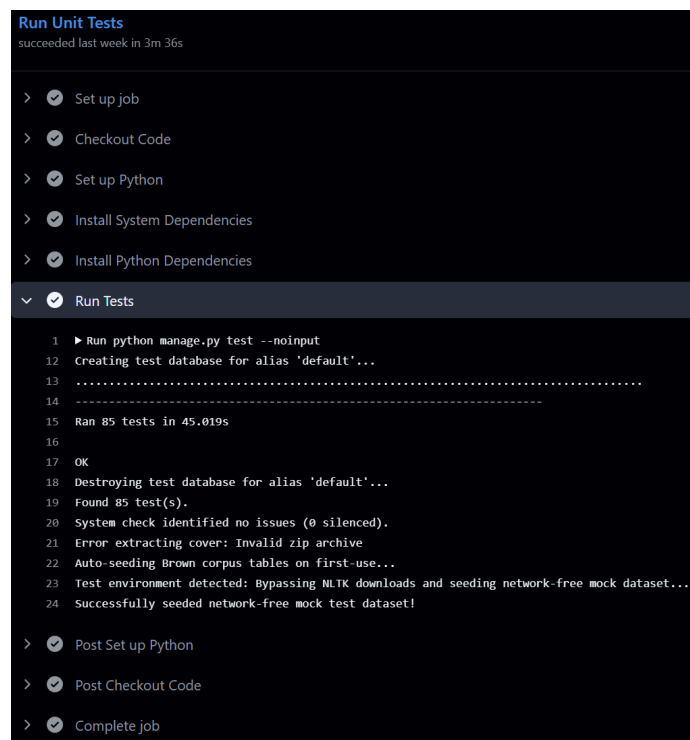


Рисунок 3.30 – Успішне виконання етапу тестування

2. Автоматичне розгортання (Deployment Stage)

Після успішного проходження тестів запускається процес розгортання нової версії системи.

На цьому етапі GitHub Actions:

- підключається до AWS за допомогою IAM-облікових даних;
- збирає новий Docker-образ застосунку;
- завантажує образ до контейнерного реєстру Amazon ECR;
- отримує поточну ECS Task Definition;
- оновлює конфігурацію контейнера новим тегом образу;
- виконує оновлення ECS Service у кластері Amazon ECS.

Під час оновлення сервісу ECS автоматично створює нові контейнери та поступово замінює попередні екземпляри без зупинки роботи системи.

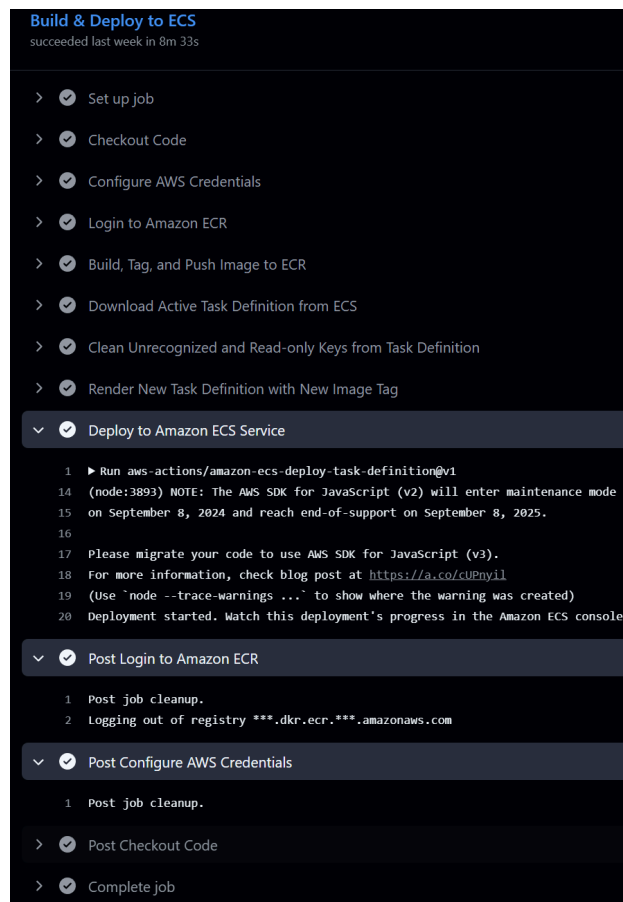


Рисунок 3.31 – Процес оновлення сервісу в Amazon ECS

Для зберігання конфіденційних параметрів використовуються GitHub Secrets. У них розміщуються ключі доступу до AWS, параметри підключення до бази даних та інші службові налаштування, необхідні під час розгортання.

Завдяки використанню GitHub Actions процес публікації нових версій системи повністю автоматизований і виконується без необхідності ручного втручання адміністратора.

Реалізація безпеки та контролю доступу

Для забезпечення безпеки системи реалізовано комплекс заходів на мережевому та інфраструктурному рівнях. Ізоляція компонентів забезпечується за допомогою AWS Security Groups, які дозволяють зовнішній доступ лише до балансувальника навантаження. Сервери застосунку, PostgreSQL та Redis розміщені у приватному сегменті мережі та недоступні для прямого підключення з Інтернету.

Контроль доступу до хмарних ресурсів реалізовано через IAM-ролі та політики AWS відповідно до принципу найменших привілеїв. Конфіденційні параметри, зокрема ключі доступу та налаштування підключення до сервісів, передаються через змінні середовища та не зберігаються у вихідному коді.

Захищений обмін даними між користувачем і системою забезпечується використанням протоколу HTTPS. Доменне ім'я lingoflow.cloud прив'язане до інфраструктури AWS, а SSL/TLS сертифікат налаштований за допомогою AWS Certificate Manager, що гарантує шифрування мережевого трафіку та захист переданих даних.

Certificate status

Identifier
d951ca36-ba16-4551-a43e-0f27d85c7de7

ARN
 arn:aws:acm:eu-north-1

Type
Amazon Issued

Status
 Issued

Domains (2)

Domain	Status	Renewal status	Type
lingoflow.cloud	 Success	-	CNAME
*.lingoflow.cloud	 Success	-	CNAME

Рисунок 3.32 – Налаштування SSL/TLS сертифіката для домену
lingoflow.cloud

Реалізовані механізми безпеки забезпечують захист мережевої інфраструктури, конфіденційність даних користувачів та безпечну роботу системи у хмарному середовищі AWS.

Реалізація моніторингу та логування

Для моніторингу стану системи використовується сервіс Amazon CloudWatch, який забезпечує централізоване збирання та зберігання логів усіх основних компонентів платформи.

Логи вебзастосунку автоматично надходять із контейнерів Amazon ECS до CloudWatch Logs, де використовуються для аналізу помилок, контролю API та відстеження стану системи. Аналогічно здійснюється логування AWS Lambda-функцій, що дозволяє контролювати виконання серверлес-компонентів і виявляти збої під час обробки запитів.

Окрім логів, CloudWatch надає метрики використання ресурсів (CPU, пам'ять, стан контейнерів ECS), що дозволяє оперативно оцінювати продуктивність і стабільність системи.

Висновок до розділу 3

У третьому розділі було здійснено розробку та практичну реалізацію програмного продукту. Детально описано ключові функціональні модулі системи, зокрема інтерфейс читання EPUB-книг, глобальний словник, персональний словник із механізмом інтервальних повторень, ігровий навчальний модуль, а також конструктор користувацьких тестів.

Окрему увагу приділено реалізації серверної частини, яка забезпечує обробку навчальних даних, роботу з мовними корпусами, генерацію завдань та взаємодію з клієнтським інтерфейсом через API. Розглянуто принципи побудови ігрових механік, що базуються на NLP-обробці текстів, а також підхід до персоналізації навчання.

Також у розділі наведено опис хмарної інфраструктури системи, яка реалізована на базі AWS. Розглянуто використання Infrastructure as Code (Terraform) для автоматизованого розгортання ресурсів, контейнеризацію застосунку за допомогою Docker, а також використання Amazon ECS для оркестрації контейнерів. Окремо описано серверлес-компоненти на базі AWS Lambda, систему зберігання даних у Amazon RDS, Redis та S3, а також процес CI/CD на основі GitHub Actions. Крім того, розглянуто питання безпеки, моніторингу та логування, що забезпечують стабільну та захищену роботу системи в хмарному середовищі.

У результаті виконано повний цикл реалізації програмного продукту, від розробки функціональних модулів до побудови масштабованої хмарної інфраструктури та автоматизації процесів розгортання.

РОЗДІЛ 4. ТЕСТУВАННЯ ТА ОЦІНЮВАННЯ ЕФЕКТИВНОСТІ СИСТЕМИ

4.1 Загальна характеристика готового програмного продукту перед тестуванням

Розроблений програмний продукт є веборієнтованою навчальною платформою для вивчення англійської мови, яка об'єднує в собі модулі читання електронних книг, персонального словника, інтервальних повторень, навчальних ігор та конструктора тестів. Система реалізована у вигляді багатокомпонентного вебзастосунку з клієнт-серверною архітектурою та підтримкою хмарної інфраструктури AWS.

Перед початком тестування система перебуває у повністю функціональному стані та розгорнута у середовищі, максимально наближеному до бойового (production-like environment). Усі основні модулі інтегровані між собою та взаємодіють через API, забезпечуючи єдиний навчальний процес користувача.

До основних функціональних компонентів, що підлягають перевірці, належать:

- модуль читання EPUB-книг з підтримкою перекладу та контекстного аналізу слів;
- система персонального словника з інтервальним повторенням (SRS);
- глобальний словник із пошуком та лексичними даними;
- ігровий навчальний модуль для гейміфікації навчання;
- конструктор користувацьких тестів;
- серверна частина з API та обробкою даних;
- хмарна інфраструктура AWS (ECS, RDS, Redis, S3, Lambda).

4.2 Підхід до тестування та тестове середовище

Тестування розробленого програмного продукту здійснюється як комплексний процес перевірки функціональності, інтеграції та стабільності всіх основних модулів системи. Основний підхід базується на поєднанні

функціонального, інтеграційного та модульного тестування, а також на використанні автоматизованих тестів, що виконуються в межах CI/CD процесу.

Загальна структура тестового покриття системи включає різні типи перевірок, що забезпечують контроль якості на всіх рівнях архітектури.

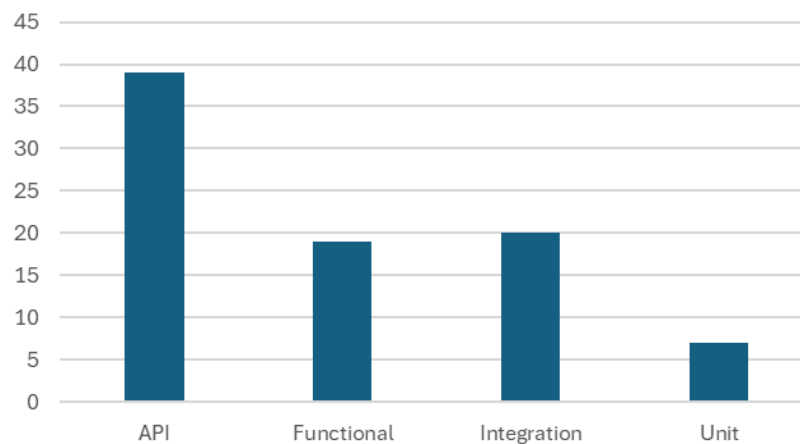


Рисунок 4.1 – Розподіл тестів за типами

Функціональне тестування спрямоване на перевірку коректності роботи окремих модулів системи, включаючи словник, систему інтервальних повторень, ігрові механіки, EPUB-читач та конструктор тестів. Інтеграційне тестування використовується для перевірки взаємодії між клієнтською частиною, серверним API, базою даних та зовнішніми сервісами, такими як Redis, S3 та AWS Lambda.

Особлива увага приділяється тестуванню системи в умовах, максимально наближених до реального використання. Для цього використовується контейнеризоване середовище на базі Docker, яке забезпечує однакову поведінку системи як у локальному середовищі розробки, так і в хмарному середовищі розгортання.

Тестове середовище включає:

- локальне середовище розробки (Docker Compose);
- CI/CD середовище на базі GitHub Actions для автоматичного запуску тестів;

- хмарне середовище AWS (ECS Fargate) для перевірки роботи системи в production умовах;
- тестову базу даних PostgreSQL, кеш Redis та об'єктне сховище S3.

Окремо використовується автоматизований процес тестування, який запускається при кожному оновленні коду в основній гілці репозиторію. Це дозволяє виявляти помилки на ранніх етапах та гарантує стабільність системи перед розгортанням.

4.3 Функціональне тестування системи

Функціональне тестування системи спрямоване на перевірку коректності реалізації бізнес-логіки основних модулів програмного продукту. У межах тестування було перевірено роботу системи автентифікації, управління словником, інтервального повторення, ігрових механік, а також базових функцій ядра платформи.

Тестові сценарії були сформовані на основі реальних сценаріїв використання системи та покривають як позитивні, так і негативні випадки роботи застосунку. Особлива увага приділялась перевірці обробки некоректних даних, захисту від несанкціонованого доступу та коректності математичних моделей навчання.

Результати функціонального тестування згруповано за модулями системи та наведено у відповідних таблицях:

Таблиця 4.1

Функціональне тестування модуля accounts.

№	Тестовий сценарій	Вхідні дані	Очікуваний результат
1	Авторизація користувача через username	Логін: auth_user, пароль:password	Успішна автентифікація через username
2	Авторизація через email без урахування регістру	Email: auth_user@gmail.com	Успішний вхід незалежно від регістру email

3	Спроба входу з неправильним паролем	Пароль: wrongpassword	Відмова в авторизації
4	Спроба входу неіснуючого користувача	Логін: ghost	Відмова в авторизації

Таблиця 4.2

Функціональне тестування модуля library

№	Тестовий сценарій	Вхідні дані	Очікуваний результат
1	Логування часу читання	seconds = 30	Успішне збереження часу читання
2	Передача некоректного часу читання	seconds = -5 або 120	HTTP 400 Bad Request

Таблиця 4.3

Функціональне тестування модуля vocabulary

№	Тестовий сценарій	Вхідні дані	Очікуваний результат
1	Розрахунок прогресу навчального набору	2 слова з різним статусом повторення	Коректний progress_percent та due_count
2	Перше успішне повторення слова (SM-2)	quality = 4	repetitions = 1, interval = 1
3	Друге успішне повторення слова (SM-2)	quality = 5	repetitions = 2, interval = 6
4	Третє та наступні повторення	quality = 5	repetitions = 3, interval = 15
5	Помилка при повторенні слова	quality < 3	Скидання repetitions та interval
6	Дострокове повторення слова з помилкою	word review раніше терміну	Повне скидання прогресу

Таблиця 4.4

Функціональне тестування модуля games

№	Тестовий сценарій	Вхідні дані	Очікуваний результат
1	Перевірка правильного слова у Spell Game	word = "test"	valid = true, нарахування балів
2	Перевірка надто короткого слова	word = "te"	valid = false
3	Перевірка слова з відсутніми літерами на полі	word = " "	valid = false
4	Генерація граматичних питань	level = a1	HTTP 200, повернення питань
5	Некоректний рівень складності	level = invalid	HTTP 400

Таблиця 4.5

Функціональне тестування модуля core

№	Тестовий сценарій	Вхідні дані	Очікуваний результат
1	Збереження результату тесту рівня	level = B2	Оновлення рівня користувача
2	Спроба запису некоректного рівня	level = X1	HTTP 400, відхилення

4.4 Інтеграційне тестування системи

Інтеграційне тестування програмної системи LingoFlow спрямоване на перевірку взаємодії між основними модулями платформи та коректності обміну даними між серверною частиною, базою даних, внутрішніми сервісами та зовнішніми компонентами.

Основною метою даного етапу є перевірка узгодженої роботи підсистем у складних сценаріях, коли кілька компонентів взаємодіють між собою в межах одного бізнес-процесу.

У межах проєкту реалізовано 20 інтеграційних тестів, які охоплюють ключові сценарії взаємодії між модулями системи, включаючи роботу з користувачами, бібліотекою, словником, граматикою, ігровими механіками, а також зовнішніми сервісами (Redis, S3, AWS Lambda та AI/NLP компоненти).

Для ілюстрації основних сценаріїв інтеграції наведено узагальнену таблицю 4.6, яка містить 8 найбільш критичних інтеграційних тестів, що демонструють різні типи взаємодій у системі - від роботи з базою даних до взаємодії з хмарною інфраструктурою та AI-модулями.

Таблиця 4.6

Інтеграційне тестування системи

№	Сценарій перевірки	Очікуваний результат
1	Створення User → сигнал post_save → trigger signal handler → створення профілю	Наявність UserProfile (OneToOne). Перевірка assertTrue/assertEqual базових параметрів
2	Завантаження .epub → extract_epub_cover → помилка zipfile → exception handler	Відсутність крашу сервера. Повернення (None, None) / assertIsNone
3	GET /read/<id>/ → створення UserBookSetting; POST highlights → збереження CFI	200/302. Створення записів у БД. Коректне збереження та відновлення CFI
4	/tts/stream/ → Redis cache check → S3 fallback → Lambda або local fallback	200 OK, audio/wav. Запис у Redis (cache.set) та S3 (save)
5	AJAX /vocabulary/save/ зі сторінки читання → створення VocabularyWord	200 OK. Об'єкт створено, book_context прив'язаний до Book
6	submit_word_review (quality 0–5) → оновлення SM-2 параметрів	Оновлення repetitions, interval, easiness, next_review. При помилці - reset
7	GET /grammar/unit/<id>/ → витяг теорії + quiz → JSON serialization → render	200 OK. HTML містить тему + валідний JSON питань

8	Synonym / Cloze / Writing AI pipelines → embeddings + tokenization + evaluation	200/302. Коректні токени, selectable:false, логування у GameLog
---	---	---

4.5 API тестування системи

API тестування програмної системи спрямоване на перевірку коректності роботи HTTP-інтерфейсів, що забезпечують взаємодію між клієнтською та серверною частинами системи. Основна увага приділяється валідації REST-ендпоінтів, механізмів авторизації, обробки JSON-запитів, рендерингу сторінок та контролю доступу до функціоналу системи.

У межах проекту реалізовано 38 API тестів, які охоплюють усі ключові модулі платформи, включаючи accounts, vocabulary, grammar, games, library, writing та core сервіси.

Розподіл API тестів за модулями представлено в узагальненій таблиці нижче.

Таблиця 4.7

Узагальнена характеристика API тестування системи

Модуль	Кількість тестів	Основний тип перевірок	Основний фокус
accounts	3	HTTP, Access Control	Авторизація та профіль користувача
library	9	API, AJAX, Cache, Cloud	Робота бібліотеки, TTS, Redis, S3
vocabulary	8	CRUD, API, NLP	Словник, SM-2, пошук, навчальні набори
grammar	8	Admin API, UI rendering	Керування граматикою та контентом
games	6	API + AI/NLP	Ігри, генерація контенту, builder
writing	4	API + ML validation	Оцінка есе та перевірка тексту
core	7	Pages + Health + Auth	Базові сторінки та системні сервіси

Усі з перелічених тестів автоматично виконуються в межах CI/CD-конвеєра при кожній зміні програмного коду. Це забезпечує раннє виявлення помилок, контроль стабільності системи та автоматичну перевірку коректності роботи системи перед розгортанням нової версії програмного продукту.

4.6 Безпека та стабільність системи

У процесі розробки програмної системи особливу увагу приділено питанням безпеки та стабільності роботи платформи. Реалізовані механізми забезпечують захист користувацьких даних, контроль доступу до функціоналу, а також стійкість системи до навантажень і відмов зовнішніх сервісів.

Безпека системи

Безпека програмної системи LingoFlow реалізована на рівні автентифікації, авторизації, обробки даних та інфраструктурного розгортання. Система використовує рольову модель доступу, яка розмежовує права між анонімними користувачами, студентами та адміністраторами, забезпечуючи контроль доступу до функціональних можливостей платформи.

Для захисту від несанкціонованих дій реалізовано перевірку прав доступу до API-ендпоінтів, валідацію вхідних даних та контроль допустимих HTTP-методів. Це дозволяє запобігати виконанню некоректних операцій і зменшує ризик порушення цілісності даних.

Під час проєктування та розгортання системи дотримувалися сучасних практик безпеки, зокрема:

- принципу найменших привілеїв (Least Privilege Principle) для користувачів і сервісів;
- використання механізмів автентифікації та контролю доступу Django;
- захисту POST-запитів за допомогою CSRF-механізмів;
- валідації та фільтрації користувацьких даних;
- використання HTTPS-з'єднань та захищених cookie-файлів;

- обмеження частоти запитів (Rate Limiting) для захисту від brute-force та DDoS-атак на ресурсоемні API-ендпоінти;
- ізоляції зовнішніх сервісів AWS через окремі ролі доступу та політики безпеки IAM;
- ведення журналів подій та моніторингу роботи системи.

Стабільність системи

Для оцінки стабільності платформи було проведено навантажувальне тестування за допомогою інструмента Locust. Тестування виконувалося у локальному середовищі розробки та імітувало типові дії користувачів: реєстрацію, авторизацію, завантаження книг, читання EPUB-файлів, роботу зі словником, граматичними матеріалами та навчальними іграми.

На рисунку 4.2 наведено результати навантажувального тестування системи. У процесі випробувань навантаження поступово збільшувалося до 250 одночасних віртуальних користувачів, при цьому система забезпечувала обробку близько 142 запитів за секунду (RPS). Медіанний час відповіді становив близько 20 мс, а 95 % усіх запитів виконувалися менш ніж за 160 мс, що забезпечує комфортну взаємодію користувача із системою.

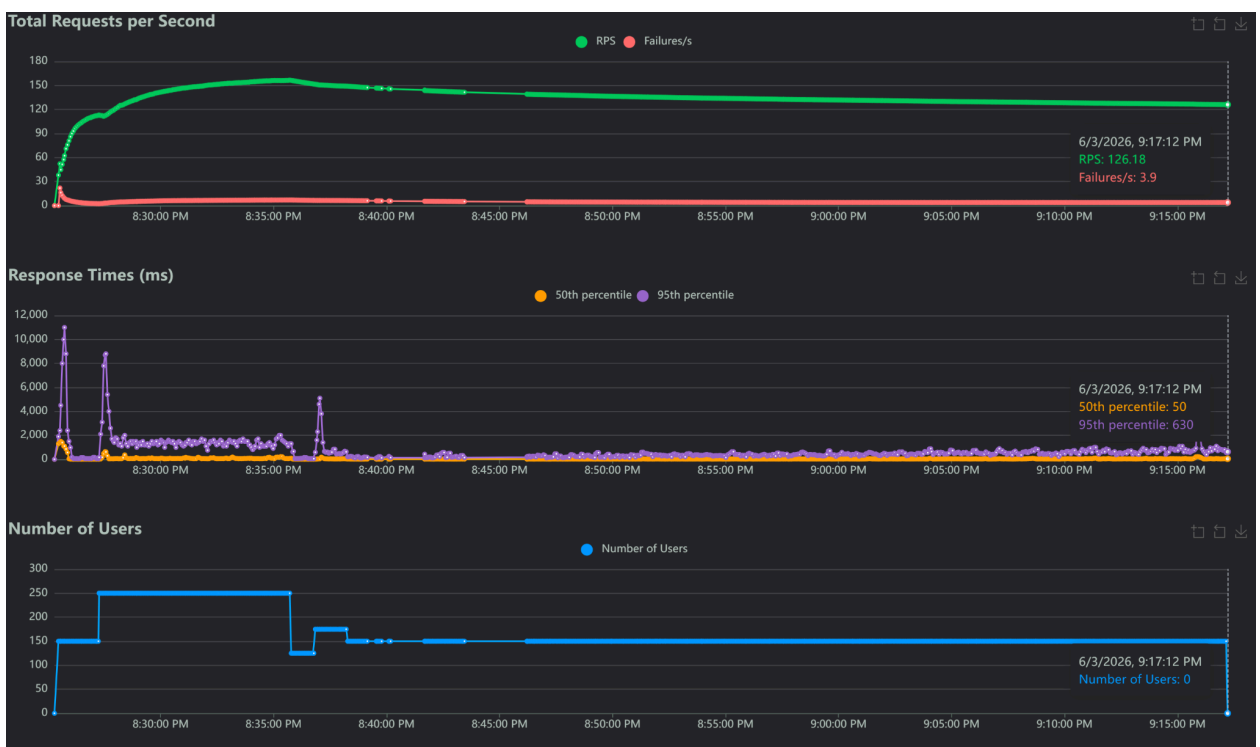


Рисунок 4.2 – Результати навантажувального тестування платформи LingoFlow за допомогою Locust

Під час тестування було зафіксовано незначну кількість помилок (близько 3,6 % від загальної кількості запитів). Їх виникнення пов'язане з обмеженнями СКБД SQLite, яка використовувалася лише в локальному середовищі для розробки та має обмежену підтримку паралельних операцій запису. У продуктивному середовищі система використовує PostgreSQL та Redis, що дозволяє усунути подібні вузькі місця та додатково підвищити продуктивність.

Отримані результати свідчать про достатній запас продуктивності та стійкість архітектури системи. З урахуванням того, що віртуальні користувачі Locust виконують дії значно частіше за реальних користувачів, отримані показники дозволяють прогнозувати підтримку приблизно 1500–2200 одночасно активних користувачів та 15–20 тисяч унікальних відвідувачів на добу без суттєвого погіршення швидкодії сервісу.

Висновок до розділу 4

У ході тестування програмної системи було перевірено її функціональність, інтеграційну взаємодію компонентів, API-інтерфейси, а також рівень безпеки та стабільності. Отримані результати підтверджують коректність реалізації основних модулів системи та відповідність програмного продукту поставленим вимогам.

Практичне використання системи демонструє її ефективність як освітньої платформи для підготовки до ЗНО/НМТ з англійської мови. Користувач може опрацьовувати тексти у форматі EPUB з контекстним аналізом лексики, формувати та закріплювати словниковий запас за допомогою інтервальних повторень, а також виконувати граматичні та контекстні вправи, сформовані на основі реальних текстів і адаптовані до формату екзаменаційних завдань.

Окрему увагу приділено розвитку навичок письма, де система здійснює оцінювання текстів із врахуванням зв'язності викладу та відповідності змісту заданій темі, що дозволяє контролювати якість письмових робіт у контексті вимог стандартизованого тестування.

Проведення тестування в умовах, наближених до реального використання, дозволило оцінити продуктивність і стабільність системи, а результати навантажувального тестування підтвердили її здатність працювати під високим навантаженням без суттєвого погіршення швидкодії.

У цілому розроблена система є функціонально завершеним та стабільним програмним продуктом, що відповідає сучасним вимогам до веборієнтованих освітніх платформ і має потенціал для подальшого розвитку та масштабування.

ВИСНОВКИ

Результатом виконання кваліфікаційної роботи є розроблена веборієнтована інформаційна система для вивчення англійської мови, яка поєднує інтерактивне навчання, персоналізацію освітнього процесу та методи обробки природної мови і машинного навчання. Основна мета роботи полягала у створенні ефективного програмного засобу для підвищення якості засвоєння лексичного та граматичного матеріалу, автоматизації оцінювання навчальних результатів та підтримки адаптивного навчання користувача.

У першому розділі виконано аналіз предметної області, сучасних підходів до цифрового навчання мов та існуючих програмних рішень. Визначено основні недоліки аналогів та сформульовано вимоги до розроблюваної системи.

У другому розділі здійснено проєктування інформаційної системи, включаючи визначення функціональних і нефункціональних вимог, розробку архітектури, функціональної структури, інформаційної моделі та підходів до розгортання в хмарному середовищі AWS.

У третьому розділі описано процес реалізації програмного продукту, зокрема серверної та клієнтської частин системи, інтеграцію модулів машинного навчання та NLP, а також використані технології, такі як Django, PostgreSQL, SentenceTransformers, spaCy та інші.

У четвертому розділі проведено тестування розробленої системи, перевірено коректність роботи основних функціональних модулів та оцінено ефективність запропонованих алгоритмів у реальних умовах використання.

Отримані результати показують, що розроблена система забезпечує ефективне та адаптивне вивчення англійської мови, підвищує якість засвоєння навчального матеріалу та дозволяє автоматизувати ключові етапи навчального процесу. Впровадження подібних рішень має практичну цінність для цифрових освітніх платформ та систем дистанційного навчання.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Understanding Natural Language Processing Pipeline. TechMags.
URL: <https://tech-mags.com/understanding-natural-language-processing-pipeline/>
2. Visualizing Word Embedding with PCA and t-SNE. Towards Data Science.
URL: <https://towardsdatascience.com/visualizing-word-embedding-with-pca-and-t-sne-961a692509f5/>
3. Semantic similarity between words (corpus linguistics concept). CorpLing Hypotheses.
URL: <https://corpling.hypotheses.org/495>
4. English levels CEFR framework. TrackTest.
URL: <https://tracktest.eu/english-levels-cefr/>
5. Spaced repetition in language learning and forgetting curve. Taalhammer.
URL: <https://www.taalhammer.com/spaced-repetition-in-language-learning/>
6. Word Sense Disambiguation overview and graph representation. Devopedia.
URL: <https://devopedia.org/word-sense-disambiguation>
7. Text classification architecture (ML pipeline illustration). ResearchGate.
URL: https://www.researchgate.net/figure/Text-classification-architecture_fig1_369247298
8. Bird S., Klein E., Loper E. Natural Language Processing with Python. O'Reilly Media.
9. Jurafsky D., Martin J. H. Speech and Language Processing. Pearson.
10. Vaswani A. et al. Attention Is All You Need. NeurIPS, 2017.
URL: <https://arxiv.org/abs/1706.03762>
11. Devlin J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
URL: <https://arxiv.org/abs/1810.04805>
12. Mikolov T. et al. Efficient Estimation of Word Representations in Vector Space (Word2Vec).
URL: <https://arxiv.org/abs/1301.3781>

13. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
URL: <https://arxiv.org/abs/1908.10084>
14. spaCy Documentation. Explosion AI.
URL: <https://spacy.io/>
15. SentenceTransformers Documentation.
URL: <https://www.sbert.net/>
16. XGBoost Documentation.
URL: <https://xgboost.readthedocs.io/>
17. Django Documentation.
URL: <https://docs.djangoproject.com/>
18. PostgreSQL Documentation.
URL: <https://www.postgresql.org/docs/>
19. Redis Documentation.
URL: <https://redis.io/docs/>
20. Amazon Web Services (AWS) Documentation.
URL: <https://docs.aws.amazon.com/>
21. AWS ECS Fargate Overview.
URL:
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html
22. Amazon S3 Documentation.
URL: <https://docs.aws.amazon.com/s3/>
23. CEFR – Common European Framework of Reference for Languages. Council of Europe.
URL:
<https://www.coe.int/en/web/common-european-framework-reference-languages>
24. Ebbinghaus Forgetting Curve and Spaced Repetition Theory.
URL: https://en.wikipedia.org/wiki/Forgetting_curve
25. Machine Learning Basics and Concepts. Google Developers.
URL: <https://developers.google.com/machine-learning>